Formal Verification Report: Tenor (Migration & Renewal Callbacks, Migration Ratifier)

- Date: July 5th, 2026
- Audit Repo: https://github.com/alexzoid-eth/tenor-contracts-fv
- Client Repo: https://github.com/tenor-labs/tenor-contracts
- Audit Commit: d2cb2ebe3d15ec37aeaf9a5aa19081d53394d8ca (July 5th, 2026)
- Author: AlexZoid
- Certora Prover version: 8.16.2

Table of Contents

About Tenor Migration Callbacks

Tenor extends the Morpho lending stack with **Morpho Midnight**, a zero-coupon, fixed-rate trading market that sits alongside the variable-rate **Morpho Blue** markets and the yield-bearing **Morpho Vault V2**. Positions are opened and closed through Midnight's `take()` settlement: an off-chain maker offer is matched against an on-chain taker, an intent ratifier validates the trade, and Midnight then invokes an `onBuy` or `onSell` callback that moves the assets and positions atomically. The callbacks in scope let users migrate borrows and lends

between Blue, Midnight, and the Vault and roll fixed-rate positions from a maturing market into a new one, without leaving an unsafe intermediate state.

The verification covers the six migration and renewal callbacks and the three supply/withdraw collateral callbacks as independent targets:

1. **BorrowBlueToMidnightCallback (BBM)** — V1 → V2 borrow migration: repays the Blue debt, withdraws the freed collateral, and reopens the borrow on Midnight.
2. **BorrowMidnightToBlueCallback (BMB)** — V2 → V1 borrow migration, the mirror of BBM: withdraws Midnight collateral and opens a Blue borrow.
3. **BorrowMidnightRenewalCallback (BMR)** — V2 → V2 borrow renewal: rolls a borrower's debt and collateral into a fresh Midnight market.
4. **LendVaultToMidnightCallback (LVM)** — vault-funded lend entry: redeems Vault V2 shares and supplies the loan token as Midnight lender credit.
5. **LendMidnightToVaultCallback (LMV)** — lend exit into the vault: withdraws a lender's Midnight credit and deposits it into the vault.
6. **LendMidnightRenewalCallback (LMR)** — V2 → V2 lend renewal: rolls a lender's credit into a fresh Midnight market, charging the renewal fee.
7. **MidnightSupplyCollateralCallback (MSC)** — supplies extra collateral into the seller's Midnight position during a take, pro-rata to the fill and guarded by an optional borrow-capacity cap.
8. **MidnightSupplyVaultSharesCallback (MSV)** — deposits the loan token into an ERC-4626 vault and supplies the minted shares as the seller's collateral.
9. **MidnightWithdrawVaultSharesCallback (MWV)** — the withdraw-side twin of MSV: redeems vault-share collateral out of a position during a take.

Beyond the callbacks, one standalone target covers the intent-validation layer:

10. **MigrationRatifier** — the on-chain gatekeeper that validates every migration before Midnight settles it.

Formal Verification Methodology

Certora Formal Verification (FV) proves smart-contract correctness against a formal specification, examining all reachable states rather than the specific paths testing and fuzzing sample. Properties are written in CVL (Certora Verification Language) and submitted with the compiled contracts to the prover, which turns bytecode and rules into a mathematical model and decides each rule's validity.

Verification Approach

Each of the nine callbacks is verified as its own independent target, each with its own configuration file, harness, and rule set. Because every callback executes only as a sub-call of Midnight's settlement, the verified contract is the `MidnightHarness` model and the callback under test is linked in via Midnight's `MORPHO_MIDNIGHT` / `MORPHO_BLUE` slots; every rule invokes `take()` on that model and asserts before/after deltas over ghost state captured around the single settlement, so each proof isolates one callback's cross-protocol effect while running Midnight's real `onBuy` / `onSell` dispatch.

- The **Midnight** market state is modelled in CVL with ghost mappings and storage hooks, narrowed to a tractable set of touched markets and users (`midnight.spec`, `midnight_one.spec`, `midnight_many.spec`). Coupling rules that must reason about two markets at once run in a single-market scalar regime via the `_one` spec variants.
- The **Morpho Blue** market state (borrow shares and collateral) is linked as a second model for the two Blue-touching callbacks (BBM, BMB) via the `MorphoHarness` (`morpho-blue/`). Blue's own valid-state invariants are reused as preconditions so the migration starts from a well-formed Blue position. On these scenes the Blue sources are compiled from the Certora-only fork `certora/harnesses/morpho-blue/` — upstream code with the `Market` struct renamed to `MarketBlue` to avoid a scene-wide type-name collision with Midnight's own `Market` struct (see the fork's README); `lib/morpho-blue` stays the faithful upstream mirror used by the Foundry build.
- The four vault-touching callbacks (LVM, LMV, MSV, MWV) run against the **real Morpho Vault V2** contract (`lib/vault-v2`), linked on the scene as `VaultV2Harness`; the vault's share ledger is mirrored into the shared ERC-20 ghost infrastructure and its ERC-20 share operations are routed to the real code (its first-stage fee/adapter pins are listed under Scope Assumptions). The callbacks' vault behaviour is not summarised in CVL; only the global `asset()` ghost remains, supplied by `erc4626_asset.spec`.
- External token movements use a CVL **ERC-20** model (`erc20.spec`) with ghost balances/allowances and a conserved total supply; the **oracle** (`oracle.spec`), **ratifier** (`ratifier.spec`), and **gates** (`gates.spec`) are summarised so that a settlement is reachable without modelling the off-chain intent machinery.

Together the runs prove that each callback can only move debt, collateral, and credit in the protocol-intended directions and amounts, never strands tokens or approvals, never touches an unrelated user, and never lets the fee recipient lose value.

The MigrationRatifier standalone target is verified directly against its own harness rather than through a `take()`. It summarizes its external dependencies (`getRate`, `settlementFee`, `continuousFee`, `cadencePeriodStart`, `isAuthorized`, `TickLib.tickToPrice`) with deterministic CVL ghosts — stable across calls within a rule, which is what the two-call monotonicity lifts over the full rate gate require — and exposes the internal helpers, the window/cadence machinery, the full rate gate, and typed production-entry bridges via `MigrationRatifierHarness`.

Types of Properties

Each callback's rule set combines several property categories:

State-Transition / Direction properties — The bulk of the per-callback rules. They capture a ghost value before `take()`, run the settlement, and assert the value changed only in the permitted direction or magnitude: old debt and collateral only shrink, new debt and collateral only grow, a reduction can land on at most one market, debt and collateral move together, and a migrated inflow is bounded by the matching outflow.

Unit / Conservation properties — Direct effects and non-effects of a single settlement: Midnight's balance changes only by the trading fee on non-settled tokens, collateral is left untouched by the lend callbacks, and an unrelated bystander's credit, debt, and balances are unchanged.

Reachability properties — `satisfy()` rules proving the intended outcomes are actually achievable (non-vacuous verification): a migration can move collateral across protocols, fully close an old position, open new debt, roll credit while charging a fee.

Reverts / Authorization properties — `@withrevert` rules confirming the callback rejects any caller other than Midnight and rejects invocations with zero assets or zero units.

Verification Process

1. **Setup phase**: Define ghost variables, storage hooks, and helper definitions to model Midnight, Morpho Blue, Morpho Vault V2, ERC-20 tokens, the oracle, the ratifier, and the gates in CVL. Establish the per-callback harness and configuration. This phase also addresses several prover limitations:
    - ERC-20 token model (`erc20.spec`): the account set is bounded, so transfers stay tractable and free of spurious reentrancy.
    - Morpho Vault V2 model (see Verification Approach): the real `lib/vault-v2` is pinned to a first-stage config so its interest-accrual loop is dead and `loop_iter: 2` suffices; the only ERC-4626 CVL residue is the global `asset()` ghost (`erc4626_asset.spec`).
    - Midnight library summaries — tick (`tick_lib.spec`), id (`id_lib.spec`), and arithmetic (`utils_lib.spec`) — alongside the scalar single-/three-market narrowings of position and market state.
    - Oracle, ratifier, and gate summaries (same specs as in the Approach): a well-behaved positive oracle price and empty ratifier/callback payloads, so a single `take()` stays within the run budget.
    - Per-callback setup (`certora/specs/setup/callbacks/`): pins the realistic make-on-behalf activation of each callback (see the Scope and Trusted Assumptions below) and the disjointness of the fee recipient, bystanders, and the callback address.
2. **Crafting Properties**: Write the shared safety layer first (`callbacks.spec`), reuse Midnight and Morpho Blue valid-state invariants as preconditions, then express each callback's direction, conservation, reachability, and revert properties as deltas around `take()`.
3. **Non-vacuity checks**: Because each property runs under several `require` narrowings, an `assert` rule could pass simply because its guarded case is unreachable under those narrowings (a vacuous proof). To rule this out, every rule has a debug-only "satisfy twin" (under each callback's `specs/callbacks/<Callback>/debug_satisfy/`) that keeps the rule body and its narrowings verbatim but swaps the final `assert(...)` for a `satisfy(...)` witnessing the exact scenario the rule guards (a

monotone bound `after <= before` becomes `satisfy(after < before)`, an implication `A => B` becomes `satisfy(A)`, a revert rule becomes `satisfy(A && reverted)`, and so on). A reachable witness confirms the rule constrains a real execution rather than an empty set of states. These twins are a sanity aid only — committed alongside the production specs but excluded from the proof suite and the property counts in this report.

Performance overlay channel (research). To shorten SMT time on the heaviest callback rules, a research channel adds per-rule *profile overlay* specs under `certora/specs/callbacks/<Callback>/perf/` that `import` the production spec and summarise the code branches a given rule provably never reads (each stub backed by a per-rule footprint analysis — the branch's write-set disjoint from the rule's read-set — and an adversarial skeptic pass). They drive parallel `perf` / `perf_satisfy` / `perf_kill` / `perf_kill_satisfy` / `perf_heavy` conf channels plus a B9-lite `override function` variant. Like the satisfy twins, the overlays are excluded from the proof suite and the property counts. Each overlay header records its stub set (STUBS) and the rules it serves (RULES).

Assumptions

Assumptions address prover timeouts, tool limitations, and state consistency, but incorrect ones can mask bugs by excluding reachable states. Each falls into one of six groups: SAFE (real-world constraints that don't reduce security coverage), UNSAFE (tractability narrowings that exclude reachable, valid scenarios not covered by a sibling rule), SCOPE (regime pins that fix which scenario a rule claims about, where the excluded region is covered elsewhere or is immaterial), PROVED (formally verified invariants or separately-proven rule-lemmas reused as preconditions), ASSERT (facts a real in-scene contract check or revert already enforces, assumed as a precondition), and TRUSTED (activation, fee-recipient configuration, settlement routing, and initialization state assumed correct because the off-chain intent, ratifier, and market-creation logic are excluded from verification). Each such `require` carries a matching `"<CATEGORY>: ..."` message string in the code.

Safe Assumptions

Environment Constraints (`env.spec`):
EVM environment constraints that hold for all realistic transactions, and that are kept stable across the two environments of before/after rules.

- Transactions carry no ETH value (`msg.value == 0`)
- The sender is non-zero and is not the contract itself
- Block timestamp is bounded to a realistic range, and the block number is non-zero
- Block number, timestamp, sender, and value are preserved across paired before/after environments

ERC-20 Token Model (`erc20.spec`):
The token model reflects well-behaved ERC-20 tokens used by the protocol.

- Total supply equals the sum of all balances
- Token decimals are realistic (between 6 and 18)
- The called token contract is not the zero address

ERC-4626 `asset()` ghost (`erc4626_asset.spec`):
The four vault-touching callbacks (LVM, LMV, MSV, MWV) run against the real Morpho Vault V2 contract, not a CVL model: its valid-state invariants are reused as preconditions under Proved Assumptions, and its first-stage fee/adapter pins are listed under Scope Assumptions. The only ERC-4626 CVL residue is a single global

`asset()` ghost — each vault's underlying-asset accessor — with no deposit/redeem/preview/convert modelling.

Bystander and Settlement Constraints (`callbacks_setup.spec`, `*_cmn.spec`, `*_setup.spec`):
Real-world disjointness and environment facts that hold for any honest settlement, independent of how the intent was configured.

- The chosen bystander user is not the active fee recipient
- For BBM and BMB, the bystander user is not Morpho Blue (the cross-protocol partner never holds a Tenor bystander position in these flows)
- The callback is not itself an ERC-4626 vault (LMV, LVM), and the take receiver is not the callback in the default flow
- The take participants (`msg.sender`, the tracked Midnight position users, `offer.maker`, `taker`) are real externally-owned accounts, each distinct from the callback and from Midnight, which are themselves distinct deployed addresses
- The renewal callbacks run with `block.chainid == INITIAL_CHAIN_ID`

Proved Assumptions

Reused as preconditions (via `requireInvariant`, or `setupValidStateVaultV2` for the vault) so each migration starts from a well-formed Midnight (and, for BBM/BMB, Morpho Blue; for the four vault-touching callbacks, Morpho Vault V2) position.

Midnight Valid State (`midnight_valid_state_one.spec`):

- A collateral slot is activated in the bitmap iff its balance is non-zero — VS-MI-04 (`collateralBitmapMatchesSlot`)
- A position with any non-zero field implies the market has been touched — VS-MI-05 (`nonEmptyPositionImpliesTouched`)
- A user is either a lender (credit) or a borrower (debt), never both — VS-MI-06 (`creditAndDebtMutuallyExclusive`)

Morpho Blue Valid State (`morpho_valid_state_one.spec`), reused by the cross-protocol borrow callbacks (BBM, BMB):

- A non-existent market has zero positions — VS-21 (`nonExistentMarketPositionsZero`)
- A borrower always has collateral — VS-22 (`alwaysCollateralized`)

Morpho Vault V2 Valid State (`vaultV2_valid_state.spec`), reused by the four vault-touching callbacks (LVM, LMV, MSV, MWV) via `setupValidStateVaultV2`:

- The ten Morpho Vault V2 valid-state invariants (share-ledger solvency, accounting coherence, and configuration well-formedness) ported from alexzoid's morpho-vault-v2-fv, proven standalone (10/10) on the ported VaultV2 target and reused here as preconditions (via `setupValidStateVaultV2`)

MigrationRatifier (`ratifier/highlevel.spec`, `unit.spec`):

- The stored fee rate is capped at `MAX_FEE_RATE` by the `setFeeConfig` guard — invariant RTF-VS-01 — reused where the fee-match gate reads it
- Net seller/buyer price is monotone in the fee — rule-lemmas RTF-UT-11 / RTF-UT-13 — imported by the

high-level gate-binding rules (`PROVED` now covers both invariants and separately-proven rule-lemmas)

> Note: The full Morpho Blue, Midnight, and Morpho Vault V2 valid-state suites were verified separately; only the invariants needed as preconditions are reused here (Morpho Blue model: see `certora/specs/setup/morpho-blue/README.md`).

Scope Assumptions

These pin a rule to its target scenario (e.g. a particular take direction, at-par settlement, or a present source position).

Make-on-behalf Callback Activation (`cmn.spec`, `*_setup.spec`):
Make-on-behalf is the only modeled migration path: the migrating user is the offer **maker**, and `takerCallback` is pinned to zero (the single-sided maker scenario excludes the taker-side callback). `MigrationRatifier.isRatified` validates only `offer.maker`, and `TenorRouter` only takes — so a two-sided activation or a taker-side migration callback is not a production-reachable path. The activated callback address and settlement side are trusted activation facts (see Trusted Assumptions).

Market and Position Narrowing (`midnight_one.spec`, `midnight_many.spec`, `midnight.spec`):
Midnight's unbounded market and user maps are narrowed to a small, fixed scope; state outside the scope is pinned to zero. The prover then reasons only about the in-scope markets and users.

- At most one market (scalar `_one` regime) or three markets (`_many` regime) are touched; market state outside the scope is zero
- Positions are narrowed to a three-user set; per-user position fields outside the set are zero
- A two-collateral model with the collateral bitmap kept consistent with the per-slot balances and length

ERC-20 Account Bounding (`erc20.spec`):

- Accounts (sender, from, to, owner, spender) are drawn from a predefined, pairwise-distinct bounded set
- Per-account balances are bounded by `max_uint128` to avoid overflow in downstream arithmetic

Tick Narrowing (`tick_lib.spec`):

- The market tick is restricted to a small fixed set of representative values

Empty Callback / Ratifier Payloads (`callbacks.spec`, `ratifier.spec`):

- `take` / `buy` / `sell` / `repay` / `liquidate` / `flash` are invoked with empty callback and ratifier data, so a single settlement stays within the run budget

Morpho Vault V2 First-Stage Pinning (`*_setup.spec`, `*_cmn.spec`):
The real Vault V2 is pinned to a first-stage configuration so its interest-accrual loop is dead and `loop_iter: 2` suffices.

- `performanceFee == managementFee == 0`, `adaptersLength == 0`, and `liquidityAdapter == 0`

Prover Configuration:

- Loop unrolling is capped at 2 iterations across all callback configurations (`loop_iter: 2` with `optimistic_loop`), and each rule is given a 4-hour SMT timeout (`smt_timeout: 14400`).
- Hashing of dynamically-sized data is optimistic (`optimistic_hashing: true`, `hashing_length_bound:`

`2048` in both callback base confs): hashed dynamic data is assumed at most 2048 bytes long; executions hashing longer data are excluded.

- Unresolved external calls in the ratifier confs are optimistic (`optimistic_fallback: true`): a call dispatching to an unknown function or fallback is assumed not to havoc the verified state; only its return data is nondeterministic.

Unsafe Assumptions

A bug hiding only in the excluded region would be missed.

- **Performance overlays** (`<Callback>/perf*`): the branch-stub perf overlays disable individual code branches purely for speed, so every assumption they add is Unsafe by construction
- **MSC fill-fraction denominator** (`MidnightSupplyCollateralCallback/cmn.spec`): the shared balance / fee-recipient / rate-distortion rules now range over both the `maxAssets` and `maxUnits` legs, but the `MidnightSupplyCollateralCallback` scene keeps a TRUSTED pin `offerSellerAssets == offer.maxAssets` (`cmn.spec:21-22`) that ties that scene's fill-fraction denominator to the `maxAssets` leg
- **Rate-domain slices** (`ratifier/unit.spec`): the price-monotonicity rules run on a `uint40`-rate slice even though the production buy branch can reach the full policy-rate range
- **IRM borrow-rate cap** (`morpho-blue/specs/setup/morpho.spec`): the per-IRM rate ghost is capped at 2e18/yr for tractability, while the canonical AdaptiveCurveIrm curve ceiling is `CURVE_STEEPNESS (4) * MAX_RATE_AT_TARGET` = 8e18/yr — rates in `(2e18/yr, 8e18/yr]` are not explored on the Blue scenes

Assert Assumptions

- **Vault-collateral validity** (`MidnightSupplyVaultShares/cmn.spec`, `MidnightWithdrawVaultShares/cmn.spec`): `validateVaultCollateral` reverts unless `vault.asset() == loanToken` and the vault is listed at its collateral slot
- **Percentage-fee cap** (`callbacks.spec`): `percentageFee` reverts above the 1% contract cap
- **Ratifier real-path facts** (`reachability.spec`): the `tickToPrice` domain (`TickLib` reverts above `MAX_TICK`)

Trusted Assumptions

Admin/ratifier-controlled values — the activated callback and its settlement side (`offer.callback`, `offer.buy`) — are Trusted; pure scenario slices excluded from the model (e.g. the taker-side `takerCallback == 0`) are Scope instead.

Callback Activation and Side Pinning (`*_cmn.spec`, `*_setup.spec`):
Each callback's setup pins the single realistic activation the ratified intent produces — the settlement side and the activated callback address.

- BBM, BMR, LMV, MSC, MSV are activated on the sell side (`offer.buy == false`, `onSell`); BMB, LMR, LVM, MWV on the buy side (`offer.buy == true`, `onBuy`)
- `offer.callback` is the callback under test (the activated make-on-behalf callback); that a take actually invokes the callback's `onSell`/`onBuy` is a Scope regime pin

Ratifier-Enforced Intent Constraints (`*_cmn.spec`, `*_setup.spec`):
The ratifier validates each intent before Midnight settles it; because the ratifier is summarised rather than executed, these validations are taken on trust.

- The callback tick equals the offer tick (`decodeCallbackTick(offer.callbackData) == offer.tick`) — for the tick-carrying callbacks (BBM, BMR, LMR, LVM); MidnightSupplyVaultShares and MidnightSupplyCollateral carry no tick field
- The callback fee rate is capped at `MAX_FEE_RATE` for BBM, BMR, LMR, and LVM, and a positive fee requires a non-zero fee recipient
- The BMB and LMV main flows are verified with the callback fee disabled (`cbFeeRate == 0`); the exception is CLB-BMB-10, which varies the rate to characterize the percentage-fee 1% cap guard (`feeRate > MAX_PERCENTAGE_FEE_RATE => revert`)

Fee Recipient Configuration (`callbacks.spec`, `*_cmn.spec`, `*_setup.spec`):
The fee recipient is an admin/ratifier-configured address, so its disjointness from the take participants is trusted rather than a free real-world fact.

- The fee recipient is not the callback, not a take participant, not Midnight, not `msg.sender`, and not an ERC-4626 vault
- For BBM and BMB, the fee recipient is not Morpho Blue (the cross-protocol partner is never a Tenor fee recipient)

Settlement Funding Flow (`*_cmn.spec`, `*_setup.spec`):
For the seller-funded flows, the ratified intent routes the seller assets to the callback so it can perform the repay or deposit; this routing is taken on trust.

- The seller assets land on the callback for BBM (Blue repay), BMR (Midnight repay), and LMV (vault deposit)

Market Initialization (`midnight.spec`, `midnight_one.spec`):

- The market's loan token and collateral token were set by a prior `touchMarket` (post-initialization state)
- The loan token and the collateral token are not the lending contract itself

Standalone-Target Model (Ratifier)

The MigrationRatifier target adds its own model assumptions, annotated in-spec with the full six-category scheme; here PROVED covers invariant RTF-VS-01 and rule-lemmas RTF-UT-11 / RTF-UT-13, and no `TRUSTED` is used because the ratifier is the verified target rather than excluded code:

- **Deterministic ghost summaries** (MigrationRatifier): `getRate` is an uninterpreted function bounded by `2^128` (the ASSUME-POLICY-1 boundary); `settlementFee` and `TickLib.tickToPrice` are bounded by `WAD`, the tick price additionally monotone (the ASSUME-TICK-1 trust boundary — TickLib is audited with Midnight and only monotonicity and the bound are relied on); `continuousFee` is bounded by `uint32` (its return type); `isAuthorized` and `cadencePeriodStart` are free ghost tables. Safe.
- **Rate-gate scope slices** (MigrationRatifier): the four rate-gate monotonicity rules run on the `ratifyRate` exposer (the full two-call `isRatified` differential does not converge locally); the lender-limit rule additionally pins `continuousFee == 0` (a Scope regime pin; the haircut is a common factor of both runs), and both limit rules carry a `SAFE:` target-maturity date bound (`now <= tgtMat <= now + max_uint32`, a ~136-year window that admits every realistic maturity — the lower edge is anyway

enforced on the real path by `_validateTargetMaturity`) so `rate * duration` cannot overflow. All four rules (and their two satisfy twins) also pin `offer.market.collateralParams.length == 0` — Safe: the rate gate never reads `collateralParams` (`idLibToIdCVL` drops them from the summarized market id), so the pin only removes a gate-unrelated calldata address-decode revert from the harness call. The `uint40`-rate domain slice used by the price-monotonicity lemmas is Unsafe. The window/cadence/target gates are characterized on the full `isRatified` flow without these slices.

- **Live-source exit pin** (MigrationRatifier): RTF-HL-02 (`v2v1ExitsHaveNoRenewalCadenceConstraint`) requires `offer.market.maturity != 0` on a V2→V1 exit — a live Midnight source has maturity>0 by construction (`maturity==0` is the V1/non-Midnight sentinel), so the excluded corner is a malformed offer, not a real scenario. Safe.

Verification Properties

Links to specific CVL spec files are provided for each property. **Status** (production run): ✅ verified · ⏱️ timed out (4-hour SMT budget exhausted) · ❌ violated.

Mutations (diffs): ❌ caught · ❓ not caught. ˢ = caught via satisfy-twin.

Migration Ratifier ✅ and ❌ icons link to the corresponding runs on the Certora cloud prover (anonymous links).

Callbacks

All properties exercise Midnight's `take()` settlement. The **Shared Safety Rules** are re-verified under every applicable callback's setup; each callback's own properties and the per-callback status follow in the tables below.

Shared Safety Rules

Property	Name	Description	Mutations
CLB-01 (CB-DUST-1)	`callbackHoldsZeroAllowance`	The callback never leaves token approvals to anyone. `allowance[t][callback][s] == 0 => allowance[t][callback][s]' == 0`	❌ BorrowMidnightRenewalCallback#35ˢ ❌ MidnightWithdrawVaultSharesCallback#2
CLB-02	`thirdPartyBalanceUnchanged`	An operation never changes balances of bystanders (users unrelated to it). `u unrelated to take => balance[t][u]' == balance[t][u]`	❌ BorrowMidnightRenewalCallback#35ˢ
CLB-03 (CB-DUST-1)	`callbackNeverHoldsTokens`	The callback never ends holding more tokens than it started — it can sweep pre-existing funds out but cannot leave new dust. `balance[t][callback]' <= balance[t][callback]`	❌ BorrowMidnightRenewalCallback#33ˢ ❌ MidnightWithdrawVaultSharesCallback#6
CLB-04 (CB-AUTH-1)	`callbackRevertsForNonMidnightCaller`	The callback rejects calls from anyone other than Midnight. `msg.sender != Midnight => REVERTS`	❌ BorrowBlueToMidnightCallback#1 ❌ MidnightSupplyCollateralCallback#1
CLB-05	`callbackRevertsOnZeroAssetsOrUnits`	The callback rejects operations with zero assets or zero units. `(assets == 0 OR units == 0) => REVERTS`	❌ LendMidnightRenewalCallback#2 ❌ MidnightSupplyCollateralCallback#2
CLB-06	`feeRecipientNeverLosesTokens`	The fee recipient's balance never decreases during an operation. `balance[t][feeRecipient]' >= balance[t][feeRecipient]`	❌ BorrowMidnightRenewalCallback#35ˢ
CLB-07 (CB-FEE-3)	`percentageFeeNeverExceedsAssets`	The flat percentage fee paid never exceeds assets/100 (sharp 1% cap; cannot exceed the principal charged). `!reverted => 100 * delta(claimableFee) <= assets`	❌ LendMidnightToVaultCallback#10
CLB-08 (CB-FEE-1)	`sellerTickFeeNeverExceedsAssets`	The seller tick fee paid never exceeds sellerAssets, so the callback's `repayBudget = sellerAssets - fee` never underflows. `!reverted => delta(balance[loanToken][feeRecipient]) <= sellerAssets`	❌ BorrowMidnightRenewalCallback#22
CLB-09 (CB-FEE-2)	`buyerTickFeePaidBoundedByUnits`	The buyer tick fee paid is bounded by the trade size `units`. `!reverted => delta(balance[loanToken][feeRecipient]) <= units`	❌ LendMidnightRenewalCallback#14

Property	Name	Description	Mutations
CLB-10	`positiveFeeIsPayable`	A positive fee is actually payable through the callback, so the CLB-07/08/09 bounds are not vacuously about a zero fee. `satisfy(delta(claimableFee) > 0)`	❌ LendMidnightRenewalCallback#8

Per-callback re-verification. Statuses per the legend above; · = rule not applicable to that callback.

Rule	BBM	BMB	BMR	LMR	LMV	LVM	MSC	MSV	MWV
CLB-01 `callbackHoldsZeroAllowance`	⏱️	✅	✅	✅	✅	✅	✅	✅	✅
CLB-02 `thirdPartyBalanceUnchanged`	✅	✅	✅	✅	✅	✅	✅	✅	✅
CLB-03 `callbackNeverHoldsTokens`	⏱️	✅	✅	✅	✅	✅	✅	✅	✅
CLB-04 `callbackRevertsForNonMidnightCaller`	✅	✅	✅	✅	✅	✅	✅	✅	✅
CLB-05 `callbackRevertsOnZeroAssetsOrUnits`	✅	✅	✅	✅	✅	✅	✅	✅	✅
CLB-06 `feeRecipientNeverLosesTokens`	✅	✅	✅	✅	✅	✅	·	·	·
CLB-07 `percentageFeeNeverExceedsAssets`	·	✅	·	·	✅	·	·	·	·
CLB-08 `sellerTickFeeNeverExceedsAssets`	✅	·	✅	·	·	·	·	·	·
CLB-09 `buyerTickFeePaidBoundedByUnits`	·	·	·	⏱️	·	⏱️	·	·	·
CLB-10 `positiveFeeIsPayable`	✅	✅	✅	✅	✅	✅	·	·	·

BorrowBlueToMidnightCallback (BBM)

Property	Name	Description	Status	Mutations
CLB-BBM-01 (CB-V1-REP-1)	`migrationOnlyReducesOldBlueDebt`	Migration can only reduce the old Blue debt, never increase it. `blueBorrowShares[id][u]' <= blueBorrowShares[id][u]`	✅	❌ BorrowBlueToMidnightCallback#15
CLB-BBM-02 (CB-DIR-1)	`migrationOnlyWithdrawsOldBlueCollateral`	Migration can only withdraw old Blue collateral, never add to it. `blueCollateral[id][u]' <= blueCollateral[id][u]`	✅	❌ BorrowBlueToMidnightCallback#16
CLB-BBM-03 (CB-DIR-1)	`migrationReducesOldDebtOnAtMostOneMarket`	One migration can reduce old debt on at most one Blue market. `NOT( blueBorrowShares[idA][u]' < blueBorrowShares[idA][u] AND blueBorrowShares[idB][u]' < blueBorrowShares[idB][u] )`	✅	❌ BorrowBlueToMidnightCallback#20[s] ❌ BorrowMidnightToBlueCallback#29[s]
CLB-BBM-04 (CB-FINAL-2)	`clearingOldDebtAlsoEmptiesOldCollateral`	Clearing the last of the old debt also empties the old collateral. `blueBorrowShares[id][u] > 0 AND blueBorrowShares[id][u]' == 0 AND blueCollateral[id][u] > 0 => blueCollateral[id][u]' == 0`	⏱️	❌ BorrowBlueToMidnightCallback#3
CLB-BBM-05 (CB-DIR-1)	`migrationOnlyAddsNewMidnightCollateral`	Migration only adds new Midnight collateral, never removes it. `mnCollateral[id][u][i]' >= mnCollateral[id][u][i]`	✅	❌ BorrowBlueToMidnightCallback#9
CLB-BBM-06 (CB-DIR-1)	`migrationConservesMigratedCollateral`	Collateral withdrawn from the old Blue position equals the collateral deposited into the new Midnight position — conserved 1:1 during migration. `blueCollateral[id][u] - blueCollateral[id][u]' == mnCollateral[mnId][u][i]' - mnCollateral[mnId][u][i] (when both sides move)`	✅	❌ BorrowBlueToMidnightCallback#4

Property	Name	Description	Status	Mutations
CLB-BBM-07	`migrationCanMoveCollateralBlueToMidnight`	A migration can actually move collateral from the old position to the new one. `satisfy(blueCollateral[id][u]' < blueCollateral[id][u] AND mnCollateral[id][u][i]' > mnCollateral[id][u][i])`	✅	❌ BorrowBlueToMidnightCallback#2
CLB-BBM-08 (CB-CLOSE-1)	`migrationCanFullyCloseOldPosition`	A migration can fully close the old position (both debt and collateral go to zero). `satisfy(blueBorrowShares[id][u]' == 0 AND blueCollateral[id][u]' == 0) (pre: both > 0)`	✅	❌ BorrowBlueToMidnightCallback#21 ❌ BorrowMidnightToBlueCallback#10
CLB-BBM-09 (CB-RATE-1)	`borrowerFeeBoundedByInterestShare`	The borrower's callback fee never exceeds feeRate applied to the interest portion of the trade, so the effective seller rate stays within (1 + feeRate/WAD) of the offer rate. `f*WAD^2 <= units*(WAD - price)*feeRate (+ one-unit ceil rounding slack)`	⏱	❌ BorrowBlueToMidnightCallback#18 ❌ BorrowMidnightRenewalCallback#36
CLB-BBM-10 (CB-FEE-4)	`tickFeeVanishesAtPar`	At par (price == WAD) with full-value settlement (assets == units) the seller tick fee vanishes (carved from interest, not principal). `price==WAD && assets==units && !reverted => feeRecipient delta == 0`	✅	❌ CallbackLib#1 ❌ CallbackLib#4 ❌ CallbackLib#5 ❌ CallbackLib#6
CLB-BBM-11 (CB-CLOSE-2)	`fullCollateralMigrationClearsAllOldDebt`	Repaying expectedBorrowAssets(seller) clears all V1 borrow shares — full collateral migration empties the old debt. `blueCollateral[id][u] > 0 AND blueCollateral[id][u]' == 0 AND blueBorrowShares[id][u] > 0 => blueBorrowShares[id][u]' == 0`	✅	❌ BorrowBlueToMidnightCallback#12[5]
CLB-BBM-12 (CB-DUST-1)	`receiverNotCallbackReverts`	The migration onSell requires the sale proceeds to route to the callback itself (else the sellerAssets it needs to repay the old Blue debt are stranded). `receiver != address(callback) => REVERTS (InvalidReceiver)`	✅	❌ BorrowBlueToMidnightCallback#22 ❌ BorrowMidnightRenewalCallback#13 ❌ LendMidnightToVaultCallback#11 ❌ MidnightSupplyVaultSharesCallback#10
CLB-BBM-13 (CB-SRC-1)	`sourceLoanTokenMismatchReverts`	onSell rejects a source Blue market whose loanToken differs from the offer loanToken. `sourceMarketParams.loanToken != market.loanToken => REVERTS (TokenMismatch)`	✅	❌ BorrowBlueToMidnightCallback#14

BorrowMidnightToBlueCallback (BMB)

Property	Name	Description	Status	Mutations
CLB-BMB-01 (CB-DIR-1)	`migrationOnlyWithdrawsOldMidnightCollateral`	Migration can only withdraw old Midnight collateral, never add to it. `mnCollateral[id][u][i]' <= mnCollateral[id][u][i]`	✅	❌ BorrowMidnightToBlueCallback#20
CLB-BMB-02 (CB-DIR-1)	`migrationReducesOldDebtOnAtMostOneMarket`	One migration can reduce old debt on at most one Midnight market. `NOT( mnDebt[idA][u]' < mnDebt[idA][u] AND mnDebt[idB][u]' < mnDebt[idB][u] )`	✅	❌ BorrowBlueToMidnightCallback#20[5] ❌ BorrowMidnightToBlueCallback#29[5]
CLB-BMB-03 (CB-DIR-1)	`migrationOnlyOpensNewBlueDebt`	Migration only opens new Blue debt, never reduces it. `blueBorrowShares[id][u]' >= blueBorrowShares[id][u]`	✅	❌ BorrowMidnightToBlueCallback#27
CLB-BMB-04 (CB-DIR-1)	`migrationOnlyAddsNewBlueCollateral`	Migration only adds new Blue collateral, never removes it. `blueCollateral[id][u]' >= blueCollateral[id][u]`	✅	❌ BorrowMidnightToBlueCallback#26
CLB-BMB-05 (CB-SRC-1)	`migrationCannotDepositMoreCollateralThanWithdrawn`	Migration cannot deposit more new collateral than it withdrew from the old position. `mnColOut > 0 AND blueColIn > 0 => blueColIn <= mnColOut (mnColOut = mnCollateral[id][u][i] - mnCollateral', blueColIn = blueCollateral' - blueCollateral)`	✅	❌ BorrowMidnightToBlueCallback#24
CLB-BMB-06 (CB-DIR-1)	`oldMidnightDebtAndNewBlueDebtMoveTogether`	Opening new Blue debt always implies the source Midnight debt drops (forward direction only; the reverse need not hold because a partial-redeem path can close Midnight debt without engaging Blue migration). `delta(blueBorrowShares) > 0 => delta(mnDebt) < 0`	✅	❌ BorrowMidnightToBlueCallback#30
CLB-BMB-07	`migrationCanOpenNewBlueDebt`	A migration can actually open new Blue debt. `satisfy(blueBorrowShares[id][u]' > blueBorrowShares[id][u])`	⏱	❌ BorrowMidnightToBlueCallback#8

Property	Name	Description	Status	Mutations
CLB-BMB-08	`migrationCanMoveCollateralMidnightToBlue`	A migration can actually move collateral from the old position to the new one. `satisfy(mnCollateral[id][u][i]' < mnCollateral[id][u][i] AND blueCollateral[id][u]' > blueCollateral[id][u])`	⏱	❌ BorrowMidnightToBlueCallback#9
CLB-BMB-09 (CB-CLOSE-1)	`migrationCanFullyCloseOldPosition`	A migration can fully close the old position (both debt and collateral go to zero). `satisfy(mnDebt[id][u]' == 0 AND mnCollateral[id][u][i]' == 0) (pre: both > 0)`	✅	❌ BorrowBlueToMidnightCallback#21 ❌ BorrowMidnightToBlueCallback#10
CLB-BMB-10 (CL-2, InvalidFeeConfig)	`percentageFeeRateAboveCapReverts`	A callback fee rate above the 1% cap (MAX_PERCENTAGE_FEE_RATE = 0.01e18) is rejected by `CallbackLib.percentageFee`, verified through the BMB onBuy entry point. `decodeCallbackFeeRate(data) > MAX_PERCENTAGE_FEE_RATE => REVERTS`	✅	❌ BorrowMidnightToBlueCallback#18
CLB-BMB-11 (CB-FINAL-3)	`migrationFinalFillTransfersAllOldMidnightCollateral`	A final fill (old Midnight debt fully cleared) transfers ALL the old Midnight collateral, never a pro-rata fraction. `mnCollateral[id][u][i] > mnCollateral[id][u][i]' AND mnDebt[id][u]' == 0 => mnCollateral[id][u][i]' == 0`	✅	❌ BorrowMidnightToBlueCallback#25

BorrowMidnightRenewalCallback (BMR)

Property	Name	Description	Status	Mutations
CLB-BMR-01 (CB-DIR-1)	`renewalReducesDebtOnAtMostOneMarket`	One renewal can reduce debt on at most one market. `NOT( mnDebt[idA][u]' < mnDebt[idA][u] AND mnDebt[idB][u]' < mnDebt[idB][u] )`	✅	❌ BorrowMidnightRenewalCallback#24
CLB-BMR-02 (CB-DIR-1)	`renewalAddsDebtOnAtMostOneMarket`	One renewal can add debt on at most one market. `NOT( mnDebt[idA][u]' > mnDebt[idA][u] AND mnDebt[idB][u]' > mnDebt[idB][u] )`	✅	❌ BorrowMidnightRenewalCallback#34[s]
CLB-BMR-03 (CB-DIR-1)	`renewalCannotAddCollateralWhenReducingDebt`	In a market where renewal reduces debt, collateral cannot rise. `mnDebt[id][u]' < mnDebt[id][u] => mnCollateral[id][u][i]' <= mnCollateral[id][u][i]`	✅	❌ BorrowMidnightRenewalCallback#23
CLB-BMR-04 (CB-DIR-1)	`renewalCannotRemoveCollateralWhenOpeningDebt`	In a market where renewal opens debt, collateral cannot drop. `mnDebt[id][u]' > mnDebt[id][u] => mnCollateral[id][u][i]' >= mnCollateral[id][u][i]`	✅	❌ BorrowMidnightRenewalCallback#23 ❌ BorrowMidnightRenewalCallback#31[s]
CLB-BMR-05 (CB-SRC-1)	`renewalCallbackNeverPullsExternalLoanToken`	A borrow renewal only uses loanToken delivered by the take, never external liquidity. `delta(balance[loanToken][callback]) <= units`	✅	❌ BorrowMidnightRenewalCallback#25 ❌ LendMidnightRenewalCallback#23
CLB-BMR-06 (CB-FINAL-4)	`renewalCannotMoveMoreCollateralThanWithdrawn`	Renewing cannot move more collateral to the new position than was withdrawn from the old one. `srcColOut > 0 AND tgtColIn > 0 => tgtColIn <= srcColOut (same collateral token) (srcColOut = collateral[src] - collateral[src]', tgtColIn = collateral[tgt]' - collateral[tgt])`	✅	❌ CollateralTransferLib#3
CLB-BMR-07	`renewalCanMoveDebtBetweenMarkets`	A renewal can actually move debt from one market to another. `satisfy(mnDebt[src][u]' < mnDebt[src][u] AND mnDebt[tgt][u]' > mnDebt[tgt][u])`	✅	❌ BorrowMidnightRenewalCallback#1 ❌ BorrowMidnightRenewalCallback#8
CLB-BMR-08	`renewalCanMigrateCollateralBetweenMarkets`	A renewal can actually migrate collateral between two markets. `satisfy(mnCollateral[src][u][i]' < mnCollateral[src][u][i] AND mnCollateral[tgt][u][j]' > mnCollateral[tgt][u][j])`	✅	❌ CollateralTransferLib#4
CLB-BMR-09 (CB-CLOSE-1)	`renewalCanFullyCloseOldPosition`	A renewal can fully close the old position (both debt and collateral go to zero). `satisfy(mnDebt[src][u]' == 0 AND mnCollateral[src][u][i]' == 0) (pre: both > 0)`	✅	❌ CollateralTransferLib#1
CLB-BMR-10 (CB-RATE-1)	`borrowerFeeBoundedByInterestShare`	The borrower's callback fee never exceeds feeRate applied to the interest portion of the trade, so the effective seller rate stays within (1 + feeRate/WAD) of the offer rate. `f*WAD^2 <= units*(WAD - price)*feeRate (+ one-unit ceil rounding slack)`	⏱	❌ BorrowBlueToMidnightCallback#18 ❌ BorrowMidnightRenewalCallback#36

Property	Name	Description	Status	Mutations
CLB-BMR-11 (CB-FEE-4)	`tickFeeVanishesAtPar`	At par (price == WAD) with full-value settlement (assets == units) the seller tick fee vanishes (carved from interest, not principal). `price==WAD && assets==units && !reverted => feeRecipient delta == 0`	✅	❌ CallbackLib#1 ❌ CallbackLib#4 ❌ CallbackLib#5 ❌ CallbackLib#6
CLB-BMR-12 (CB-SAME-1)	`callbackRevertsForSameSourceMarket`	A renewal into the same Midnight market is rejected. `toId(callbackData.sourceMarket) == marketId => REVERTS (SameMarket)`	✅	❌ BorrowMidnightRenewalCallback#3 ❌ LendMidnightRenewalCallback#24
CLB-BMR-13 (CB-DUST-1)	`receiverNotCallbackReverts`	The renewal onSell requires the sale proceeds to route to the callback itself (else the sellerAssets it needs to repay the source debt are stranded). `receiver != address(callback) => REVERTS (InvalidReceiver)`	✅	❌ BorrowBlueToMidnightCallback#22 ❌ BorrowMidnightRenewalCallback#13 ❌ LendMidnightToVaultCallback#11 ❌ MidnightSupplyVaultSharesCallback#10

LendVaultToMidnightCallback (LVM)

Property	Name	Description	Status	Mutations
CLB-LVM-01 (CB-SRC-1)	`vaultFundedLendOnlyMovesLoanToken`	A vault-funded lend moves Midnight's balance of any non-vault, non-loanToken asset only by the trading fee it accrues. `t != loanToken AND ERC4626asset[t] == 0 => delta(balance[t][Midnight]) == delta(claimableSettlementFee[t])`	⏱	❌ LendVaultToMidnightCallback#9
CLB-LVM-02	`vaultFundedLendLeavesCollateralUnchanged`	Lending funded from a vault never touches anyone's collateral. `collateral[id][u][i]' == collateral[id][u][i]`	✅	❌ LendVaultToMidnightCallback#7
CLB-LVM-03	`vaultFundedLendCanRaiseCredit`	A vault-funded lend can actually increase a lender's credit. `satisfy(credit[id][u]' > credit[id][u])`	✅	❌ LendVaultToMidnightCallback#5
CLB-LVM-04 (CB-DIR-1)	`vaultFundedLendNeverTouchesUnrelatedUser`	A vault-funded lend never touches an unrelated user's credit or debt. `u bystander => credit[id][u]' == credit[id][u] AND debt[id][u]' == debt[id][u]`	✅	❌ LendVaultToMidnightCallback#12
CLB-LVM-05 (CB-RATE-2)	`lenderFeeBoundedByInterestShare`	The lender's callback overcharge never exceeds feeRate on the trade's interest portion, so the effective lender rate stays within (1 - feeRate/WAD) of the offer rate. `f*WAD^2 <= units*(WAD - price)*feeRate (+ one-unit floor rounding slack)`	⏱	❌ LendVaultToMidnightCallback#11
CLB-LVM-06 (CB-FEE-4)	`tickFeeVanishesAtPar`	At par (price == WAD) with full-value settlement (assets == units) the buyer tick fee vanishes (carved from interest, not principal). `price==WAD && assets==units && !reverted => feeRecipient delta == 0`	✅	❌ CallbackLib#1 ❌ CallbackLib#4 ❌ CallbackLib#5 ❌ CallbackLib#6
CLB-LVM-07 (CB-DUST-1)	`vaultAssetMismatchReverts`	Vault-funded onBuy rejects any vault whose asset() differs from the market loanToken. `vault.asset() != loanToken => REVERTS (TokenMismatch)`	✅	❌ LendMidnightToVaultCallback#3 ❌ LendVaultToMidnightCallback#4 ❌ MidnightSupplyVaultSharesCallback#4

LendMidnightToVaultCallback (LMV)

Property	Name	Description	Status	Mutations
CLB-LMV-01 (CB-SRC-1)	`vaultExitConservesMidnightBalanceMinusFee`	Exiting credit into a vault moves Midnight's balance of any non-vault, non-loanToken asset only by the trading fee it accrues. `t != loanToken AND ERC4626asset[t] == 0 => delta(balance[t][Midnight]) == delta(claimableSettlementFee[t])`	⏱	❌ LendMidnightToVaultCallback#20
CLB-LMV-02	`vaultExitLeavesCollateralUnchanged`	Exiting a credit position into a vault never touches anyone's collateral. `collateral[id][u][i]' == collateral[id][u][i]`	✅	❌ LendMidnightToVaultCallback#21
CLB-LMV-03 (CB-DIR-1)	`vaultExitNeverTouchesUnrelatedUser`	A vault exit never touches the credit or debt of an unrelated user. `u bystander => credit[id][u]' == credit[id][u] AND debt[id][u]' == debt[id][u]`	✅	❌ LendMidnightToVaultCallback#13
CLB-LMV-04 (CB-CLOSE-1)	`vaultExitCanFullyCloseCredit`	An exit-to-vault can fully close a lender's source credit (zero remaining). `satisfy(credit[id][u]' == 0) (pre: credit[id][u] > 0)`	✅	❌ LendMidnightToVaultCallback#7

Property	Name	Description	Status	Mutations
CLB-LMV-05 (CB-DUST-1)	`receiverNotCallbackReverts`	The vault-exit onSell requires the sale proceeds to route to the callback itself (else the sellerAssets it needs to fund the vault deposit are stranded). `receiver != address(callback) => REVERTS (InvalidReceiver)`	✅	❌ BorrowBlueToMidnightCallback#22 ❌ BorrowMidnightRenewalCallback#13 ❌ LendMidnightToVaultCallback#11 ❌ MidnightSupplyVaultSharesCallback#10
CLB-LMV-06 (CB-DUST-1)	`vaultAssetMismatchReverts`	Vault-exit onSell rejects any vault whose asset() differs from the market loanToken. `vault.asset() != loanToken => REVERTS (TokenMismatch)`	✅	❌ LendMidnightToVaultCallback#3 ❌ LendVaultToMidnightCallback#4 ❌ MidnightSupplyVaultSharesCallback#4

LendMidnightRenewalCallback (LMR)

Property	Name	Description	Status	Mutations
CLB-LMR-01 (CB-DIR-1)	`renewalAddsCreditOnAtMostOneMarket`	A renewal can add credit on at most one market. `NOT( credit[idA][u]' > credit[idA][u] AND credit[idB][u]' > credit[idB][u] )`	✅	❌ LendMidnightRenewalCallback#21[s]
CLB-LMR-02 (CB-DIR-1)	`renewalReducesCreditOnAtMostOneMarket`	A renewal can reduce credit on at most one market. `NOT( credit[idA][u]' < credit[idA][u] AND credit[idB][u]' < credit[idB][u] )`	⏱	❌ LendMidnightRenewalCallback#19
CLB-LMR-03 (CB-SRC-1)	`renewalCallbackNeverPullsExternalLoanToken`	A renewal only uses loanToken delivered by the take, never external liquidity. `delta(balance[loanToken][callback]) <= units`	✅	❌ BorrowMidnightRenewalCallback#25 ❌ LendMidnightRenewalCallback#23
CLB-LMR-04 (CB-DIR-1)	`renewalNeverTouchesUnrelatedLenderCredit`	A renewal never touches the credit of an unrelated lender. `u bystander => credit[id][u]' == 0`	✅	❌ LendMidnightRenewalCallback#25[s]
CLB-LMR-05	`renewalCanFullyCloseOldCredit`	A renewal can fully close an old credit position. `satisfy(credit[id][u]' == 0) (pre: credit[id][u] > 0)`	✅	❌ LendMidnightRenewalCallback#16
CLB-LMR-06	`renewalCanMoveCreditWithPositiveFee`	A renewal can move credit and charge a positive fee at the same time. `satisfy(credit[src]' < credit[src] AND credit[tgt]' > credit[tgt] AND srcLoss > tgtGain)`	✅	❌ LendMidnightRenewalCallback#17
CLB-LMR-07 (CB-FEE-4)	`tickFeeVanishesAtPar`	At par (price == WAD) with full-value settlement (assets == units) the buyer tick fee vanishes (carved from interest, not principal). `price==WAD && assets==units && !reverted => feeRecipient delta == 0`	✅	❌ CallbackLib#1 ❌ CallbackLib#4 ❌ CallbackLib#5 ❌ CallbackLib#6
CLB-LMR-08 (CB-SAME-1)	`callbackRevertsForSameSourceMarket`	A renewal into the same Midnight market is rejected. `toId(callbackData.sourceMarket) == marketId => REVERTS (SameMarket)`	✅	❌ BorrowMidnightRenewalCallback#3 ❌ LendMidnightRenewalCallback#24

MidnightSupplyCollateralCallback (MSC)

Property	Name	Description	Status	Mutations
CLB-MSC-01	`supplyMonotoneCollateral`	A supply take never decreases anyone's collateral (the callback only supplies, never withdraws). `collateral[id][u][i]' >= collateral[id][u][i]`	✅	❌ MidnightSupplyCollateralCallback#21 ❌ MidnightSupplyVaultSharesCallback#12
CLB-MSC-02	`bystanderUntouched`	A supply take never touches a non-participant's collateral, debt, or credit. `u != taker AND u != maker => collateral/debt/credit[id][u]' == ..[id][u]`	✅	❌ MidnightSupplyCollateralCallback#20 ❌ MidnightSupplyVaultSharesCallback#22
CLB-MSC-03	`proRataUpperBound`	A partial fill never supplies more collateral than the configured per-slot amount. `collateral[seller][i]' - collateral[seller][i] <= amounts[i]`	⏱	❌ MidnightSupplyCollateralCallback#18
CLB-MSC-04 (CB-SC-CAP-1)	`borrowCapacityUsageWithinCap`	After a supply fill, the borrower's borrow-capacity usage stays within maxBorrowCapacityUsage — debt over lltv-weighted borrowing capacity (Midnight's isHealthy() ratio), not raw LTV (the health gate). The on-chain `ceil(debt'*WAD / capacity') <= maxBCU` is asserted as the equivalent product form. `maxBCU > 0 AND debt' > 0 => debt' * WAD <= maxBCU * capacity'` (capacity' = mulDivDown(mulDivDown(col0', price0, ORACLE_PRICE_SCALE), lltv0, WAD))	⏱	❌ MidnightSupplyCollateralCallback#23

Property	Name	Description	Status	Mutations
CLB-MSC-05 (CB-SC-CAP-1 liveness)	`maxBorrowCapacityUsageFillReachable`	A maxBorrowCapacityUsage-guarded supply fill can succeed with rising collateral and live debt. `satisfy(maxBCU > 0 AND collateral[seller][0]' > collateral[seller][0] AND debt[seller]' > 0)`	✅	❌ MidnightSupplyCollateralCallback#13
CLB-MSC-06	`supplyCanRaiseCollateral`	A supply fill can actually raise the borrower's collateral. `satisfy(collateral[seller][0]' > collateral[seller][0])`	✅	❌ MidnightSupplyCollateralCallback#9
CLB-MSC-07	`collateralLengthMismatchReverts`	The callback rejects an amounts[] array whose length mismatches the market collaterals. `amounts.length != market.collateralParams.length => REVERTS`	✅	❌ MidnightSupplyCollateralCallback#4
CLB-MSC-08	`offerSellerAssetsZeroReverts`	The callback rejects a zero offerSellerAssets (the fill-fraction denominator). `offerSellerAssets == 0 => REVERTS`	✅	❌ MidnightSupplyCollateralCallback#14
CLB-MSC-09 (CB-DUST-2)	`receiverIsCallbackReverts`	The supply onSell rejects routing the sale proceeds to the callback itself — they would be permanently locked. `receiver == address(callback) => REVERTS (InvalidReceiver)`	✅	❌ MidnightSupplyCollateralCallback#10

MidnightSupplyVaultSharesCallback (MSV)

Property	Name	Description	Status	Mutations
CLB-MSV-01	`supplyMonotoneCollateral`	A supply take never decreases anyone's collateral (the callback only supplies, never withdraws). `collateral[id][u][i]' >= collateral[id][u][i]`	✅	❌ MidnightSupplyCollateralCallback#21 ❌ MidnightSupplyVaultSharesCallback#12
CLB-MSV-02	`bystanderUntouched`	A supply take never touches a non-participant's collateral, debt, or credit. `u != taker AND u != maker => collateral/debt/credit[id][u]' == ..[id][u]`	✅	❌ MidnightSupplyCollateralCallback#20 ❌ MidnightSupplyVaultSharesCallback#22
CLB-MSV-03	`onlyVaultSlotReceivesSupply`	Supply lands only on the configured vault slot (slot 0); other collateral slots are untouched. `i != 0 => collateral[seller][i]' == collateral[seller][i]`	✅	❌ MidnightSupplyVaultSharesCallback#14
CLB-MSV-04	`suppliedSharesMatchMintedShares`	Every newly minted vault share becomes the seller's collateral. `delta(collateral[seller][0]) == delta(totalSupply[vault])`	✅	❌ MidnightSupplyVaultSharesCallback#11
CLB-MSV-05	`vaultShareBeneficiaryIsSeller`	Only the seller's vault-share collateral can increase (shares cannot be supplied for a third party). `collateral[u][0]' > collateral[u][0] => u == seller`	✅	❌ MidnightSupplyVaultSharesCallback#15
CLB-MSV-06	`supplyCanRaiseVaultCollateral`	A vault-deposit supply can actually raise the seller's collateral. `satisfy(collateral[seller][0]' > collateral[seller][0])`	✅	❌ MidnightSupplyVaultSharesCallback#8 ❌ MidnightSupplyVaultSharesCallback#9 ❌ MidnightSupplyVaultSharesCallback#18 ❌ MidnightSupplyVaultSharesCallback#20
CLB-MSV-07 (user-fund safety)	`noExtraPullWhenPercentZero`	With additionalDepositPercent == 0 the callback pulls no loanToken from the seller. `additionalDepositPercent == 0 => balance[loanToken][seller]' == balance[loanToken][seller]`	✅	❌ MidnightSupplyVaultSharesCallback#13
CLB-MSV-08	`vaultAssetMismatchReverts`	The callback rejects a vault whose underlying asset is not the loan token. `vault.asset() != loanToken => REVERTS`	✅	❌ LendMidnightToVaultCallback#3 ❌ LendVaultToMidnightCallback#4 ❌ MidnightSupplyVaultSharesCallback#4
CLB-MSV-09	`vaultNotAtIndexReverts`	The callback rejects a vault not listed at the configured collateral index. `collateralParams[collateralIndex].token != vault => REVERTS`	✅	❌ MidnightSupplyVaultSharesCallback#5
CLB-MSV-10 (CB-DUST-1)	`receiverNotCallbackReverts`	The vault-share supply onSell requires the sale proceeds to route to the callback itself (else the sellerAssets it needs to mint the vault shares are stranded). `receiver != address(callback) => REVERTS (InvalidReceiver)`	✅	❌ BorrowBlueToMidnightCallback#22 ❌ BorrowMidnightRenewalCallback#13 ❌ LendMidnightToVaultCallback#11 ❌ MidnightSupplyVaultSharesCallback#10
CLB-MSV-11 (user-fund safety)	`extraPullMatchesPercentFormula`	With additionalDepositPercent > 0 the callback pulls exactly the expected extra loanToken from the seller (positive-percent complement of CLB-MSV-07). `additionalDepositPercent > 0 => balance[loanToken][seller] - balance[loanToken][seller]' == mulDivUp(sellerAssets, additionalDepositPercent, WAD)`	✅	❌ MidnightSupplyVaultSharesCallback#21

MidnightWithdrawVaultSharesCallback (MWV)

Property	Name	Description	Status	Mutations
CLB-MWV-01 (CB-VAULT-WD-1)	`takeCanDropCollateralOnNarrowedMarket`	A vault-share withdraw take can actually reduce a position's collateral. `satisfy(collateral[id][u][i]' < collateral[id][u][i])`	✅	❌ MidnightWithdrawVaultSharesCallback#5
CLB-MWV-02 (CB-VAULT-WD-1)	`takeLeavesVaultShareBalanceUnchanged`	A withdraw take leaves the callback's vault-share balance unchanged — onBuy nets the withdrawCollateral share-in against the vault.withdraw share-out. `vaultToken == collateralParams[0].token => balance[vaultToken][callback]' == balance[vaultToken][callback]`	✅	❌ MidnightWithdrawVaultSharesCallback#1 ❌ MidnightWithdrawVaultSharesCallback#8[s]

Migration Ratifier

The MigrationRatifier standalone target is verified directly against its own harness (see Verification Approach), running as six per-category configurations under `certora/confs/ratifier/`. Each rule's `PROPERTIES.md` id is shown in parentheses in the Property column (three rules assert client-code guards that have no catalog entry and carry local ids instead: RTF-WL-1, RTF-CFEE-1, RTF-RC-V1V2); ✅ is a full proof within the target's trust boundary.

MigrationRatifier

The MigrationRatifier production confs run with `rule_sanity: none`; sanity is covered by the `debug_advanced/` variants (`rule_sanity: advanced`). Most rules drive the production entry `isRatified` directly; the four rate-gate monotonicity rules use the `ratifyRate` gate-isolation exposer, and the PriceLib/helper math (RTF-UT-05..10, PRICE-1..4 + ORCH-4/13) is asserted directly on the exposed real pure functions. External dependencies are summarised with deterministic ghosts (see Standalone-Target Model).

Valid state (`ratifier/valid_state.spec`) — inductive invariants on the stored fee configuration

Property	Name	Description	Status	Mutations
RTF-VS-01 (ORCH-1)	`feeRateNeverExceedsCallbackCap`	no reachable state stores an above-cap fee rate; setFeeConfig's guard is the inductive step. `invariant: feeConfigs[cb][id].feeRate <= cap(cb) (cap = 0 on the V2->V1 exits, 0.5e18 otherwise)`	✅	❌ BaseMigrationRatifier#39
RTF-VS-02 (ORCH-2)	`nonZeroFeeRateImpliesRecipient`	a stored non-zero fee rate always carries a recipient. `invariant: feeConfigs[cb][id].feeRate > 0 => feeConfigs[cb][id].feeRecipient != 0`	✅	❌ BaseMigrationRatifier#40

Access control (`ratifier/access_control.spec`) — owner-/authorization-gated storage mutations

Property	Name	Description	Status	Mutations
RTF-AC-01 (ORCH-1)	`feeConfigChangeRequiresOwner`	only the owner can change a stored fee config (access-control facet of the ORCH-1 fee-config family; the value-bound is RTF-VS-01). `feeConfigs[cb][id]' != feeConfigs[cb][id] => msg.sender == owner`	✅	❌ BaseMigrationRatifier#2
RTF-AC-02 (REG-1)	`userParamsChangeRequiresAuthorization`	a user's stored params change only when the caller is onBehalf or Midnight-authorized for them. `userParams[onBehalf][cb][src][tgt]' != userParams[onBehalf][cb][src][tgt] => sender == onBehalf OR isAuthorized(onBehalf, sender)`	✅	❌ MigrationRatifier#8

Reverts (`ratifier/revert.spec`) — implication revert characterizations of the entry guards and validation gates, driven through `isRatified`

Property	Name	Description	Status	Mutations
RTF-RV-01 (ORCH-NEW-8, InvalidRatifierData)	`invalidRatifierDataLengthReverts`	isRatified rejects ratifierData that is not exactly 64 bytes (the abi.encode(src,tgt) length). `ratifierData.length != 64 => revert(isRatified)`	✅	❌ MigrationRatifier#9
RTF-RV-02 (ORCH-NEW-6, InvalidReceiver)	`makerReceiverMustBePinned`	isRatified rejects an offer whose maker-seller receiver is not pinned — address(0) on a buy, offer.callback on a sell. `receiverIfMakerIsSeller != (offer.buy ? 0 : offer.callback) => revert(isRatified)`	✅	❌ MigrationRatifier#10
RTF-RV-03 (ORCH-NEW-7, InvalidGroup)	`migrationGroupNamespaceEnforced`	isRatified rejects an offer whose group is outside the reserved migration-group namespace. `(offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER => revert(isRatified)`	✅	❌ MigrationRatifier#11
RTF-RV-04	`unconfiguredTupleAlwaysReverts`	isRatified reverts when the stored params tuple is unconfigured or malformed. `policy==0 || minDuration==0 || maxDuration<minDuration => revert(isRatified)`	✅	❌ BaseMigrationRatifier#10
RTF-RV-05 (DEFAULT-4)	`tickMustMatchOffer`	for V2→V2 (BMR/LMR) and V1→V2 (BBM/LVM) callbacks, callbackData.tick must equal offer.tick (V2→V1 exits exempt). `(isV2ToV2(cb) || isV1ToV2(cb)) && cd.tick != offer.tick => revert(isRatified)`	✅	❌ BaseMigrationRatifier#25
RTF-RV-06 (ORCH-9)	`targetMaturityMustExceedSource`	isRatified rejects a target maturity that does not strictly exceed the source. `targetMaturity > 0 && targetMaturity <= sourceMaturity => revert(isRatified)`	✅	❌ BaseMigrationRatifier#13
RTF-RV-07 (ORCH-10)	`targetMaturityWithinDurationBand`	isRatified rejects a target maturity outside the stored [now+minDuration, now+maxDuration] band. `tgtMat>0 && (tgtMat < now+minDuration || tgtMat > now+maxDuration) => revert(isRatified)`	✅	❌ BaseMigrationRatifier#44
RTF-RV-08 (ORCH-11)	`targetMaturityOnCadenceGrid`	with a cadence configured, isRatified rejects a target maturity off the cadence grid. `tgtMat>0 && renewalCadence != 0 && cadencePeriodStart(tgtMat) != tgtMat => revert(isRatified)`	✅	❌ BaseMigrationRatifier#15
RTF-RV-09 (DEFAULT-3)	`ratifierDataMustMatchCallbackMarkets`	isRatified rejects an offer whose ratifierData markets disagree with the callback-derived markets. `(cSrc, cTgt) != (ratifierSrc, ratifierTgt) => revert(isRatified)`	✅	❌ MigrationRatifier#5
RTF-RV-10 (DEFAULT-2)	`callbackFeeMustMatchEffectiveConfig`	isRatified rejects callbackData whose fee fields disagree with the effective fee config. `(cFeeRate, cFeeRecip) != getEffectiveFeeConfig(cb, feeMarketId) => revert(isRatified)`	✅	❌ BaseMigrationRatifier#11 ❌ BaseMigrationRatifier#12
RTF-RV-11 (ORCH-8)	`v2SourceWindowEnforcedBeforeOpen`	a V2 (Midnight) source taken before its renewal window opens is rejected. `now < srcMat - renewalWindow (mathint) => revert(isRatified)`	✅	❌ BaseMigrationRatifier#45
RTF-RV-12 (ORCH-7)	`variableSourceWindowEnforced`	a V1→V2 enter (variable source, sourceMaturity==0) needs a configured cadence whose nearest boundary is not in the future. `isV1ToV2(cb) && (cad==0 || cadencePeriodStart(now)>now) => revert(isRatified)`	✅	❌ BaseMigrationRatifier#36
RTF-RV-13 (ORCH-10)	`targetMaturityWithinDurationBand_boundaryAccepted`	satisfy companion: a target maturity exactly at now+minDuration (inclusive lower band edge) is acceptable. `satisfy !revert(isRatified)`	✅	❌ BaseMigrationRatifier#22
RTF-RV-14 (ORCH-8)	`v2SourceWindowEnforcedBeforeOpen_boundaryAccepted`	satisfy companion: a fixed V2 source with renewalWindow == sourceMaturity (window opens at time 0) is acceptable. `satisfy !revert(isRatified)`	✅	❌ BaseMigrationRatifier#23
RTF-RV-15 (ORCH-7)	`variableSourceWindowEnforced_boundaryAccepted`	satisfy companion: a V1→V2 enter whose nearest cadence boundary is exactly now is ratifiable. `satisfy !revert(isRatified)`	✅	❌ BaseMigrationRatifier#24
RTF-RV-16 (RTF-WL-1)	`unauthorizedCallbackReverts`	isRatified rejects an offer whose callback is not one of the six authorized migration callbacks (the route whitelist; the callback-context decoder's final else-branch). `callback not in {BMR,LMR,BBM,LVM,BMB,LMV} => revert(isRatified)`	✅	❌ BaseMigrationRatifier#43
RTF-RV-17 (RTF-CFEE-1)	`continuousFeeCapReverts`	on the two Midnight-target lend flows (LVM enter, LMR renewal) isRatified rejects a target whose lifetime continuous fee would consume the whole WAD face value. `cb in {LVM,LMR} && continuousFee != 0 && continuousFee*zeroFloorSub(maturity,now) >= WAD => revert(isRatified)`	✅	❌ BaseMigrationRatifier#47

Unit (`ratifier/unit.spec`) — write-fidelity, the rate-gate directionality map, the per-callback helpers, and the PriceLib rate-limit math

Property	Name	Description	Status	Mutations
RTF-UT-01 (ORCH-3)	`getEffectiveFeeConfigMarketOverridesActionDefault`	getEffectiveFeeConfig returns the market slot when its recipient is set, else the bytes32(0) default. `cfg[cb][id].recipient != 0 ? cfg[cb][id] : cfg[cb][bytes32(0)]`	✅	❌ BaseMigrationRatifier#18
RTF-UT-02 (ORCH-15, REG-2)	`setParamsWritesTupleAndLeavesOthers`	setParams writes exactly the addressed tuple and leaves every other tuple untouched. `userParams[u][cb][s][t]' == p && (u2,cb2,s2,t2) != (u,cb,s,t) => userParams[u2][cb2][s2][t2]' unchanged`	✅	❌ MigrationRatifier#2
RTF-UT-03 (REG-3)	`clearParamsZeroesTupleAndLeavesOthers`	clearParams zeroes the addressed tuple and leaves every other tuple untouched. `userParams[u][cb][s][t]' == 0 && (u2,cb2,s2,t2) != (u,cb,s,t) => userParams[u2][cb2][s2][t2]' unchanged`	✅	❌ MigrationRatifier#3
RTF-UT-04 (DEFAULT-1, RATE-3)	`userIsBuyMatchesBuySideCallbacks`	the rate-gate buy-side flag is set for exactly the three Midnight-buy callbacks. `_userIsBuy(cb) <=> cb in { LEND_VAULT_TO_MIDNIGHT, BORROW_MIDNIGHT_TO_BLUE, LEND_MIDNIGHT_RENEWAL }`	✅	❌ BaseMigrationRatifier#19
RTF-UT-05 (PRICE-1)	`priceFollowsZeroCouponFormula`	computePrice == WAD^2/(WAD + rate*dur), in (0, WAD], floored for the buyer / ceiled for the seller. `buy == floor(WAD^2/denom) && sell == ceil(WAD^2/denom) && 0 < price <= WAD (denom = WAD + rate*dur)`	✅	❌ PriceLib#1 ❌ PriceLib#2
RTF-UT-06 (PRICE-2)	`priceRoundsInProtectedUserFavor`	rounding favors each side — the buyer's (floor) price never exceeds the seller's (ceil) price. `computePrice(true, rate, dur) <= computePrice(false, rate, dur)`	✅	❌ PriceLib#1
RTF-UT-07 (PRICE-3)	`effectiveRateSelectsTighterBound`	computeEffectiveRate selects the tighter bound — min for the borrower, max for the lender. `computeEffectiveRate(false,p,l) == min(p,l) && computeEffectiveRate(true,p,l) == max(p,l)`	✅	❌ PriceLib#3
RTF-UT-08 (PRICE-4)	`satisfiesRateLimitComparisonDirection`	satisfiesRateLimit enforces the borrower ceiling (assets*WAD* >= *units*price) and the lender floor (<=). `satisfies(false,..) <=> a*WAD >= u*priceBorrow && satisfies(true,..) <=> a*WAD <= u*priceLend`	✅	❌ PriceLib#4 ❌ PriceLib#7
RTF-UT-09 (ORCH-4)	`maxFeeRateZeroOnV2ToV1Exits`	the per-callback fee-rate cap is zero on the V2→V1 exits (BMB/LMV), MAX_FEE_RATE otherwise. `_maxFeeRate(cb) == (isV2ToV1(cb) ? 0 : MAX_FEE_RATE)`	✅	❌ BaseMigrationRatifier#20
RTF-UT-10 (ORCH-13)	`computeDurationPerCallback`	per-callback accrual duration; the V2→V1 exit clamps to 0 once the source has matured. `renewal: tgt - max(now,src) ; enter: tgt - now ; exit: now>=src ? 0 : src-now`	✅	❌ BaseMigrationRatifier#21
RTF-UT-11 (decomposition, lifts RTF-HL-04)	`netSellerPriceMonotoneInFee`	the net seller price is non-increasing in the fee rate (a borrower enter prices the offer down as the fee grows). `feeLo <= feeHi => netSellerPrice(p, sf, feeLo) >= netSellerPrice(p, sf, feeHi)`	✅	❌ CallbackLib#3 ❌ RouterLib#1
RTF-UT-12 (borrower limit-monotonicity — establishes RATE-1 directly)	`satisfiesRateLimitMonotoneInBorrowerLimit`	the borrower rate gate is monotone in the limit — a higher limit only loosens acceptance. `limLo <= limHi => ( satisfies(false,u,a,limLo,pol,dur) => satisfies(false,u,a,limHi,pol,dur) )`	✅	❌ PriceLib#7
RTF-UT-13 (decomposition, lifts RTF-HL-05)	`netBuyerPriceMonotoneInFee`	the net buyer price is non-decreasing in the fee rate (a lender enter prices the offer up as the fee grows). `feeLo <= feeHi => netBuyerPrice(p, sf, feeLo) <= netBuyerPrice(p, sf, feeHi)`	✅	❌ CallbackLib#3 ❌ RouterLib#2
RTF-UT-14 (lender limit-monotonicity — establishes RATE-2 directly)	`satisfiesRateLimitMonotoneInLenderLimit`	the lender rate gate is monotone in the limit — a higher limit only tightens acceptance. `limLo <= limHi => ( satisfies(true,u,a,limHi,pol,dur) => satisfies(true,u,a,limLo,pol,dur) )`	✅	❌ PriceLib#4
RTF-UT-15	`isRatifiedReturnsCallbackSuccess`	isRatified returns the Midnight success token on every accepting (non-reverting) path - the producer side of the take() handshake (the midnight suite proves the consumer side against an untrusted-ratifier summary). `!revert(isRatified) => return == keccak256("morpho.midnight.callbackSuccess")`	✅	❌ MigrationRatifier#13
RTF-UT-16	`setFeeConfigWritesSlotAndLeavesOthers`	setFeeConfig stores exactly the addressed (callback, tenorMarketId) fee slot and leaves every other fee slot untouched. `feeConfigs[cb][id]' == (recipient, rate) && (cb2,id2) != (cb,id) => feeConfigs[cb2][id2]' unchanged`	✅	❌ BaseMigrationRatifier#48

High-level (`ratifier/highlevel.spec`) — window/cadence & own-storage differentials, fee-slot isolation, rate-gate monotonicity

Property	Name	Description	Status	Mutations
RTF-HL-01 (ORCH-7)	`v1v2migrationsHaveNoRenewalWindowConstraint`	V1→V2 migrations (BBM, LVM) have no renewal window constraint. `revert(isRatified | renewalWindow=w1) == revert(isRatified | renewalWindow=w2)`	✅	❌ BaseMigrationRatifier#31

Property	Name	Description	Status	Mutations
RTF-HL-02	`v2v1ExitsHaveNoRenewalCadenceConstraint`	V2→V1 exits (BMB, LMV) have no renewal cadence constraint. `revert(isRatified | renewalCadence=c1) == revert(isRatified | renewalCadence=c2)`	✅	❌ BaseMigrationRatifier#1
RTF-HL-03 (ORCH-14)	`feeMarketIdIgnoresCrossMarketSlot`	only the selected fee slot gates the verdict; any other market's slot is ignored. `idX != feeMarketId && idX != 0 => revert(isRatified | cfg[cb][idX]=A) == revert(... =B)`	✅	❌ BaseMigrationRatifier#41
RTF-HL-04E (decomposition bridge, RTF-HL-04)	`borrowerRateGateMatchesNetSellerThreshold`	the real borrower (BBM) rate gate reverts exactly when the reconstructed net-seller threshold fails — one real solve that RTF-HL-04 then lifts monotonically. `revert(ratifyRate) <=> !satisfiesRateLimit(false, WAD, netSellerPrice(tickPrice, 0, feeRate), limit, policy, dur)`	✅	❌ BaseMigrationRatifier#32 ❌ BaseMigrationRatifier#33
RTF-HL-04 (CB-RATE-1)	`higherFeeOnlyTightensBorrowerRateGate`	on a borrower enter (BBM, sell side) a larger fee only tightens the rate gate. `feeRate_lo <= feeRate_hi => ( revert(ratifyRate | lo) => revert(ratifyRate | hi) )`	✅	❌ PriceLib#8
RTF-HL-05E (decomposition bridge, RTF-HL-05)	`lenderRateGateMatchesNetBuyerThreshold`	the real lender (LVM) rate gate reverts exactly when the reconstructed net-buyer threshold fails — one real solve that RTF-HL-05 then lifts monotonically. `revert(ratifyRate) <=> !satisfiesRateLimit(true, WAD, netBuyerPrice(tickPrice, 0, feeRate), limit, policy, dur)`	✅	❌ BaseMigrationRatifier#32 ❌ BaseMigrationRatifier#33
RTF-HL-05 (CB-RATE-2)	`higherFeeOnlyTightensLenderRateGate`	on a lender enter (LVM, buy side) a larger fee only tightens the rate gate. `feeRate_lo <= feeRate_hi => ( revert(ratifyRate | lo) => revert(ratifyRate | hi) )`	✅	❌ PriceLib#9
RTF-HL-06	`isRatifiedReadsOnlyAddressedParams`	only the addressed userParams[offer.maker][offer.callback][src][tgt] tuple gates the verdict; other tuples cannot. `(u2,cb2,s2,t2) != (offer.maker,offer.callback,iSrc,iTgt) => revert(isRatified | set other tuple) == revert(...)`	✅	❌ MigrationRatifier#12
RTF-HL-07	`getRatePrincipalForwardedFaithfully`	the rate policy is consulted for the right principal — offer.maker as the policy user. `!revert(isRatified) => getRate user == offer.maker`	✅	❌ BaseMigrationRatifier#42

Reachability (`ratifier/reachability.spec`) — `satisfy()` witnesses that post-maturity takes stay executable

Property	Name	Description	Status	Mutations
RTF-RC-01 (ORCH-5)	`postMaturityV2ToV2Executable`	a V2->V2 renewal taken at or after source maturity still has an accepting execution — post-maturity timing alone never blocks it (the renewal window is satisfied once now >= sourceMaturity).	✅	❌ BaseMigrationRatifier#28
RTF-RC-02 (ORCH-6)	`postMaturityV2ToV1Executable`	a V2->V1 exit taken at or after source maturity still has an accepting execution — post-maturity the window check is satisfied (now>=sourceMaturity>=renewalPeriodStart); only target-maturity validation is skipped (V1 has no maturity).	✅	❌ BaseMigrationRatifier#29
RTF-RC-03 (RTF-RC-V1V2)	`entryV1ToV2Executable`	a V1->V2 enter (Blue/vault source, live Midnight target) has an accepting execution — completes the RC family (RC-01 V2->V2, RC-02 V2->V1, RC-03 V1->V2).	✅	❌ BaseMigrationRatifier#46

Verification Results

139 properties — 90 across the nine callbacks + shared safety layer and 49 for the standalone Migration Ratifier: 120 verified, 19 timed out (4-hour SMT budget), 0 violated.

Group	Verified	Timeout	Total
Shared Safety Rules (re-verified under each callback)	5	5	10
BorrowBlueToMidnightCallback (BBM)	11	2	13

Group	Verified	Timeout	Total
BorrowMidnightToBlueCallback (BMB)	8	3	11
BorrowMidnightRenewalCallback (BMR)	10	3	13
LendVaultToMidnightCallback (LVM)	5	2	7
LendMidnightToVaultCallback (LMV)	5	1	6
LendMidnightRenewalCallback (LMR)	7	1	8
MidnightSupplyCollateralCallback (MSC)	7	2	9
MidnightSupplyVaultSharesCallback (MSV)	11	0	11
MidnightWithdrawVaultSharesCallback (MWV)	2	0	2
MigrationRatifier (standalone, 47 rules + 2 invariants)	49	0	49
Total	**120**	**19**	**139**

A shared rule counts as timed out if any of its per-callback re-verifications hits the budget: 55 of 63 applicable matrix cells verify (CLB-09 times out in both of its setups).

Mutation Testing

Mutation testing injects deliberate one-line bugs into the verified `src/` contracts and re-runs the properties expected to catch them, measuring how tightly the proofs constrain the implementation. A property that flips to a counterexample **caught** the bug. The results below — each injected bug as a one-line diff and the properties that caught it — are self-contained. Status icons per the legend.

Coverage by rule

Which rule caught which mutations (one rule can catch several). A ˢ marks a catch via the rule's satisfy-twin (the mutation reverts `take()`, so only the witness flags it).

Rule (PROP)	Mutations
`borrowCapacityUsageWithinCap` (CB-SC-CAP-1)	❌ MidnightSupplyCollateralCallback#23
`borrowerFeeBoundedByInterestShare` (CB-RATE-1)	❌ BorrowBlueToMidnightCallback#18 ❌ BorrowMidnightRenewalCallback#36
`borrowerRateGateMatchesNetSellerThreshold` (BBM)	❌ BaseMigrationRatifier#32 ❌ BaseMigrationRatifier#33
`buyerTickFeePaidBoundedByUnits` (CB-FEE-2)	❌ LendMidnightRenewalCallback#14
`bystanderUntouched`	❌ MidnightSupplyCollateralCallback#20 ❌ MidnightSupplyVaultSharesCallback#22
`callbackFeeMustMatchEffectiveConfig` (DEFAULT-2)	❌ BaseMigrationRatifier#11 ❌ BaseMigrationRatifier#12
`callbackHoldsZeroAllowance` (CB-DUST-1)	❌ MidnightWithdrawVaultSharesCallback#2
`callbackHoldsZeroAllowance__satisfy` (CB-DUST-1)	❌ BorrowMidnightRenewalCallback#35ˢ
`callbackNeverHoldsTokens` (CB-DUST-1)	❌ MidnightWithdrawVaultSharesCallback#6
`callbackNeverHoldsTokens__satisfy` (CB-DUST-1)	❌ BorrowMidnightRenewalCallback#33ˢ
`callbackRevertsForNonMidnightCaller` (CB-AUTH-1)	❌ BorrowBlueToMidnightCallback#1 ❌ MidnightSupplyCollateralCallback#1
`callbackRevertsForSameSourceMarket` (CB-SAME-1)	❌ BorrowMidnightRenewalCallback#3 ❌ LendMidnightRenewalCallback#24
`callbackRevertsOnZeroAssetsOrUnits`	❌ LendMidnightRenewalCallback#2 ❌ MidnightSupplyCollateralCallback#2
`clearParamsZeroesTupleAndLeavesOthers` (REG-3)	❌ MigrationRatifier#3
`clearingOldDebtAlsoEmptiesOldCollateral` (CB-FINAL-2)	❌ BorrowBlueToMidnightCallback#3
`collateralLengthMismatchReverts`	❌ MidnightSupplyCollateralCallback#4
`computeDurationPerCallback` (ORCH-13)	❌ BaseMigrationRatifier#21
`continuousFeeCapReverts` (LVM, LMR)	❌ BaseMigrationRatifier#47
`effectiveRateSelectsTighterBound` (PRICE-3)	❌ PriceLib#3
`entryV1ToV2Executable`	❌ BaseMigrationRatifier#46
`extraPullMatchesPercentFormula`	❌ MidnightSupplyVaultSharesCallback#21
`feeConfigChangeRequiresOwner`	❌ BaseMigrationRatifier#2

Rule (PROP)	Mutations
`feeMarketIdIgnoresCrossMarketSlot` (ORCH-14)	❌ BaseMigrationRatifier#41
`feeRateNeverExceedsCallbackCap` (ORCH-1)	❌ BaseMigrationRatifier#39
`feeRecipientNeverLosesTokens__satisfy`	❌ BorrowMidnightRenewalCallback#35[s]
`fullCollateralMigrationClearsAllOldDebt__satisfy` (CB-CLOSE-2)	❌ BorrowBlueToMidnightCallback#12[s]
`getEffectiveFeeConfigMarketOverridesActionDefault` (ORCH-3)	❌ BaseMigrationRatifier#18
`getRatePrincipalForwardedFaithfully`	❌ BaseMigrationRatifier#42
`higherFeeOnlyTightensBorrowerRateGate`	❌ PriceLib#8
`higherFeeOnlyTightensLenderRateGate`	❌ PriceLib#9
`invalidRatifierDataLengthReverts`	❌ MigrationRatifier#9
`isRatifiedReadsOnlyAddressedParams`	❌ MigrationRatifier#12
`isRatifiedReturnsCallbackSuccess`	❌ MigrationRatifier#13
`lenderFeeBoundedByInterestShare` (CB-RATE-2)	❌ LendVaultToMidnightCallback#11
`lenderRateGateMatchesNetBuyerThreshold` (LVM)	❌ BaseMigrationRatifier#32 ❌ BaseMigrationRatifier#33
`makerReceiverMustBePinned`	❌ MigrationRatifier#10
`maxBorrowCapacityUsageFillReachable`	❌ MidnightSupplyCollateralCallback#13
`maxFeeRateZeroOnV2ToV1Exits` (ORCH-4)	❌ BaseMigrationRatifier#20
`migrationCanFullyCloseOldPosition` (CB-CLOSE-1)	❌ BorrowBlueToMidnightCallback#21 ❌ BorrowMidnightToBlueCallback#10
`migrationCanMoveCollateralBlueToMidnight`	❌ BorrowBlueToMidnightCallback#2
`migrationCanMoveCollateralMidnightToBlue`	❌ BorrowMidnightToBlueCallback#9
`migrationCanOpenNewBlueDebt`	❌ BorrowMidnightToBlueCallback#8
`migrationCannotDepositMoreCollateralThanWithdrawn` (CB-SRC-1)	❌ BorrowMidnightToBlueCallback#24
`migrationConservesMigratedCollateral` (CB-DIR-1)	❌ BorrowBlueToMidnightCallback#4
`migrationFinalFillTransfersAllOldMidnightCollateral` (CB-FINAL-3)	❌ BorrowMidnightToBlueCallback#25
`migrationGroupNamespaceEnforced`	❌ MigrationRatifier#11
`migrationOnlyAddsNewBlueCollateral` (CB-DIR-1)	❌ BorrowMidnightToBlueCallback#26
`migrationOnlyAddsNewMidnightCollateral` (CB-DIR-1)	❌ BorrowBlueToMidnightCallback#9
`migrationOnlyOpensNewBlueDebt` (CB-DIR-1)	❌ BorrowMidnightToBlueCallback#27
`migrationOnlyReducesOldBlueDebt` (CB-V1-REP-1)	❌ BorrowBlueToMidnightCallback#15
`migrationOnlyWithdrawsOldBlueCollateral` (CB-DIR-1)	❌ BorrowBlueToMidnightCallback#16
`migrationOnlyWithdrawsOldMidnightCollateral` (CB-DIR-1)	❌ BorrowMidnightToBlueCallback#20

Rule (PROP)	Mutations
`migrationReducesOldDebtOnAtMostOneMarket__satisfy` (CB-DIR-1)	❌ BorrowBlueToMidnightCallback#20[s] ❌ BorrowMidnightToBlueCallback#29[s]
`netBuyerPriceMonotoneInFee`	❌ CallbackLib#3 ❌ RouterLib#2
`netSellerPriceMonotoneInFee`	❌ CallbackLib#3 ❌ RouterLib#1
`noExtraPullWhenPercentZero`	❌ MidnightSupplyVaultSharesCallback#13
`nonZeroFeeRateImpliesRecipient` (ORCH-2)	❌ BaseMigrationRatifier#40
`offerSellerAssetsZeroReverts`	❌ MidnightSupplyCollateralCallback#14
`oldMidnightDebtAndNewBlueDebtMoveTogether` (CB-DIR-1)	❌ BorrowMidnightToBlueCallback#30
`onlyVaultSlotReceivesSupply`	❌ MidnightSupplyVaultSharesCallback#14
`percentageFeeNeverExceedsAssets` (CB-FEE-3)	❌ LendMidnightToVaultCallback#10
`percentageFeeRateAboveCapReverts`	❌ BorrowMidnightToBlueCallback#18
`positiveFeeIsPayable`	❌ LendMidnightRenewalCallback#8
`postMaturityV2ToV1Executable` (ORCH-6)	❌ BaseMigrationRatifier#29
`postMaturityV2ToV2Executable` (ORCH-5)	❌ BaseMigrationRatifier#28
`priceFollowsZeroCouponFormula` (PRICE-1)	❌ PriceLib#1 ❌ PriceLib#2
`priceRoundsInProtectedUserFavor` (PRICE-2)	❌ PriceLib#1
`proRataUpperBound`	❌ MidnightSupplyCollateralCallback#18
`ratifierDataMustMatchCallbackMarkets` (DEFAULT-3)	❌ MigrationRatifier#5
`receiverIsCallbackReverts`	❌ MidnightSupplyCollateralCallback#10
`receiverNotCallbackReverts`	❌ BorrowBlueToMidnightCallback#22 ❌ BorrowMidnightRenewalCallback#13 ❌ LendMidnightToVaultCallback#11 ❌ MidnightSupplyVaultSharesCallback#10
`renewalAddsCreditOnAtMostOneMarket__satisfy` (CB-DIR-1)	❌ LendMidnightRenewalCallback#21[s]
`renewalAddsDebtOnAtMostOneMarket__satisfy` (CB-DIR-1)	❌ BorrowMidnightRenewalCallback#34[s]
`renewalCallbackNeverPullsExternalLoanToken` (CB-SRC-1)	❌ BorrowMidnightRenewalCallback#25 ❌ LendMidnightRenewalCallback#23
`renewalCanFullyCloseOldCredit` (CB-CLOSE-1)	❌ LendMidnightRenewalCallback#16
`renewalCanFullyCloseOldPosition` (CB-CLOSE-1)	❌ CollateralTransferLib#1
`renewalCanMigrateCollateralBetweenMarkets`	❌ CollateralTransferLib#4
`renewalCanMoveCreditWithPositiveFee`	❌ LendMidnightRenewalCallback#17
`renewalCanMoveDebtBetweenMarkets`	❌ BorrowMidnightRenewalCallback#1 ❌ BorrowMidnightRenewalCallback#8
`renewalCannotAddCollateralWhenReducingDebt` (CB-DIR-1)	❌ BorrowMidnightRenewalCallback#23
`renewalCannotMoveMoreCollateralThanWithdrawn` (CB-FINAL-4)	❌ CollateralTransferLib#3
`renewalCannotRemoveCollateralWhenOpeningDebt` (CB-DIR-1)	❌ BorrowMidnightRenewalCallback#23

Rule (PROP)	Mutations
`renewalCannotRemoveCollateralWhenOpeningDebt__satisfy` (CB-DIR-1)	❌ BorrowMidnightRenewalCallback#31[s]
`renewalNeverTouchesUnrelatedLenderCredit__satisfy` (CB-DIR-1)	❌ LendMidnightRenewalCallback#25[s]
`renewalReducesCreditOnAtMostOneMarket` (CB-DIR-1)	❌ LendMidnightRenewalCallback#19
`renewalReducesDebtOnAtMostOneMarket` (CB-DIR-1)	❌ BorrowMidnightRenewalCallback#24
`satisfiesRateLimitComparisonDirection` (PRICE-4)	❌ PriceLib#4 ❌ PriceLib#7
`satisfiesRateLimitMonotoneInBorrowerLimit`	❌ PriceLib#7
`satisfiesRateLimitMonotoneInLenderLimit`	❌ PriceLib#4
`sellerTickFeeNeverExceedsAssets` (CB-FEE-1)	❌ BorrowMidnightRenewalCallback#22
`setFeeConfigWritesSlotAndLeavesOthers`	❌ BaseMigrationRatifier#48
`setParamsWritesTupleAndLeavesOthers` (ORCH-15, REG-2)	❌ MigrationRatifier#2
`sourceLoanTokenMismatchReverts`	❌ BorrowBlueToMidnightCallback#14
`suppliedSharesMatchMintedShares`	❌ MidnightSupplyVaultSharesCallback#11
`supplyCanRaiseCollateral`	❌ MidnightSupplyCollateralCallback#9
`supplyCanRaiseVaultCollateral`	❌ MidnightSupplyVaultSharesCallback#8 ❌ MidnightSupplyVaultSharesCallback#9 ❌ MidnightSupplyVaultSharesCallback#18 ❌ MidnightSupplyVaultSharesCallback#20
`supplyMonotoneCollateral`	❌ MidnightSupplyCollateralCallback#21 ❌ MidnightSupplyVaultSharesCallback#12
`takeCanDropCollateralOnNarrowedMarket` (CB-VAULT-WD-1)	❌ MidnightWithdrawVaultSharesCallback#5
`takeLeavesVaultShareBalanceUnchanged` (CB-VAULT-WD-1)	❌ MidnightWithdrawVaultSharesCallback#1
`takeLeavesVaultShareBalanceUnchanged__satisfy` (CB-VAULT-WD-1)	❌ MidnightWithdrawVaultSharesCallback#8[s]
`targetMaturityMustExceedSource` (ORCH-9)	❌ BaseMigrationRatifier#13
`targetMaturityOnCadenceGrid` (ORCH-11)	❌ BaseMigrationRatifier#15
`targetMaturityWithinDurationBand` (ORCH-10)	❌ BaseMigrationRatifier#44
`targetMaturityWithinDurationBand_boundaryAccepted` (ORCH-10)	❌ BaseMigrationRatifier#22
`thirdPartyBalanceUnchanged__satisfy`	❌ BorrowMidnightRenewalCallback#35[s]
`tickFeeVanishesAtPar` (CB-FEE-4)	❌ CallbackLib#1 ❌ CallbackLib#4 ❌ CallbackLib#5 ❌ CallbackLib#6
`tickMustMatchOffer` (DEFAULT-4)	❌ BaseMigrationRatifier#25
`unauthorizedCallbackReverts`	❌ BaseMigrationRatifier#43
`unconfiguredTupleAlwaysReverts`	❌ BaseMigrationRatifier#10
`userIsBuyMatchesBuySideCallbacks` (DEFAULT-1, RATE-3)	❌ BaseMigrationRatifier#19
`userParamsChangeRequiresAuthorization` (REG-1)	❌ MigrationRatifier#8

Rule (PROP)	Mutations
`v1v2migrationsHaveNoRenewalWindowConstraint` (ORCH-7)	❌ BaseMigrationRatifier#31
`v2SourceWindowEnforcedBeforeOpen` (ORCH-8)	❌ BaseMigrationRatifier#45
`v2SourceWindowEnforcedBeforeOpen_boundaryAccepted` (ORCH-8)	❌ BaseMigrationRatifier#23
`v2v1ExitsHaveNoRenewalCadenceConstraint` (BMB, LMV)	❌ BaseMigrationRatifier#1
`variableSourceWindowEnforced` (ORCH-7)	❌ BaseMigrationRatifier#36
`variableSourceWindowEnforced_boundaryAccepted` (ORCH-7)	❌ BaseMigrationRatifier#24
`vaultAssetMismatchReverts`	❌ LendMidnightToVaultCallback#3 ❌ LendVaultToMidnightCallback#4 ❌ MidnightSupplyVaultSharesCallback#4
`vaultExitCanFullyCloseCredit` (CB-CLOSE-1)	❌ LendMidnightToVaultCallback#7
`vaultExitConservesMidnightBalanceMinusFee` (CB-SRC-1)	❌ LendMidnightToVaultCallback#20
`vaultExitLeavesCollateralUnchanged`	❌ LendMidnightToVaultCallback#21
`vaultExitNeverTouchesUnrelatedUser` (CB-DIR-1)	❌ LendMidnightToVaultCallback#13
`vaultFundedLendCanRaiseCredit`	❌ LendVaultToMidnightCallback#5
`vaultFundedLendLeavesCollateralUnchanged`	❌ LendVaultToMidnightCallback#7
`vaultFundedLendNeverTouchesUnrelatedUser` (CB-DIR-1)	❌ LendVaultToMidnightCallback#12
`vaultFundedLendOnlyMovesLoanToken` (CB-SRC-1)	❌ LendVaultToMidnightCallback#9
`vaultNotAtIndexReverts`	❌ MidnightSupplyVaultSharesCallback#5
`vaultShareBeneficiaryIsSeller`	❌ MidnightSupplyVaultSharesCallback#15

Each mutation below shows the one-line diff it injects and two copy-paste commands: the rule on clean `src/` (proves) and the same rule with the mutant applied via `certora/mutations/run_mutation.sh` (**VIOLATED** = mutant caught; the script restores `src/` afterwards). For the heaviest rules the commands use a perf-overlay conf (same rule, footprint-stubbed harness — see Methodology); the verdict is unaffected.

`BaseMigrationRatifier` — `src/ratifiers/BaseMigrationRatifier.sol`

❌ BaseMigrationRatifier #1 — _ratifyWindow guard ==0 -> !=0

- **Mutant:** `certora/mutations/BaseMigrationRatifier/1.sol`
- **Caught by:** `v2v1ExitsHaveNoRenewalCadenceConstraint` (BMB, LMV)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule v2v1ExitsHaveNoRenewalCadenceConstraint`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 1`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -271,7 +271,7 @@
         view
         returns (uint256 renewalPeriodStart)
     {
-        if (sourceMaturity == 0) {
+        if (sourceMaturity != 0) {  // MUTATION: rebased
             if (params.renewalCadence == address(0)) revert InvalidRenewalParams();
             renewalPeriodStart =
IRenewalCadence(params.renewalCadence).cadencePeriodStart(block.timestamp);
             // Invariant check: a compliant cadence returns a period start <= the queried
timestamp.
```

❌ BaseMigrationRatifier #2 — Comment out the onlyOwner modifier to allow non-owners to call setFeeConfig

- **Mutant:** `certora/mutations/BaseMigrationRatifier/2.sol`
- **Caught by:** `feeConfigChangeRequiresOwner`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/access_control.conf --rule feeConfigChangeRequiresOwner`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 2`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -84,7 +84,7 @@
     /// @inheritdoc IMigrationRatifier
     function setFeeConfig(address callback, bytes32 tenorMarketId, uint256 _feeRate,
address _feeRecipient)
         external
-        onlyOwner
+        // onlyOwner  // MUTATION: rebased
     {
         if (_feeRate > _maxFeeRate(callback)) revert InvalidFeeConfig();
         if (_feeRate > 0 && _feeRecipient == address(0)) revert InvalidFeeConfig();
```

❌ BaseMigrationRatifier #10 — Flip || to && to allow single unconfigured field

- **Mutant:** `certora/mutations/BaseMigrationRatifier/10.sol`
- **Caught by:** `unconfiguredTupleAlwaysReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule unconfiguredTupleAlwaysReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 10`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -131,7 +131,7 @@
         UserMigrationParams memory params
     ) internal view {
         if (
-            params.interestRatePolicy == address(0) || params.minDuration == 0
+            params.interestRatePolicy == address(0) && params.minDuration == 0  //
MUTATION: rebased
                 || params.maxDuration < params.minDuration
         ) {
             revert InvalidRenewalParams();
```

❌ BaseMigrationRatifier #11 — Invert feeMarketId selection (wrong market for fee lookup)

- **Mutant:** `certora/mutations/BaseMigrationRatifier/11.sol`
- **Caught by:** `callbackFeeMustMatchEffectiveConfig` (DEFAULT-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule callbackFeeMustMatchEffectiveConfig`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 11`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -149,7 +149,7 @@
         _validateMarketPair(src, tgt, callbackSourceMarketId, callbackTargetMarketId);

         // The fee config is keyed on the Midnight market: the target for entries and
renewals, the source for exits.
-        bytes32 feeMarketId = targetMaturity == 0 ? callbackSourceMarketId :
callbackTargetMarketId;
+        bytes32 feeMarketId = targetMaturity != 0 ? callbackSourceMarketId :
callbackTargetMarketId;  // MUTATION: rebased
         FeeConfig memory expectedFee = getEffectiveFeeConfig(callback, feeMarketId);
         if (callbackFeeRate != expectedFee.feeRate || callbackFeeRecipient !=
expectedFee.feeRecipient) {
             revert InvalidFeeConfig();
```

❌ BaseMigrationRatifier #12 — Accept fee rate mismatch when recipient matches

- **Mutant:** `certora/mutations/BaseMigrationRatifier/12.sol`
- **Caught by:** `callbackFeeMustMatchEffectiveConfig` (DEFAULT-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule callbackFeeMustMatchEffectiveConfig`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 12`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -151,7 +151,7 @@
         // The fee config is keyed on the Midnight market: the target for entries and
renewals, the source for exits.
         bytes32 feeMarketId = targetMaturity == 0 ? callbackSourceMarketId :
callbackTargetMarketId;
         FeeConfig memory expectedFee = getEffectiveFeeConfig(callback, feeMarketId);
-        if (callbackFeeRate != expectedFee.feeRate || callbackFeeRecipient !=
expectedFee.feeRecipient) {
+        if (callbackFeeRate == expectedFee.feeRate || callbackFeeRecipient !=
expectedFee.feeRecipient) {  // MUTATION: rebased
             revert InvalidFeeConfig();
         }
```

❌ BaseMigrationRatifier #13 — Allow targetMaturity == sourceMaturity (off-by-one)

- **Mutant:** `certora/mutations/BaseMigrationRatifier/13.sol`
- **Caught by:** `targetMaturityMustExceedSource` (ORCH-9)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule targetMaturityMustExceedSource`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 13`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -290,7 +290,7 @@
         internal
         view
     {
-        if (targetMaturity <= sourceMaturity) revert InvalidTargetMaturity();
+        if (targetMaturity < sourceMaturity) revert InvalidTargetMaturity();  // MUTATION:
rebased
         uint256 minTarget = block.timestamp + params.minDuration;
         uint256 maxTarget = block.timestamp + params.maxDuration;
         if (targetMaturity < minTarget || targetMaturity > maxTarget) {
```

❌ BaseMigrationRatifier #15 — cadence-grid guard != -> == : accepts off-grid maturities, rejects on-grid

- **Mutant:** `certora/mutations/BaseMigrationRatifier/15.sol`
- **Caught by:** `targetMaturityOnCadenceGrid` (ORCH-11)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule targetMaturityOnCadenceGrid`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 15`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -298,7 +298,7 @@
         }
         if (
             params.renewalCadence != address(0)
-                &&
IRenewalCadence(params.renewalCadence).cadencePeriodStart(targetMaturity) != targetMaturity
+                &&
IRenewalCadence(params.renewalCadence).cadencePeriodStart(targetMaturity) == targetMaturity
// MUTATION: rebased
         ) revert InvalidTargetMaturity();
     }
```

❌ BaseMigrationRatifier #18 — Flip the condition from != to == to invert the override logic; market config is returned even when not set, instead of falling back to default

- **Mutant:** `certora/mutations/BaseMigrationRatifier/18.sol`
- **Caught by:** `getEffectiveFeeConfigMarketOverridesActionDefault` (ORCH-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule getEffectiveFeeConfigMarketOverridesActionDefault`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 18`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -101,7 +101,7 @@
         returns (FeeConfig memory config)
     {
         config = feeConfigs[callback][tenorMarketId];
-        if (config.feeRecipient != address(0)) return config;
+        if (config.feeRecipient == address(0)) return config;  // MUTATION: rebased
         return feeConfigs[callback][bytes32(0)];
     }
```

❌ BaseMigrationRatifier #19 — Change the disjunction || to conjunction && so no callback can satisfy the condition, breaking the buy-side flag directionality

- **Mutant:** `certora/mutations/BaseMigrationRatifier/19.sol`
- **Caught by:** `userIsBuyMatchesBuySideCallbacks` (DEFAULT-1, RATE-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule userIsBuyMatchesBuySideCallbacks`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 19`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -374,7 +374,7 @@
     /// @dev The user buys credit on Midnight when entering or renewing a lend position, or
exiting a borrow
     /// position; the user sells when entering or renewing a borrow position, or exiting a
lend position.
     function _userIsBuy(address callback) internal view returns (bool) {
-        return callback == LEND_VAULT_TO_MIDNIGHT_CALLBACK || callback ==
BORROW_MIDNIGHT_TO_BLUE_CALLBACK
+        return callback == LEND_VAULT_TO_MIDNIGHT_CALLBACK && callback ==
BORROW_MIDNIGHT_TO_BLUE_CALLBACK  // MUTATION: rebased
             || callback == LEND_MIDNIGHT_RENEWAL_CALLBACK;
     }
```

❌ **BaseMigrationRatifier #20 — _maxFeeRate exit-cap removed: cap nonzero (MAX_FEE_RATE) on V2->V1 exits instead of 0**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/20.sol`
- **Caught by:** `maxFeeRateZeroOnV2ToV1Exits` (ORCH-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule maxFeeRateZeroOnV2ToV1Exits`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 20`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -109,7 +109,7 @@
     /// otherwise.
     function _maxFeeRate(address callback) internal view returns (uint256) {
         if (callback == BORROW_MIDNIGHT_TO_BLUE_CALLBACK || callback ==
LEND_MIDNIGHT_TO_VAULT_CALLBACK) {
-            return MAX_FEE_RATE_FIXED_TO_VARIABLE;
+            return MAX_FEE_RATE;  // MUTATION: rebased
         }
         return MAX_FEE_RATE;
     }
```

❌ **BaseMigrationRatifier #21 — Replace zeroFloorSub with plain subtraction to allow underflow on exits after source maturity, violating the clamping to 0 invariant**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/21.sol`
- **Caught by:** `computeDurationPerCallback` (ORCH-13)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule computeDurationPerCallback`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 21`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -366,7 +366,7 @@
         } else if (callback == BORROW_BLUE_TO_MIDNIGHT_CALLBACK || callback ==
LEND_VAULT_TO_MIDNIGHT_CALLBACK) {
             return targetMaturity - block.timestamp;
         } else {
-            return UtilsLib.zeroFloorSub(sourceMaturity, block.timestamp);
+            return sourceMaturity - block.timestamp;  // MUTATION: rebased
         }
     }
```

❌ **BaseMigrationRatifier #22 — Tightens the lower duration-band check from < to <=, so a target maturity exactly at now+minDuration is now rejected; the witness that a boundary target maturity is accepted becomes unreachable because the take reverts.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/22.sol`
- **Caught by:** `targetMaturityWithinDurationBand_boundaryAccepted` (ORCH-10)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule targetMaturityWithinDurationBand_boundaryAccepted`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 22`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -293,7 +293,7 @@
         if (targetMaturity <= sourceMaturity) revert InvalidTargetMaturity();
         uint256 minTarget = block.timestamp + params.minDuration;
         uint256 maxTarget = block.timestamp + params.maxDuration;
-        if (targetMaturity < minTarget || targetMaturity > maxTarget) {
+        if (targetMaturity <= minTarget || targetMaturity > maxTarget) {  // MUTATION:
rebased
             revert InvalidTargetMaturity();
         }
         if (
```

❌ **BaseMigrationRatifier #23 — Window param guard > -> >= : rejects renewalWindow == sourceMaturity (window opens at time 0), a valid config. Caught by the boundary-accepted satisfy companion.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/23.sol`
- **Caught by:** `v2SourceWindowEnforcedBeforeOpen_boundaryAccepted` (ORCH-8)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule v2SourceWindowEnforcedBeforeOpen_boundaryAccepted`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh`

`BaseMigrationRatifier 23`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -277,7 +277,7 @@
             // Invariant check: a compliant cadence returns a period start <= the queried
timestamp.
             if (renewalPeriodStart > block.timestamp) revert InvalidRenewalParams();
         } else {
-            if (params.renewalWindow > sourceMaturity) revert InvalidRenewalParams();
+            if (params.renewalWindow >= sourceMaturity) revert InvalidRenewalParams();  //
MUTATION: rebased
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
             if (block.timestamp < renewalPeriodStart) revert InvalidRenewalWindow();
         }
```

❌ **BaseMigrationRatifier #24 — Cadence-boundary guard > -> >= : rejects a V1->V2 enter whose nearest boundary is exactly now, a valid config. Caught by the boundary-accepted satisfy companion.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/24.sol`
- **Caught by:** `variableSourceWindowEnforced_boundaryAccepted` (ORCH-7)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule variableSourceWindowEnforced_boundaryAccepted`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 24`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -275,7 +275,7 @@
             if (params.renewalCadence == address(0)) revert InvalidRenewalParams();
             renewalPeriodStart =
IRenewalCadence(params.renewalCadence).cadencePeriodStart(block.timestamp);
             // Invariant check: a compliant cadence returns a period start <= the queried
timestamp.
-            if (renewalPeriodStart > block.timestamp) revert InvalidRenewalParams();
+            if (renewalPeriodStart >= block.timestamp) revert InvalidRenewalParams();  //
MUTATION: rebased
         } else {
             if (params.renewalWindow > sourceMaturity) revert InvalidRenewalParams();
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
```

❌ **BaseMigrationRatifier #25 — tick guard != -> == : reverts on a matching tick, accepts a mismatch**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/25.sol`
- **Caught by:** `tickMustMatchOffer` (DEFAULT-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule tickMustMatchOffer`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 25`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -201,7 +201,7 @@
         if (callback == BORROW_MIDNIGHT_RENEWAL_CALLBACK || callback ==
LEND_MIDNIGHT_RENEWAL_CALLBACK) {
             IBorrowMidnightRenewalCallback.CallbackData memory decoded =
                 abi.decode(callbackData, (IBorrowMidnightRenewalCallback.CallbackData));
-            if (decoded.tick != offer.tick) revert InvalidCallbackData();
+            if (decoded.tick == offer.tick) revert InvalidCallbackData();  // MUTATION:
tick guard != -> == : reverts on a matching tick, accep
             return (
                 decoded.sourceMarket.toTenorMarketId(),
                 offer.market.toTenorMarketId(),
```

❌ **BaseMigrationRatifier #28 — Flips the renewal-window guard from < to >=, so every fixed-source take reverts instead of only early ones; the witness that a post-maturity renewal is executable disappears because the take always reverts.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/28.sol`
- **Caught by:** `postMaturityV2ToV2Executable` (ORCH-5)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/reachability.conf --rule postMaturityV2ToV2Executable`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 28`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -279,7 +279,7 @@
         } else {
             if (params.renewalWindow > sourceMaturity) revert InvalidRenewalParams();
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
-            if (block.timestamp < renewalPeriodStart) revert InvalidRenewalWindow();
+            if (block.timestamp >= renewalPeriodStart) revert InvalidRenewalWindow(); //
MUTATION: rebased
         }
         if (targetMaturity > 0) _validateTargetMaturity(sourceMaturity, targetMaturity,
params);
     }
```

❌ **BaseMigrationRatifier #29 — Inverts the target-maturity guard from >0 to ==0, so an exit with zero target maturity now runs target-maturity validation and always reverts; the witness that a post-maturity exit is executable disappears because the take reverts.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/29.sol`
- **Caught by:** `postMaturityV2ToV1Executable` (ORCH-6)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/reachability.conf --rule postMaturityV2ToV1Executable`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 29`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -281,7 +281,7 @@
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
             if (block.timestamp < renewalPeriodStart) revert InvalidRenewalWindow();
         }
-        if (targetMaturity > 0) _validateTargetMaturity(sourceMaturity, targetMaturity,
params);
+        if (targetMaturity == 0) _validateTargetMaturity(sourceMaturity, targetMaturity,
params);  // MUTATION: rebased
     }

     /// @dev Reverts unless `targetMaturity` is after `sourceMaturity`, within the user's
duration bounds,
```

❌ **BaseMigrationRatifier #31 — Adds a renewalWindow != 0 revert to the variable-source migration path, so the stored renewal window now gates a V1-to-V2 migration; the assert that such a migration ignores the renewal window flips to a counterexample.**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/31.sol`
- **Caught by:** `v1v2migrationsHaveNoRenewalWindowConstraint` (ORCH-7)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule v1v2migrationsHaveNoRenewalWindowConstraint`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 31`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -272,7 +272,7 @@
         returns (uint256 renewalPeriodStart)
     {
         if (sourceMaturity == 0) {
-            if (params.renewalCadence == address(0)) revert InvalidRenewalParams();
+            if (params.renewalCadence == address(0) || params.renewalWindow != 0) revert
InvalidRenewalParams();  // MUTATION: rebased
             renewalPeriodStart =
IRenewalCadence(params.renewalCadence).cadencePeriodStart(block.timestamp);
             // Invariant check: a compliant cadence returns a period start <= the queried
timestamp.
             if (renewalPeriodStart > block.timestamp) revert InvalidRenewalParams();
```

❌ **BaseMigrationRatifier #32 — make-on-behalf check == -> != : real gate uses taker settlementFee, diverging from the fee=0 reconstruction**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/32.sol`
- **Caught by:** `borrowerRateGateMatchesNetSellerThreshold` (BBM) · `lenderRateGateMatchesNetBuyerThreshold` (LVM)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/ratifier/highlevel.conf --rule borrowerRateGateMatchesNetSellerThreshold lenderRateGateMatchesNetBuyerThreshold`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 32`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -333,7 +333,7 @@
             );
         uint256 tickPrice = TickLib.tickToPrice(offer.tick);
         bytes32 marketId = IdLib.toId(offer.market);
-        uint256 settlementFee = offer.maker == user
+        uint256 settlementFee = offer.maker != user  // MUTATION: rebased
             ? 0
             : MORPHO_MIDNIGHT.settlementFee(marketId,
UtilsLib.zeroFloorSub(offer.market.maturity, block.timestamp));
         uint256 effPrice = userIsBuy
```

❌ BaseMigrationRatifier #33 — wrong var: rate-limit slot clobbered with policyRate, real gate diverges from reconstruction's real limit

- **Mutant:** `certora/mutations/BaseMigrationRatifier/33.sol`
- **Caught by:** `lenderRateGateMatchesNetBuyerThreshold` (LVM) · `borrowerRateGateMatchesNetSellerThreshold` (BBM)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule lenderRateGateMatchesNetBuyerThreshold borrowerRateGateMatchesNetSellerThreshold`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 33`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -341,7 +341,7 @@
             : RouterLib.netSellerPrice(tickPrice, settlementFee, feeConfig.feeRate);
         uint256 effUnitsPerWad = _effectiveUnitsPerWad(callback, marketId, offer);
         if (!PriceLib.satisfiesRateLimit(
-                userIsBuy, effUnitsPerWad, effPrice, params.limitRatePerSecond, policyRate,
duration
+                userIsBuy, effUnitsPerWad, effPrice, policyRate, policyRate, duration  //
MUTATION: rebased
             )) revert InvalidOfferRate();
     }
```

❌ BaseMigrationRatifier #36 — cadence-boundary guard > -> < : allows future cadence boundary

- **Mutant:** `certora/mutations/BaseMigrationRatifier/36.sol`
- **Caught by:** `variableSourceWindowEnforced` (ORCH-7)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule variableSourceWindowEnforced`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 36`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -275,7 +275,7 @@
             if (params.renewalCadence == address(0)) revert InvalidRenewalParams();
             renewalPeriodStart =
IRenewalCadence(params.renewalCadence).cadencePeriodStart(block.timestamp);
             // Invariant check: a compliant cadence returns a period start <= the queried
timestamp.
-            if (renewalPeriodStart > block.timestamp) revert InvalidRenewalParams();
+            if (renewalPeriodStart < block.timestamp) revert InvalidRenewalParams();  //
MUTATION: rebased
         } else {
             if (params.renewalWindow > sourceMaturity) revert InvalidRenewalParams();
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
```

❌ BaseMigrationRatifier #39 — fee-cap check inverted: over-cap rate can be stored

- **Mutant:** `certora/mutations/BaseMigrationRatifier/39.sol`
- **Caught by:** `feeRateNeverExceedsCallbackCap` (ORCH-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/valid_state.conf --rule feeRateNeverExceedsCallbackCap`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 39`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -86,7 +86,7 @@
         external
         onlyOwner
     {
-        if (_feeRate > _maxFeeRate(callback)) revert InvalidFeeConfig();
+        if (_feeRate <= _maxFeeRate(callback)) revert InvalidFeeConfig();  // MUTATION:
rebased
         if (_feeRate > 0 && _feeRecipient == address(0)) revert InvalidFeeConfig();
         FeeConfig storage slot = feeConfigs[callback][tenorMarketId];
         slot.feeRecipient = _feeRecipient;
```

❌ BaseMigrationRatifier #40 — recipient guard == -> != : nonzero rate with zero recipient stored

- **Mutant:** `certora/mutations/BaseMigrationRatifier/40.sol`
- **Caught by:** `nonZeroFeeRateImpliesRecipient` (ORCH-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/valid_state.conf --rule nonZeroFeeRateImpliesRecipient`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 40`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -87,7 +87,7 @@
         onlyOwner
     {
         if (_feeRate > _maxFeeRate(callback)) revert InvalidFeeConfig();
-        if (_feeRate > 0 && _feeRecipient == address(0)) revert InvalidFeeConfig();
+        if (_feeRate > 0 && _feeRecipient != address(0)) revert InvalidFeeConfig();  //
MUTATION: rebased
         FeeConfig storage slot = feeConfigs[callback][tenorMarketId];
         slot.feeRecipient = _feeRecipient;
         slot.feeRate = uint96(_feeRate);
```

❌ BaseMigrationRatifier #41 — fee-market ternary swapped: consults wrong market slot

- **Mutant:** `certora/mutations/BaseMigrationRatifier/41.sol`
- **Caught by:** `feeMarketIdIgnoresCrossMarketSlot` (ORCH-14)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule feeMarketIdIgnoresCrossMarketSlot`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 41`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -149,7 +149,7 @@
         _validateMarketPair(src, tgt, callbackSourceMarketId, callbackTargetMarketId);

         // The fee config is keyed on the Midnight market: the target for entries and
renewals, the source for exits.
-        bytes32 feeMarketId = targetMaturity == 0 ? callbackSourceMarketId :
callbackTargetMarketId;
+        bytes32 feeMarketId = targetMaturity == 0 ? callbackTargetMarketId :
callbackSourceMarketId;  // MUTATION: rebased
         FeeConfig memory expectedFee = getEffectiveFeeConfig(callback, feeMarketId);
         if (callbackFeeRate != expectedFee.feeRate || callbackFeeRecipient !=
expectedFee.feeRecipient) {
             revert InvalidFeeConfig();
```

❌ **BaseMigrationRatifier #42 — getRate user arg -> address(0): forwards wrong principal owner**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/42.sol`
- **Caught by:** `getRatePrincipalForwardedFaithfully`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule getRatePrincipalForwardedFaithfully`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 42`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -325,7 +325,7 @@
                 sourceTenorMarketId,
                 targetTenorMarketId,
                 renewalPeriodStart,
-                user,
+                address(0), // MUTATION: pass zero instead of user
                 taker,
                 sourceMaturity,
                 targetMaturity,
```

❌ **BaseMigrationRatifier #43 — unauthorized-callback revert disabled (if(false)): accepts unknown callback**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/43.sol`
- **Caught by:** `unauthorizedCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule unauthorizedCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 43`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -257,7 +257,7 @@
                 decoded.feeRecipient
             );
         } else {
-            revert InvalidCallback();
+            if (false) revert InvalidCallback();  // MUTATION: rebased
         }
     }
```

❌ **BaseMigrationRatifier #44 — duration-band || -> && : out-of-band maturity no longer reverts**

- **Mutant:** `certora/mutations/BaseMigrationRatifier/44.sol`
- **Caught by:** `targetMaturityWithinDurationBand` (ORCH-10)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/ratifier/revert.conf --rule targetMaturityWithinDurationBand`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 44`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -293,7 +293,7 @@
         if (targetMaturity <= sourceMaturity) revert InvalidTargetMaturity();
         uint256 minTarget = block.timestamp + params.minDuration;
         uint256 maxTarget = block.timestamp + params.maxDuration;
-        if (targetMaturity < minTarget || targetMaturity > maxTarget) {
+        if (targetMaturity < minTarget && targetMaturity > maxTarget) {  // MUTATION:
rebased
             revert InvalidTargetMaturity();
         }
         if (
```

❌ BaseMigrationRatifier #45 — source-window guard < -> > : before-open no longer reverts

- **Mutant:** certora/mutations/BaseMigrationRatifier/45.sol
- **Caught by:** v2SourceWindowEnforcedBeforeOpen (ORCH-8)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule v2SourceWindowEnforcedBeforeOpen`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 45`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -279,7 +279,7 @@
         } else {
             if (params.renewalWindow > sourceMaturity) revert InvalidRenewalParams();
             renewalPeriodStart = sourceMaturity - params.renewalWindow;
-            if (block.timestamp < renewalPeriodStart) revert InvalidRenewalWindow();
+            if (block.timestamp > renewalPeriodStart) revert InvalidRenewalWindow();  //
MUTATION: rebased
         }
         if (targetMaturity > 0) _validateTargetMaturity(sourceMaturity, targetMaturity,
params);
     }
```

❌ BaseMigrationRatifier #46 — target-maturity guard <= -> >= : V1->V2 (srcMat==0) always reverts, witness unreachable

- **Mutant:** certora/mutations/BaseMigrationRatifier/46.sol
- **Caught by:** entryV1ToV2Executable
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/reachability.conf --rule entryV1ToV2Executable`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh`

`BaseMigrationRatifier 46`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -290,7 +290,7 @@
         internal
         view
     {
-        if (targetMaturity <= sourceMaturity) revert InvalidTargetMaturity();
+        if (targetMaturity >= sourceMaturity) revert InvalidTargetMaturity();  // MUTATION:
rebased
         uint256 minTarget = block.timestamp + params.minDuration;
         uint256 maxTarget = block.timestamp + params.maxDuration;
         if (targetMaturity < minTarget || targetMaturity > maxTarget) {
```

❌ BaseMigrationRatifier #47 — continuous-fee lifetime * -> + : high fee no longer reaches WAD cap, over-cap offer accepted

- **Mutant:** `certora/mutations/BaseMigrationRatifier/47.sol`
- **Caught by:** `continuousFeeCapReverts` (LVM, LMR)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule continuousFeeCapReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 47`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -396,7 +396,7 @@
         uint256 continuousFee = MORPHO_MIDNIGHT.continuousFee(marketId);
         if (continuousFee == 0) return WAD;
         uint256 timeToMaturity = UtilsLib.zeroFloorSub(offer.market.maturity,
block.timestamp);
-        uint256 fee = continuousFee * timeToMaturity;
+        uint256 fee = continuousFee + timeToMaturity;  // MUTATION: rebased
         if (fee >= WAD) revert InvalidTargetMaturity();
         return WAD - fee;
     }
```

❌ BaseMigrationRatifier #48 — Comment out the feeRecipient assignment so setFeeConfig stores only the rate, breaking fee-slot write fidelity

- **Mutant:** `certora/mutations/BaseMigrationRatifier/48.sol`
- **Caught by:** `setFeeConfigWritesSlotAndLeavesOthers`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule setFeeConfigWritesSlotAndLeavesOthers`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BaseMigrationRatifier 48`

```
--- a/src/ratifiers/BaseMigrationRatifier.sol
+++ b/src/ratifiers/BaseMigrationRatifier.sol
@@ -89,7 +89,7 @@
         if (_feeRate > _maxFeeRate(callback)) revert InvalidFeeConfig();
         if (_feeRate > 0 && _feeRecipient == address(0)) revert InvalidFeeConfig();
         FeeConfig storage slot = feeConfigs[callback][tenorMarketId];
-        slot.feeRecipient = _feeRecipient;
+        // slot.feeRecipient = _feeRecipient;   // MUTATION: feeRecipient no longer stored
         slot.feeRate = uint96(_feeRate);
         emit FeeConfigSet(callback, tenorMarketId, _feeRate, _feeRecipient);
     }
```

`BorrowBlueToMidnightCallback` — `src/callbacks/BorrowBlueToMidnightCallback.sol`

❌ BorrowBlueToMidnightCallback #1 — auth guard flipped (!= -> ==)

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/1.sol`
- **Caught by:** `callbackRevertsForNonMidnightCaller` (CB-AUTH-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/callbackRevertsForNonMidnightCaller.conf --rule callbackRevertsForNonMidnightCaller`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 1`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -52,7 +52,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  // MUTATION: rebased
         if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ BorrowBlueToMidnightCallback #2 — supplyCollateral amount forced to 0: collateral never lands on Midnight, migration cannot move it

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/2.sol`
- **Caught by:** `migrationCanMoveCollateralBlueToMidnight`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationCanMoveCollateralBlueToMidnight.conf --rule migrationCanMoveCollateralBlueToMidnight`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 2`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -89,7 +89,7 @@
         if (collateralMigrated > 0) {
             MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));

IERC20(sourceMarketParams.collateralToken).forceApprove(address(MORPHO_MIDNIGHT),
collateralMigrated);
-            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
seller);
+            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, 0, seller);  //
MUTATION: rebased
         }

         emit BorrowMigratedBlueToMidnight(
```

❌ BorrowBlueToMidnightCallback #3 — onSell final-fill collateral blueCollateral -1 : debt fully clears but 1 wei collateral remains (coupling broken)

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/3.sol`
- **Caught by:** `clearingOldDebtAlsoEmptiesOldCollateral` (CB-FINAL-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/clearingOldDebtAlsoEmptiesOldCollateral.conf --rule clearingOldDebtAlsoEmptiesOldCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 3`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -84,7 +84,7 @@
             MORPHO_BLUE.repay(sourceMarketParams, repayBudget, 0, seller, "");
         }

-        uint256 collateralMigrated = isFinalFill ? blueCollateral :
blueCollateral.mulDivDown(repayBudget, blueDebt);
+        uint256 collateralMigrated = isFinalFill ? blueCollateral - 1 :
blueCollateral.mulDivDown(repayBudget, blueDebt);  // MUTATION: rebased

         if (collateralMigrated > 0) {
             MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));
```

❌ BorrowBlueToMidnightCallback #4 — onSell Midnight supply amount collateralMigrated -1 : mnIn = blueOut-1, breaks 1:1 conservation

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/4.sol`
- **Caught by:** `migrationConservesMigratedCollateral` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationConservesMigratedCollateral.conf --rule migrationConservesMigratedCollateral`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 4`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -89,7 +89,7 @@
         if (collateralMigrated > 0) {
             MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));

IERC20(sourceMarketParams.collateralToken).forceApprove(address(MORPHO_MIDNIGHT),
collateralMigrated);
-            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
seller);
+            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated -
1, seller);  // MUTATION: rebased
         }

         emit BorrowMigratedBlueToMidnight(
```

❌ BorrowBlueToMidnightCallback #9 — Supplying collateral to the new Midnight market is replaced by withdrawing it, so the Midnight collateral shrinks instead of growing and the rule requiring migration to only add new-market collateral produces a counterexample.

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/9.sol`
- **Caught by:** `migrationOnlyAddsNewMidnightCollateral` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/perf/migrationOnlyAddsNewMidnightCollateral.conf --rule migrationOnlyAddsNewMidnightCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 9`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -89,7 +89,7 @@
         if (collateralMigrated > 0) {
             MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));

IERC20(sourceMarketParams.collateralToken).forceApprove(address(MORPHO_MIDNIGHT),
collateralMigrated);
-            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
seller);
+            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated,
seller, address(this));  // MUTATION: rebased
         }

         emit BorrowMigratedBlueToMidnight(
```

❌ **BorrowBlueToMidnightCallback #12 — Changing the excess-repayment check from greater-than to greater-or-equal makes the final fill (where repayBudget equals the debt) revert, so the witness showing all Blue collateral fully withdrawn can no longer be produced.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/12.sol`
- **Caught by:** `fullCollateralMigrationClearsAllOldDebt__satisfy` (CB-CLOSE-2)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/perf_kill_satisfy/fullCollateralMigrationClearsAllOldDebt.conf --rule fullCollateralMigrationClearsAllOldDebt__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 12`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -74,7 +74,7 @@
         }
         uint256 repayBudget = sellerAssets - fee;

-        if (repayBudget > blueDebt) revert CallbackLib.ExcessRepayment();
+        if (repayBudget >= blueDebt) revert CallbackLib.ExcessRepayment();  // MUTATION: rebased
         IERC20(loanToken).forceApprove(address(MORPHO_BLUE), repayBudget);

         bool isFinalFill = repayBudget == blueDebt;
```

❌ **BorrowBlueToMidnightCallback #14 — Flipping the source-market loan-token check from not-equal to equal accepts a source market whose loan token differs from the offer's, so the rule requiring a mismatched loan token to revert produces a counterexample.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/14.sol`
- **Caught by:** `sourceLoanTokenMismatchReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/sourceLoanTokenMismatchReverts.conf --rule sourceLoanTokenMismatchReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 14`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -59,7 +59,7 @@
         CallbackData memory callbackData = abi.decode(data, (CallbackData));
         MarketParams memory sourceMarketParams = callbackData.sourceMarketParams;
         address loanToken = market.loanToken;
-        if (sourceMarketParams.loanToken != loanToken) revert CallbackLib.TokenMismatch();
+        if (sourceMarketParams.loanToken == loanToken) revert CallbackLib.TokenMismatch();
// MUTATION: rebased
         (bool found, uint256 collateralIndex) =
market.findCollateral(sourceMarketParams.collateralToken);
         if (!found) revert CallbackLib.TokenMismatch();
```

❌ **BorrowBlueToMidnightCallback #15 — Replacing the Blue repay with a borrow makes the seller's old Blue debt increase instead of decrease, so the rule requiring migration to only reduce the old Blue debt produces a counterexample.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/15.sol`
- **Caught by:** `migrationOnlyReducesOldBlueDebt` (CB-V1-REP-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/perf/migrationOnlyReducesOldBlueDebt.conf --rule migrationOnlyReducesOldBlueDebt`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 15`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -81,7 +81,7 @@
         if (isFinalFill) {
             MORPHO_BLUE.repay(sourceMarketParams, 0, bluePosition.borrowShares, seller,
"");
         } else {
-            MORPHO_BLUE.repay(sourceMarketParams, repayBudget, 0, seller, "");
+            MORPHO_BLUE.borrow(sourceMarketParams, repayBudget, 0, seller, seller); //
MUTATION: rebased
         }

         uint256 collateralMigrated = isFinalFill ? blueCollateral :
blueCollateral.mulDivDown(repayBudget, blueDebt);
```

❌ **BorrowBlueToMidnightCallback #16 — Replacing the Blue collateral withdrawal with a supply makes the seller's old Blue collateral grow instead of shrink, so the rule requiring migration to only withdraw old Blue collateral produces a counterexample.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/16.sol`
- **Caught by:** `migrationOnlyWithdrawsOldBlueCollateral` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowBlueToMidnightCallback/perf/migrationOnlyWithdrawsOldBlueCollateral.conf --rule migrationOnlyWithdrawsOldBlueCollateral`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 16`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -87,7 +87,7 @@
         uint256 collateralMigrated = isFinalFill ? blueCollateral :
blueCollateral.mulDivDown(repayBudget, blueDebt);

         if (collateralMigrated > 0) {
-            MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));
+            MORPHO_BLUE.supplyCollateral(sourceMarketParams, collateralMigrated, seller,
""); // MUTATION: rebased

IERC20(sourceMarketParams.collateralToken).forceApprove(address(MORPHO_MIDNIGHT),
collateralMigrated);
            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
seller);
         }
```

❌ BorrowBlueToMidnightCallback #18 — Doubling the amount transferred to the fee recipient makes the borrower pay twice the intended fee, so the rule bounding the borrower fee by its interest share produces a counterexample.

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/18.sol`
- **Caught by:** `borrowerFeeBoundedByInterestShare` (CB-RATE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/perf/borrowerFeeBoundedByInterestShare.conf --rule borrowerFeeBoundedByInterestShare`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 18`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -70,7 +70,7 @@

         uint256 fee = CallbackLib.sellerFeeFromTick(callbackData.tick,
callbackData.feeRate, units, sellerAssets);
         if (fee > 0) {
-            SafeTransferLib.safeTransfer(loanToken, callbackData.feeRecipient, fee);
+            SafeTransferLib.safeTransfer(loanToken, callbackData.feeRecipient, fee * 2); //
MUTATION: rebased
         }
         uint256 repayBudget = sellerAssets - fee;
```

❌ **BorrowBlueToMidnightCallback #20 — Flipping the caller check from not-equal to equal makes onSell revert whenever Midnight (its only legitimate caller) invokes it, so take() always reverts and the witness that migration reduces old debt on at most one market can no longer be produced.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/20.sol`
- **Caught by:** `migrationReducesOldDebtOnAtMostOneMarket__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/debug_satisfy/migrationReducesOldDebtOnAtMostOneMarket.conf --rule migrationReducesOldDebtOnAtMostOneMarket__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 20`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -52,7 +52,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  //
MUTATION: rebased
         if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ **BorrowBlueToMidnightCallback #21 — Migrating one wei less than the full Blue collateral on the final fill always leaves a wei behind, so the seller's collateral never reaches zero and the witness showing the old position can be fully closed can no longer be produced.**

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/21.sol`
- **Caught by:** `migrationCanFullyCloseOldPosition` (CB-CLOSE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/perf_kill/migrationCanFullyCloseOldPosition.conf --rule migrationCanFullyCloseOldPosition`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 21`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -84,7 +84,7 @@
             MORPHO_BLUE.repay(sourceMarketParams, repayBudget, 0, seller, "");
         }

-        uint256 collateralMigrated = isFinalFill ? blueCollateral :
blueCollateral.mulDivDown(repayBudget, blueDebt);
+        uint256 collateralMigrated = isFinalFill ? blueCollateral - 1 :
blueCollateral.mulDivDown(repayBudget, blueDebt);  // MUTATION: rebased

         if (collateralMigrated > 0) {
             MORPHO_BLUE.withdrawCollateral(sourceMarketParams, collateralMigrated, seller,
address(this));
```

❌ BorrowBlueToMidnightCallback #22 — onSell receiver guard flipped (!= -> ==): receiver!=callback no longer reverts. Caught by receiverNotCallbackReverts (CLB-BBM-12, receiverNotCallback => reverted via callbackCallWithRevert) — the flip leaves a non-reverting receiver!=callback trace, assert violated.

- **Mutant:** `certora/mutations/BorrowBlueToMidnightCallback/22.sol`
- **Caught by:** `receiverNotCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/receiverNotCallbackReverts.conf --rule receiverNotCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowBlueToMidnightCallback 22`

```
--- a/src/callbacks/BorrowBlueToMidnightCallback.sol
+++ b/src/callbacks/BorrowBlueToMidnightCallback.sol
@@ -53,7 +53,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver == address(this)) revert CallbackLib.InvalidReceiver();  // MUTATION:
rebased
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

`BorrowMidnightRenewalCallback` — `src/callbacks/BorrowMidnightRenewalCallback.sol`

❌ BorrowMidnightRenewalCallback #1 — onlyMidnight guard flipped != -> == : the legitimate Midnight caller reverts, so renewal can never roll debt

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/1.sol`
- **Caught by:** `renewalCanMoveDebtBetweenMarkets`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCanMoveDebtBetweenMarkets.conf --rule renewalCanMoveDebtBetweenMarkets`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 1`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -45,7 +45,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  // MUTATION: rebased
         if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ BorrowMidnightRenewalCallback #3 — Allow renewal into the same market by flipping equality check; violates CLB-BMR-12

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/3.sol`
- **Caught by:** `callbackRevertsForSameSourceMarket` (CB-SAME-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/callbackRevertsForSameSourceMarket.conf --rule callbackRevertsForSameSourceMarket`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 3`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -60,7 +60,7 @@
         }
         uint256 repayBudget = sellerAssets - fee;
         bytes32 sourceMarketId = IdLib.toId(callbackData.sourceMarket);
-        if (sourceMarketId == marketId) revert CallbackLib.SameMarket();
+        if (sourceMarketId != marketId) revert CallbackLib.SameMarket();  // MUTATION: rebased
         uint256 sourceDebtBefore = MORPHO_MIDNIGHT.debt(sourceMarketId, seller);

         if (sourceDebtBefore == 0) revert CallbackLib.ZeroAmount();
```

❌ BorrowMidnightRenewalCallback #8 — Changing repayBudget to 0 prevents any debt repayment on the source market, making it impossible to satisfy the goal of moving debt from source to target market.

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/8.sol`
- **Caught by:** `renewalCanMoveDebtBetweenMarkets`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCanMoveDebtBetweenMarkets.conf --rule renewalCanMoveDebtBetweenMarkets`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 8`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -67,7 +67,7 @@
         if (repayBudget > sourceDebtBefore) revert CallbackLib.ExcessRepayment();

         IERC20(market.loanToken).forceApprove(address(MORPHO_MIDNIGHT), repayBudget);
-        MORPHO_MIDNIGHT.repay(callbackData.sourceMarket, repayBudget, seller, address(0), "");
+        MORPHO_MIDNIGHT.repay(callbackData.sourceMarket, 0, seller, address(0), "");  // MUTATION: rebased

         (address[] memory collateralTokens, uint256[] memory collateralAmounts) = MORPHO_MIDNIGHT.transferCollaterals(
             callbackData.sourceMarket, market, seller, sourceMarketId, sourceDebtBefore, repayBudget
```

❌ **BorrowMidnightRenewalCallback #13 — onSell receiver guard flipped (!= -> ==): receiver!=callback no longer reverts. Caught by receiverNotCallbackReverts (CLB-BMR-13, receiverNotCallback => reverted via callbackCallWithRevert) — the flip leaves a non-reverting receiver!=callback trace, assert violated.**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/13.sol`
- **Caught by:** `receiverNotCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/receiverNotCallbackReverts.conf --rule receiverNotCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 13`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -46,7 +46,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver == address(this)) revert CallbackLib.InvalidReceiver();  // MUTATION: rebased
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ **BorrowMidnightRenewalCallback #22 — fee transfer doubled: the fee recipient receives fee*2, pushing the paid tick fee past the sellerAssets bound on a non-reverting take — sellerTickFeeNeverExceedsAssets is violated.**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/22.sol`
- **Caught by:** `sellerTickFeeNeverExceedsAssets` (CB-FEE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/sellerTickFeeNeverExceedsAssets.conf --rule sellerTickFeeNeverExceedsAssets`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 22`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -56,7 +56,7 @@
         uint256 fee = CallbackLib.sellerFeeFromTick(callbackData.tick,
callbackData.feeRate, units, sellerAssets);

         if (fee > 0) {
-            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
+            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee *
2);  // MUTATION: rebased
         }
         uint256 repayBudget = sellerAssets - fee;
         bytes32 sourceMarketId = IdLib.toId(callbackData.sourceMarket);
```

❌ **BorrowMidnightRenewalCallback #23 — onSell transferCollaterals source/target markets swapped : collateral flows target->source (CLB-BMR-03 add-while-reduce, CLB-BMR-04 remove-while-open)**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/23.sol`
- **Caught by:** `renewalCannotAddCollateralWhenReducingDebt` (CB-DIR-1) · `renewalCannotRemoveCollateralWhenOpeningDebt` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_kill/renewalCannotAddCollateralWhenReducingDebt.conf --rule renewalCannotAddCollateralWhenReducingDebt renewalCannotRemoveCollateralWhenOpeningDebt`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 23`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -70,7 +70,7 @@
         MORPHO_MIDNIGHT.repay(callbackData.sourceMarket, repayBudget, seller, address(0),
"");

         (address[] memory collateralTokens, uint256[] memory collateralAmounts) =
MORPHO_MIDNIGHT.transferCollaterals(
-            callbackData.sourceMarket, market, seller, sourceMarketId, sourceDebtBefore,
repayBudget
+            market, callbackData.sourceMarket, seller, sourceMarketId, sourceDebtBefore,
repayBudget  // MUTATION: rebased
         );

         emit BorrowRenewed(seller, sourceMarketId, marketId, repayBudget, collateralTokens,
collateralAmounts, fee);
```

❌ BorrowMidnightRenewalCallback #24 — The callback inserts an extra self-funded repayment on the target market, so debt drops on both the source and target markets, and the rule requiring at most one market's debt to fall reports a counterexample.

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/24.sol`
- **Caught by:** `renewalReducesDebtOnAtMostOneMarket` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf/renewalReducesDebtOnAtMostOneMarket.conf --rule renewalReducesDebtOnAtMostOneMarket`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 24`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -73,6 +73,8 @@
             callbackData.sourceMarket, market, seller, sourceMarketId, sourceDebtBefore,
repayBudget
         );

+        IERC20(market.loanToken).forceApprove(address(MORPHO_MIDNIGHT), units + 1);  //
MUTATION: rebased insert
+        MORPHO_MIDNIGHT.repay(market, units + 1, seller, address(0), "");
         emit BorrowRenewed(seller, sourceMarketId, marketId, repayBudget, collateralTokens,
collateralAmounts, fee);

         return CALLBACK_SUCCESS;
```

❌ **BorrowMidnightRenewalCallback #25 — onSell inserts safeTransferFrom(units+1) into the callback : loanToken inflow exceeds units bound**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/25.sol`
- **Caught by:** `renewalCallbackNeverPullsExternalLoanToken` (CB-SRC-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf/renewalCallbackNeverPullsExternalLoanToken.conf --rule renewalCallbackNeverPullsExternalLoanToken`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 25`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -68,6 +68,7 @@

         IERC20(market.loanToken).forceApprove(address(MORPHO_MIDNIGHT), repayBudget);
         MORPHO_MIDNIGHT.repay(callbackData.sourceMarket, repayBudget, seller, address(0),
"");
+        SafeTransferLib.safeTransferFrom(market.loanToken, seller, address(this), units +
1);  // MUTATION: onSell inserts safeTransferFrom(units+1)

         (address[] memory collateralTokens, uint256[] memory collateralAmounts) =
MORPHO_MIDNIGHT.transferCollaterals(
             callbackData.sourceMarket, market, seller, sourceMarketId, sourceDebtBefore,
repayBudget
```

❌ **BorrowMidnightRenewalCallback #31 — The loan-token match check is inverted so that matching source and target tokens revert, which every real renewal needs, so take() always reverts and the witness opening new target debt without removing collateral can no longer be produced.**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/31.sol`
- **Caught by:** `renewalCannotRemoveCollateralWhenOpeningDebt__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/debug_satisfy/renewalCannotRemoveCollateralWhenOpeningDebt.conf --rule renewalCannotRemoveCollateralWhenOpeningDebt__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 31`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -51,7 +51,7 @@

         CallbackData memory callbackData = abi.decode(data, (CallbackData));

-        if (callbackData.sourceMarket.loanToken != market.loanToken) revert
CallbackLib.TokenMismatch();
+        if (callbackData.sourceMarket.loanToken == market.loanToken) revert
CallbackLib.TokenMismatch();  // MUTATION: rebased

         uint256 fee = CallbackLib.sellerFeeFromTick(callbackData.tick,
callbackData.feeRate, units, sellerAssets);
```

❌ BorrowMidnightRenewalCallback #33 — onSell receiver guard inverted (!= to ==) so every take-driven onSell reverts and the receiver-narrowed callbackNeverHoldsTokens_satisfy witness becomes UNSAT.

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/33.sol`
- **Caught by:** `callbackNeverHoldsTokens__satisfy` (CB-DUST-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_satisfy/callbackNeverHoldsTokens.conf --rule callbackNeverHoldsTokens__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 33`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -46,7 +46,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver == address(this)) revert CallbackLib.InvalidReceiver();  // MUTATION:
rebased
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ **BorrowMidnightRenewalCallback #34 — Flipping the Midnight-caller guard to == makes onSell revert on its first instruction whenever the caller is Midnight, which it always is inside take(), so the renewalAddsDebtOnAtMostOneMarket satisfy witness can never reach its assert point and turns unsatisfiable.**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/34.sol`
- **Caught by:** `renewalAddsDebtOnAtMostOneMarket__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_satisfy/renewalAddsDebtOnAtMostOneMarket.conf --rule renewalAddsDebtOnAtMostOneMarket__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 34`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -45,7 +45,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  //
MUTATION: rebased
         if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ **BorrowMidnightRenewalCallback #35 — Flipping the Midnight-caller guard to == makes onSell revert immediately on every in-model take because the caller is always Midnight, so the thirdPartyBalanceUnchanged, callbackHoldsZeroAllowance, and feeRecipientNeverLosesTokens satisfy witnesses become unsatisfiable.**

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/35.sol`
- **Caught by:** `thirdPartyBalanceUnchanged__satisfy` · `callbackHoldsZeroAllowance__satisfy` (CB-DUST-1) · `feeRecipientNeverLosesTokens__satisfy`
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/debug_satisfy/thirdPartyBalanceUnchanged.conf --rule thirdPartyBalanceUnchanged__satisfy callbackHoldsZeroAllowance__satisfy feeRecipientNeverLosesTokens__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 35`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -45,7 +45,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  //
MUTATION: rebased
         if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ BorrowMidnightRenewalCallback #36 — Doubling the seller fee transfer pushes the fee recipient's loanToken balance delta past the interest-share bound, so borrowerFeeBoundedByInterestShare flips to a counterexample.

- **Mutant:** `certora/mutations/BorrowMidnightRenewalCallback/36.sol`
- **Caught by:** `borrowerFeeBoundedByInterestShare` (CB-RATE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_kill/borrowerFeeBoundedByInterestShare.conf --rule borrowerFeeBoundedByInterestShare`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightRenewalCallback 36`

```
--- a/src/callbacks/BorrowMidnightRenewalCallback.sol
+++ b/src/callbacks/BorrowMidnightRenewalCallback.sol
@@ -56,7 +56,7 @@
         uint256 fee = CallbackLib.sellerFeeFromTick(callbackData.tick,
callbackData.feeRate, units, sellerAssets);

         if (fee > 0) {
-            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
+            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee *
2);  // MUTATION: rebased
         }
         uint256 repayBudget = sellerAssets - fee;
         bytes32 sourceMarketId = IdLib.toId(callbackData.sourceMarket);
```

`BorrowMidnightToBlueCallback` — `src/callbacks/BorrowMidnightToBlueCallback.sol`

❌ BorrowMidnightToBlueCallback #8 — Commenting out the borrow call prevents Blue debt from being opened, making the satisfy clause impossible to prove (cannot demonstrate that blueSharesAfter > sharesBefore).

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/8.sol`
- **Caught by:** `migrationCanOpenNewBlueDebt`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanOpenNewBlueDebt.conf --rule migrationCanOpenNewBlueDebt`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 8`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -81,7 +81,7 @@
         }

         uint256 borrowAmount = buyerAssets + fee;
-        MORPHO_BLUE.borrow(callbackData.targetMarketParams, borrowAmount, 0, buyer, address(this));
+        // MORPHO_BLUE.borrow(callbackData.targetMarketParams, borrowAmount, 0, buyer, address(this));  // MUTATION: rebased

         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

❌ BorrowMidnightToBlueCallback #9 — withdrawCollateral commented out: collateral never leaves Midnight, migration cannot move it

- **Mutant:** certora/mutations/BorrowMidnightToBlueCallback/9.sol
- **Caught by:** migrationCanMoveCollateralMidnightToBlue
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanMoveCollateralMidnightToBlue.conf --rule migrationCanMoveCollateralMidnightToBlue`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 9`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -73,7 +73,7 @@
             sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units, sourceDebtBefore);

         if (collateralMigrated > 0) {
-            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated, buyer, address(this));
+            // MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated, buyer, address(this));  // MUTATION: rebased

             IERC20(callbackData.targetMarketParams.collateralToken)
                 .forceApprove(address(MORPHO_BLUE), collateralMigrated);
```

❌ **BorrowMidnightToBlueCallback #10 — Setting the amount withdrawn from Midnight to zero leaves the borrower's old collateral in place, so no fill can ever fully close the old position and the rule's witness becomes unsatisfiable.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/10.sol`
- **Caught by:** `migrationCanFullyCloseOldPosition` (CB-CLOSE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanFullyCloseOldPosition.conf --rule migrationCanFullyCloseOldPosition`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 10`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -73,7 +73,7 @@
             sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units,
sourceDebtBefore);

         if (collateralMigrated > 0) {
-            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated,
buyer, address(this));
+            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, 0, buyer,
address(this));  // MUTATION: rebased

             IERC20(callbackData.targetMarketParams.collateralToken)
                 .forceApprove(address(MORPHO_BLUE), collateralMigrated);
```

❌ **BorrowMidnightToBlueCallback #18 — fee-cap guard inverted (> -> <): every legal below-cap feeRate now reverts while an above-cap rate is accepted — percentageFeeRateAboveCapReverts (aboveCap => reverted) is violated.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/18.sol`
- **Caught by:** `percentageFeeRateAboveCapReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/percentageFeeRateAboveCapReverts.conf --rule percentageFeeRateAboveCapReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 18`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -58,7 +58,7 @@
     /// @dev Returns the flat percentage fee assets * feeRate / WAD, rounded down.
     /// @dev Reverts if feeRate > MAX_PERCENTAGE_FEE_RATE (1%).
     function percentageFee(uint256 assets, uint256 feeRate) internal pure returns (uint256
fee) {
-        if (feeRate > MAX_PERCENTAGE_FEE_RATE) revert InvalidFeeConfig();
+        if (feeRate < MAX_PERCENTAGE_FEE_RATE) revert InvalidFeeConfig();  // MUTATION:
rebased
         fee = assets.mulDivDown(feeRate, WAD);
     }
```

❌ **BorrowMidnightToBlueCallback #20 — Replacing the Midnight withdrawCollateral call with supplyCollateral makes the borrower's old Midnight collateral grow instead of shrink, so the rule that the migration can only reduce old collateral flips to a counterexample.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/20.sol`
- **Caught by:** `migrationOnlyWithdrawsOldMidnightCollateral` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/migrationOnlyWithdrawsOldMidnightCollateral.conf --rule migrationOnlyWithdrawsOldMidnightCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 20`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -73,7 +73,7 @@
             sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units,
sourceDebtBefore);

         if (collateralMigrated > 0) {
-            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated,
buyer, address(this));
+            MORPHO_MIDNIGHT.supplyCollateral(market, collateralIndex, collateralMigrated,
buyer);  // MUTATION: rebased

             IERC20(callbackData.targetMarketParams.collateralToken)
                 .forceApprove(address(MORPHO_BLUE), collateralMigrated);
```

❌ **BorrowMidnightToBlueCallback #24 — Withdrawing one unit less from Midnight than is supplied to Blue makes the amount deposited into Blue exceed the amount withdrawn from Midnight, so the deposit-at-most-withdrawn rule flips to a counterexample.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/24.sol`
- **Caught by:** `migrationCannotDepositMoreCollateralThanWithdrawn` (CB-SRC-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/migrationCannotDepositMoreCollateralThanWithdrawn.conf --rule migrationCannotDepositMoreCollateralThanWithdrawn`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 24`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -73,7 +73,7 @@
             sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units, sourceDebtBefore);

         if (collateralMigrated > 0) {
-            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated, buyer, address(this));
+            MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated - 1, buyer, address(this));  // MUTATION: rebased

             IERC20(callbackData.targetMarketParams.collateralToken)
                 .forceApprove(address(MORPHO_BLUE), collateralMigrated);
```

❌ BorrowMidnightToBlueCallback #25 — Migrating one unit less than the full source collateral on the final fill leaves a unit of old Midnight collateral behind after the debt is fully repaid, so the rule that the final fill drains all old collateral flips to a counterexample.

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/25.sol`
- **Caught by:** `migrationFinalFillTransfersAllOldMidnightCollateral` (CB-FINAL-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/migrationFinalFillTransfersAllOldMidnightCollateral.conf --rule migrationFinalFillTransfersAllOldMidnightCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 25`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -70,7 +70,7 @@
         }

         uint256 collateralMigrated =
-            sourceDebtAfter == 0 ? sourceCollateral : sourceCollateral.mulDivDown(units, sourceDebtBefore);
+            sourceDebtAfter == 0 ? sourceCollateral - 1 : sourceCollateral.mulDivDown(units, sourceDebtBefore);  // MUTATION: rebased

         if (collateralMigrated > 0) {
             MORPHO_MIDNIGHT.withdrawCollateral(market, collateralIndex, collateralMigrated, buyer, address(this));
```

❌ **BorrowMidnightToBlueCallback #26 — Replacing the Blue supplyCollateral call with withdrawCollateral makes the borrower's new Blue collateral shrink instead of grow, so the rule that the migration can only add new Blue collateral flips to a counterexample.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/26.sol`
- **Caught by:** `migrationOnlyAddsNewBlueCollateral` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/migrationOnlyAddsNewBlueCollateral.conf --rule migrationOnlyAddsNewBlueCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 26`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -77,7 +77,7 @@

             IERC20(callbackData.targetMarketParams.collateralToken)
                 .forceApprove(address(MORPHO_BLUE), collateralMigrated);
-            MORPHO_BLUE.supplyCollateral(callbackData.targetMarketParams,
collateralMigrated, buyer, "");
+            MORPHO_BLUE.withdrawCollateral(callbackData.targetMarketParams,
collateralMigrated, buyer, address(this));  // MUTATION: rebased
         }

         uint256 borrowAmount = buyerAssets + fee;
```

❌ **BorrowMidnightToBlueCallback #27 — Replacing the Blue borrow call with repay makes the borrower's new Blue debt shares fall instead of rise, so the rule that the migration can only open new Blue debt flips to a counterexample.**

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/27.sol`
- **Caught by:** `migrationOnlyOpensNewBlueDebt` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/migrationOnlyOpensNewBlueDebt.conf --rule migrationOnlyOpensNewBlueDebt`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 27`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -81,7 +81,7 @@
         }

         uint256 borrowAmount = buyerAssets + fee;
-        MORPHO_BLUE.borrow(callbackData.targetMarketParams, borrowAmount, 0, buyer,
address(this));
+        MORPHO_BLUE.repay(callbackData.targetMarketParams, borrowAmount, 0, buyer, "");  //
MUTATION: rebased

         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

❌ BorrowMidnightToBlueCallback #29 — Inverts the caller guard (!= to ==), so onBuy reverts OnlyMidnight on every take() (which always comes from Midnight) and the reachability witness goes UNSAT.

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/29.sol`
- **Caught by:** `migrationReducesOldDebtOnAtMostOneMarket__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/BorrowMidnightToBlueCallback/debug_satisfy/migrationReducesOldDebtOnAtMostOneMarket.conf --rule migrationReducesOldDebtOnAtMostOneMarket__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 29`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -48,7 +48,7 @@
         address buyer,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  //
MUTATION: rebased
         if (buyerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ BorrowMidnightToBlueCallback #30 — Borrowing on behalf of the callback contract instead of the buyer opens new Blue debt for a party whose old Midnight debt did not fall, so the rule coupling old and new debt movements flips to a counterexample.

- **Mutant:** `certora/mutations/BorrowMidnightToBlueCallback/30.sol`
- **Caught by:** `oldMidnightDebtAndNewBlueDebtMoveTogether` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/BorrowMidnightToBlueCallback/perf/oldMidnightDebtAndNewBlueDebtMoveTogether.conf --rule oldMidnightDebtAndNewBlueDebtMoveTogether`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh BorrowMidnightToBlueCallback 30`

```
--- a/src/callbacks/BorrowMidnightToBlueCallback.sol
+++ b/src/callbacks/BorrowMidnightToBlueCallback.sol
@@ -81,7 +81,7 @@
         }

         uint256 borrowAmount = buyerAssets + fee;
-        MORPHO_BLUE.borrow(callbackData.targetMarketParams, borrowAmount, 0, buyer, address(this));
+        MORPHO_BLUE.borrow(callbackData.targetMarketParams, borrowAmount, 0, address(this), address(this));  // MUTATION: rebased

         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

`CallbackLib` — `src/libraries/CallbackLib.sol`

❌ CallbackLib #1 — _interestFeeComponent sign flip (WAD-price)->(WAD+price): nonzero interest fee at par, so the tick fee no longer vanishes

- **Mutant:** `certora/mutations/CallbackLib/1.sol`
- **Caught by:** `tickFeeVanishesAtPar` (CB-FEE-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/tickFeeVanishesAtPar.conf --rule tickFeeVanishesAtPar`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CallbackLib 1`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -67,7 +67,7 @@
     /// @dev The caller must handle feeRate == 0 before calling.
     function _interestFeeComponent(uint256 price, uint256 feeRate) private pure returns (uint256) {
         if (feeRate > WAD) revert InvalidFeeConfig();
-        return (WAD - price).mulDivDown(feeRate, WAD);
+        return (WAD + price).mulDivDown(feeRate, WAD);  // MUTATION: rebased
     }

     /// @dev Returns the seller-side effective price, price * WAD / (WAD + feeShareOfInterest), rounded up.
```

❌ **CallbackLib #3 — transposed mulDivDown args (feeRate,WAD)->(WAD,feeRate): fee share inverse in feeRate breaks net-price fee-monotonicity**

- **Mutant:** `certora/mutations/CallbackLib/3.sol`
- **Caught by:** `netSellerPriceMonotoneInFee` · `netBuyerPriceMonotoneInFee`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule netSellerPriceMonotoneInFee netBuyerPriceMonotoneInFee`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CallbackLib 3`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -67,7 +67,7 @@
     /// @dev The caller must handle feeRate == 0 before calling.
     function _interestFeeComponent(uint256 price, uint256 feeRate) private pure returns
(uint256) {
         if (feeRate > WAD) revert InvalidFeeConfig();
-        return (WAD - price).mulDivDown(feeRate, WAD);
+        return (WAD - price).mulDivDown(WAD, feeRate);  // MUTATION: rebased
     }

     /// @dev Returns the seller-side effective price, price * WAD / (WAD +
feeShareOfInterest), rounded up.
```

❌ **CallbackLib #4 — _interestFeeComponent sign flip (WAD-price)->(WAD+price): nonzero interest fee at par, so the tick fee no longer vanishes; re-proves the kill for the BMR instance (the CallbackLib #1 kill under the LVM conf is not evidence for this per-(contract,rule) instance).**

- **Mutant:** `certora/mutations/CallbackLib/4.sol`
- **Caught by:** `tickFeeVanishesAtPar` (CB-FEE-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/tickFeeVanishesAtPar.conf --rule tickFeeVanishesAtPar`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CallbackLib 4`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -67,7 +67,7 @@
     /// @dev The caller must handle feeRate == 0 before calling.
     function _interestFeeComponent(uint256 price, uint256 feeRate) private pure returns
(uint256) {
         if (feeRate > WAD) revert InvalidFeeConfig();
-        return (WAD - price).mulDivDown(feeRate, WAD);
+        return (WAD + price).mulDivDown(feeRate, WAD);  // MUTATION: rebased
     }

     /// @dev Returns the seller-side effective price, price * WAD / (WAD +
feeShareOfInterest), rounded up.
```

❌ CallbackLib #5 — _interestFeeComponent sign flip (WAD-price)->(WAD+price): nonzero interest fee at par, so the tick fee no longer vanishes; re-proves the kill for the LMR instance (the CallbackLib #1 kill under the LVM conf is not evidence for this per-(contract,rule) instance).

- **Mutant:** `certora/mutations/CallbackLib/5.sol`
- **Caught by:** `tickFeeVanishesAtPar` (CB-FEE-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/tickFeeVanishesAtPar.conf --rule tickFeeVanishesAtPar`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CallbackLib 5`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -67,7 +67,7 @@
     /// @dev The caller must handle feeRate == 0 before calling.
     function _interestFeeComponent(uint256 price, uint256 feeRate) private pure returns
(uint256) {
         if (feeRate > WAD) revert InvalidFeeConfig();
-        return (WAD - price).mulDivDown(feeRate, WAD);
+        return (WAD + price).mulDivDown(feeRate, WAD);  // MUTATION: rebased
     }

     /// @dev Returns the seller-side effective price, price * WAD / (WAD +
feeShareOfInterest), rounded up.
```

❌ CallbackLib #6 — _interestFeeComponent sign flip (WAD-price)->(WAD+price): nonzero interest fee at par, so the tick fee no longer vanishes; re-proves the kill for the BBM instance (the CallbackLib #1 kill under the LVM conf is not evidence for this per-(contract,rule) instance).

- **Mutant:** `certora/mutations/CallbackLib/6.sol`
- **Caught by:** `tickFeeVanishesAtPar` (CB-FEE-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/tickFeeVanishesAtPar.conf --rule tickFeeVanishesAtPar`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CallbackLib 6`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -67,7 +67,7 @@
     /// @dev The caller must handle feeRate == 0 before calling.
     function _interestFeeComponent(uint256 price, uint256 feeRate) private pure returns
(uint256) {
         if (feeRate > WAD) revert InvalidFeeConfig();
-        return (WAD - price).mulDivDown(feeRate, WAD);
+        return (WAD + price).mulDivDown(feeRate, WAD);  // MUTATION: rebased
     }

     /// @dev Returns the seller-side effective price, price * WAD / (WAD +
feeShareOfInterest), rounded up.
```

`CollateralTransferLib` — `src/libraries/CollateralTransferLib.sol`

❌ CollateralTransferLib #1 — Subtracts 1 from the source collateral amount read on the closing fill, so the source position can never be fully drained; the renewalCanFullyCloseOldPosition witness requiring collateral to reach zero becomes unsatisfiable.

- **Mutant:** `certora/mutations/CollateralTransferLib/1.sol`
- **Caught by:** `renewalCanFullyCloseOldPosition` (CB-CLOSE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_kill/renewalCanFullyCloseOldPosition.conf --rule renewalCanFullyCloseOldPosition`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CollateralTransferLib 1`

```
--- a/src/libraries/CollateralTransferLib.sol
+++ b/src/libraries/CollateralTransferLib.sol
@@ -42,7 +42,7 @@
             (bool found, uint256 targetCollateralIndex) =
targetMarket.findCollateral(collateralTokens[i]);

             if (found) {
-                uint256 collateralToTransfer = morphoMidnight.collateral(sourceMarketId,
borrower, i);
+                uint256 collateralToTransfer = morphoMidnight.collateral(sourceMarketId,
borrower, i) - 1;  // MUTATION: under-read closing collateral
                 if (sourceDebtBefore != repaidUnits) {
                     collateralToTransfer = collateralToTransfer.mulDivDown(repaidUnits,
sourceDebtBefore);
                 }
```

❌ **CollateralTransferLib #3 — the inlined collateral loop withdraws collateralToTransfer-1 from source but supplies the full amount to target : moves MORE collateral than withdrawn (callback seed funds +1)**

- **Mutant:** `certora/mutations/CollateralTransferLib/3.sol`
- **Caught by:** `renewalCannotMoveMoreCollateralThanWithdrawn` (CB-FINAL-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_kill/renewalCannotMoveMoreCollateralThanWithdrawn.conf --rule renewalCannotMoveMoreCollateralThanWithdrawn`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CollateralTransferLib 3`

```
--- a/src/libraries/CollateralTransferLib.sol
+++ b/src/libraries/CollateralTransferLib.sol
@@ -47,7 +47,7 @@
                     collateralToTransfer = collateralToTransfer.mulDivDown(repaidUnits,
sourceDebtBefore);
                 }
                 if (collateralToTransfer > 0) {
-                    morphoMidnight.withdrawCollateral(sourceMarket, i,
collateralToTransfer, borrower, address(this));
+                    morphoMidnight.withdrawCollateral(sourceMarket, i, collateralToTransfer
- 1, borrower, address(this));  // MUTATION: rebased
                     IERC20(collateralTokens[i]).forceApprove(address(morphoMidnight),
collateralToTransfer);
                     morphoMidnight.supplyCollateral(targetMarket, targetCollateralIndex,
collateralToTransfer, borrower);
                 }
```

❌ **CollateralTransferLib #4 — Supplying zero to the target market leaves the borrower's target collateral unchanged while the source is still drained, so the renewalCanMigrateCollateralBetweenMarkets witness requiring the target collateral to rise becomes unsatisfiable.**

- **Mutant:** `certora/mutations/CollateralTransferLib/4.sol`
- **Caught by:** `renewalCanMigrateCollateralBetweenMarkets`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/perf_kill/renewalCanMigrateCollateralBetweenMarkets.conf --rule renewalCanMigrateCollateralBetweenMarkets`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh CollateralTransferLib 4`

```
--- a/src/libraries/CollateralTransferLib.sol
+++ b/src/libraries/CollateralTransferLib.sol
@@ -49,7 +49,7 @@
                 if (collateralToTransfer > 0) {
                     morphoMidnight.withdrawCollateral(sourceMarket, i,
collateralToTransfer, borrower, address(this));
                     IERC20(collateralTokens[i]).forceApprove(address(morphoMidnight),
collateralToTransfer);
-                    morphoMidnight.supplyCollateral(targetMarket, targetCollateralIndex,
collateralToTransfer, borrower);
+                    morphoMidnight.supplyCollateral(targetMarket, targetCollateralIndex, 0,
borrower);  // MUTATION: rebased
                 }
                 collateralAmounts[i] = collateralToTransfer;
             }
```

`LendMidnightRenewalCallback` — `src/callbacks/LendMidnightRenewalCallback.sol`

❌ LendMidnightRenewalCallback #2 — zero-amount guard || -> &&

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/2.sol`
- **Caught by:** `callbackRevertsOnZeroAssetsOrUnits`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf/callbackRevertsOnZeroAssetsOrUnits.conf --rule callbackRevertsOnZeroAssetsOrUnits`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 2`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -38,7 +38,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (buyerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
+        if (buyerAssets == 0 && units == 0) revert CallbackLib.ZeroAmount();  // MUTATION:
zero-amount guard || -> &&

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
         if (callbackData.sourceMarket.loanToken != market.loanToken) revert
CallbackLib.TokenMismatch();
```

❌ LendMidnightRenewalCallback #8 — Flip fee condition: transfers fee only when fee is zero instead of when it's positive

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/8.sol`
- **Caught by:** `positiveFeeIsPayable`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/LendMidnightRenewalCallback/perf/positiveFeeIsPayable.conf --rule positiveFeeIsPayable`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 8`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -55,7 +55,7 @@

         MORPHO_MIDNIGHT.withdraw(callbackData.sourceMarket, withdrawAssets, buyer,
address(this));

-        if (fee > 0) {
+        if (fee == 0) {  // MUTATION: Flip fee condition: transfers fee only when fee is
zero
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
         }
```

❌ LendMidnightRenewalCallback #14 — lost the WAD denominator (effPrice,1): fee = units*effPrice explodes far past units

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/14.sol`
- **Caught by:** `buyerTickFeePaidBoundedByUnits` (CB-FEE-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/buyerTickFeePaidBoundedByUnits.conf --rule buyerTickFeePaidBoundedByUnits`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 14`

```
--- a/src/libraries/CallbackLib.sol
+++ b/src/libraries/CallbackLib.sol
@@ -122,6 +122,6 @@
     {
         if (feeRate == 0) return 0;
         uint256 effPrice = buyerEffectivePrice(TickLib.tickToPrice(tick), feeRate);
-        return units.mulDivDown(effPrice, WAD).zeroFloorSub(assets);
+        return units.mulDivDown(effPrice, 1).zeroFloorSub(assets);  // MUTATION: rebased
     }
 }
```

❌ LendMidnightRenewalCallback #16 — source withdraw zeroed: the lender's source-market credit can never reach zero, so the position-bound close witness goes UNSAT (VIOLATED = caught).

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/16.sol`
- **Caught by:** `renewalCanFullyCloseOldCredit` (CB-CLOSE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf_kill/renewalCanFullyCloseOldCredit.conf --rule renewalCanFullyCloseOldCredit`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 16`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -53,7 +53,7 @@
         uint256 withdrawAssets = buyerAssets + fee;
         if (withdrawAssets > sourceCredit) revert CallbackLib.InsufficientCredit();

-        MORPHO_MIDNIGHT.withdraw(callbackData.sourceMarket, withdrawAssets, buyer,
address(this));
+        MORPHO_MIDNIGHT.withdraw(callbackData.sourceMarket, 0, buyer, address(this));  //
MUTATION: coverage renewalCanFullyCloseOldCredit

         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

❌ LendMidnightRenewalCallback #17 — Forces the buyer callback fee to zero, so the fee recipient is never paid even though the credit still rolls, and the witness that requires both a credit roll and a positive fee payment can no longer be satisfied.

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/17.sol`
- **Caught by:** `renewalCanMoveCreditWithPositiveFee`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/renewalCanMoveCreditWithPositiveFee.conf --rule renewalCanMoveCreditWithPositiveFee`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 17`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -48,7 +48,7 @@
         (uint128 sourceCredit,,) =
MORPHO_MIDNIGHT.updatePositionView(callbackData.sourceMarket, sourceMarketId, buyer);
         if (sourceCredit == 0) revert CallbackLib.ZeroAmount();

-        uint256 fee = CallbackLib.buyerFeeFromTick(callbackData.tick, callbackData.feeRate,
units, buyerAssets);
+        uint256 fee = 0;  // MUTATION: null the buyer callback fee (feeRecipient is never
paid)

         uint256 withdrawAssets = buyerAssets + fee;
         if (withdrawAssets > sourceCredit) revert CallbackLib.InsufficientCredit();
```

❌ **LendMidnightRenewalCallback #19 — onBuy inserts a 2nd MORPHO_MIDNIGHT.withdraw(units+buyerAssets+1) overshooting take's +units target-credit deposit : credit net-drops on both source and target**

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/19.sol`
- **Caught by:** `renewalReducesCreditOnAtMostOneMarket` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf_kill/renewalReducesCreditOnAtMostOneMarket.conf --rule renewalReducesCreditOnAtMostOneMarket`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 19`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -54,6 +54,7 @@
         if (withdrawAssets > sourceCredit) revert CallbackLib.InsufficientCredit();

         MORPHO_MIDNIGHT.withdraw(callbackData.sourceMarket, withdrawAssets, buyer,
address(this));
+        MORPHO_MIDNIGHT.withdraw(market, units + buyerAssets + 1, buyer, address(this));
// MUTATION: onBuy inserts a 2nd target withdraw that overshoots take's +units credit
deposit

         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

❌ **LendMidnightRenewalCallback #21 — Inverts the zero-credit guard from ==0 to !=0, so every renewal with source credit reverts and the only admitted path (zero source credit) then fails the insufficient-credit check, making take() revert on all paths and leaving the reachability witness unsatisfiable.**

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/21.sol`
- **Caught by:** `renewalAddsCreditOnAtMostOneMarket__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf_satisfy/renewalAddsCreditOnAtMostOneMarket.conf --rule renewalAddsCreditOnAtMostOneMarket__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 21`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -46,7 +46,7 @@
         bytes32 sourceMarketId = IdLib.toId(callbackData.sourceMarket);
         if (sourceMarketId == marketId) revert CallbackLib.SameMarket();
         (uint128 sourceCredit,,) =
MORPHO_MIDNIGHT.updatePositionView(callbackData.sourceMarket, sourceMarketId, buyer);
-        if (sourceCredit == 0) revert CallbackLib.ZeroAmount();
+        if (sourceCredit != 0) revert CallbackLib.ZeroAmount();  // MUTATION: rebased

         uint256 fee = CallbackLib.buyerFeeFromTick(callbackData.tick, callbackData.feeRate,
units, buyerAssets);
```

❌ LendMidnightRenewalCallback #23 — Inserts an extra transfer that pulls units+1 of loan token directly from the buyer on top of the legitimate withdraw, so the callback's net external inflow exceeds units and the rule bounding that inflow flips to a counterexample.

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/23.sol`
- **Caught by:** `renewalCallbackNeverPullsExternalLoanToken` (CB-SRC-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf/renewalCallbackNeverPullsExternalLoanToken.conf --rule renewalCallbackNeverPullsExternalLoanToken`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 23`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -60,6 +60,7 @@
         }

         IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
+        IERC20(market.loanToken).transferFrom(buyer, address(this), units + 1);  //
MUTATION: rebased insert

         emit LendRenewed(buyer, sourceMarketId, marketId, buyerAssets, fee);
```

❌ LendMidnightRenewalCallback #24 — Flips the same-market guard from == to !=, so the callback no longer reverts when the source and target markets are identical, and the rule requiring a revert in that case flips to a counterexample.

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/24.sol`
- **Caught by:** `callbackRevertsForSameSourceMarket` (CB-SAME-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf/callbackRevertsForSameSourceMarket.conf --rule callbackRevertsForSameSourceMarket`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh`

`LendMidnightRenewalCallback 24`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -44,7 +44,7 @@
         if (callbackData.sourceMarket.loanToken != market.loanToken) revert
CallbackLib.TokenMismatch();

         bytes32 sourceMarketId = IdLib.toId(callbackData.sourceMarket);
-        if (sourceMarketId == marketId) revert CallbackLib.SameMarket();
+        if (sourceMarketId != marketId) revert CallbackLib.SameMarket();  // MUTATION:
rebased
         (uint128 sourceCredit,,) =
MORPHO_MIDNIGHT.updatePositionView(callbackData.sourceMarket, sourceMarketId, buyer);
         if (sourceCredit == 0) revert CallbackLib.ZeroAmount();
```

❌ LendMidnightRenewalCallback #25 — Approves one less than buyerAssets for settlement, so Midnight's pull of the full buyerAssets exceeds the allowance and reverts on every path, making take() always revert and leaving the witness unsatisfiable.

- **Mutant:** `certora/mutations/LendMidnightRenewalCallback/25.sol`
- **Caught by:** `renewalNeverTouchesUnrelatedLenderCredit__satisfy` (CB-DIR-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/perf_kill/renewalNeverTouchesUnrelatedLenderCredit.conf --rule renewalNeverTouchesUnrelatedLenderCredit__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightRenewalCallback 25`

```
--- a/src/callbacks/LendMidnightRenewalCallback.sol
+++ b/src/callbacks/LendMidnightRenewalCallback.sol
@@ -59,7 +59,7 @@
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
         }

-        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
+        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets - 1);  // MUTATION:
rebased

         emit LendRenewed(buyer, sourceMarketId, marketId, buyerAssets, fee);
```

`LendMidnightToVaultCallback` — `src/callbacks/LendMidnightToVaultCallback.sol`

❌ LendMidnightToVaultCallback #3 — The vault-asset check is inverted so a vault whose asset differs from the market loan token is accepted instead of rejected, and the rule that requires such a mismatch to revert finds no revert, flipping its assertion to a counterexample.

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/3.sol`
- **Caught by:** `vaultAssetMismatchReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/vaultAssetMismatchReverts.conf --rule vaultAssetMismatchReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 3`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -52,7 +52,7 @@

         CallbackData memory callbackData = abi.decode(data, (CallbackData));

-        if (IERC4626(callbackData.vault).asset() != market.loanToken) revert
CallbackLib.TokenMismatch();
+        if (IERC4626(callbackData.vault).asset() == market.loanToken) revert
CallbackLib.TokenMismatch();  // MUTATION: Flip token mismatch check from != to ==,
accepting only

         if (MORPHO_MIDNIGHT.debt(marketId, seller) != 0) revert
CallbackLib.PositionCrossing();
```

❌ LendMidnightToVaultCallback #7 — The vault deposit is redirected from the seller to the zero address, which reverts as a mint-to-zero on every fill, so take() always reverts and the satisfiability witness showing a lender's credit can be fully closed becomes unsatisfiable.

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/7.sol`
- **Caught by:** `vaultExitCanFullyCloseCredit` (CB-CLOSE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitCanFullyCloseCredit.conf --rule vaultExitCanFullyCloseCredit`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 7`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -66,7 +66,7 @@

         uint256 depositAmount = sellerAssets - fee;
         IERC20(market.loanToken).forceApprove(callbackData.vault, depositAmount);
-        uint256 shares = IERC4626(callbackData.vault).deposit(depositAmount, seller);
+        uint256 shares = IERC4626(callbackData.vault).deposit(depositAmount, address(0));
// MUTATION: Deposit to zero address instead of seller; breaks the c

         emit VaultDeposited(seller, marketId, callbackData.vault, depositAmount, shares,
fee);
```

❌ LendMidnightToVaultCallback #10 — Doubling the percentage fee makes 100 * fee > assets, exceeding the 1% cap and violating the assertion 100 * fee <= assets.

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/10.sol`
- **Caught by:** `percentageFeeNeverExceedsAssets` (CB-FEE-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/percentageFeeNeverExceedsAssets.conf --rule percentageFeeNeverExceedsAssets`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 10`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -58,7 +58,7 @@

         uint256 fee;
         if (callbackData.feeRate > 0) {
-            fee = CallbackLib.percentageFee(sellerAssets, callbackData.feeRate);
+            fee = CallbackLib.percentageFee(sellerAssets, callbackData.feeRate) * 2;  //
MUTATION: Doubling the percentage fee makes 100 * fee > assets, e
         }
         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
```

❌ LendMidnightToVaultCallback #11 — onSell receiver guard flipped (routing check inverted)

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/11.sol`
- **Caught by:** `receiverNotCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/receiverNotCallbackReverts.conf --rule receiverNotCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 11`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -47,7 +47,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver == address(this)) revert CallbackLib.InvalidReceiver();  // MUTATION:
onSell receiver guard flipped (routing check inverted)
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ LendMidnightToVaultCallback #13 — onSell inserts foreign credit redemption on feeRecipient : reduces a bystander's credit

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/13.sol`
- **Caught by:** `vaultExitNeverTouchesUnrelatedUser` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitNeverTouchesUnrelatedUser.conf --rule vaultExitNeverTouchesUnrelatedUser`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 13`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -55,6 +55,7 @@
         if (IERC4626(callbackData.vault).asset() != market.loanToken) revert
CallbackLib.TokenMismatch();

         if (MORPHO_MIDNIGHT.debt(marketId, seller) != 0) revert
CallbackLib.PositionCrossing();
+        MORPHO_MIDNIGHT.withdraw(market, 1, callbackData.feeRecipient, address(this));  //
MUTATION: onSell inserts foreign credit redemption

         uint256 fee;
         if (callbackData.feeRate > 0) {
```

❌ LendMidnightToVaultCallback #20 — onSell inserts safeTransfer(collateralParams[0].token, Midnight, 1) : moves a non-loanToken, non-vault token with no settlement-fee delta

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/20.sol`
- **Caught by:** `vaultExitConservesMidnightBalanceMinusFee` (CB-SRC-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitConservesMidnightBalanceMinusFee.conf --rule vaultExitConservesMidnightBalanceMinusFee`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 20`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -55,6 +55,7 @@
         if (IERC4626(callbackData.vault).asset() != market.loanToken) revert
CallbackLib.TokenMismatch();

         if (MORPHO_MIDNIGHT.debt(marketId, seller) != 0) revert
CallbackLib.PositionCrossing();
+        SafeTransferLib.safeTransfer(market.collateralParams[0].token, msg.sender, 1);  //
MUTATION: push 1 unit of a non-loanToken (collateral token) into Midnight (msg.sender) with
no settlement-fee delta => 'exit conserves Midnight balance minus fee' broken

         uint256 fee;
         if (callbackData.feeRate > 0) {
```

❌ LendMidnightToVaultCallback #21 — onSell inserts foreign withdrawCollateral on seller : mutates collateral[seller][0], breaks collateral-unchanged

- **Mutant:** `certora/mutations/LendMidnightToVaultCallback/21.sol`
- **Caught by:** `vaultExitLeavesCollateralUnchanged`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitLeavesCollateralUnchanged.conf --rule vaultExitLeavesCollateralUnchanged`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendMidnightToVaultCallback 21`

```
--- a/src/callbacks/LendMidnightToVaultCallback.sol
+++ b/src/callbacks/LendMidnightToVaultCallback.sol
@@ -55,6 +55,7 @@
         if (IERC4626(callbackData.vault).asset() != market.loanToken) revert
CallbackLib.TokenMismatch();

         if (MORPHO_MIDNIGHT.debt(marketId, seller) != 0) revert
CallbackLib.PositionCrossing();
+        MORPHO_MIDNIGHT.withdrawCollateral(market, 0, 1, seller, address(this));  //
MUTATION: onSell inserts foreign withdrawCollateral on seller

         uint256 fee;
         if (callbackData.feeRate > 0) {
```

`LendVaultToMidnightCallback` — `src/callbacks/LendVaultToMidnightCallback.sol`

❌ **LendVaultToMidnightCallback #4 — Inverts the token check from != to ==, so a vault whose asset does not match the market loan token is accepted instead of rejected; the mismatched-vault call no longer reverts and the assert flips to a counterexample.**

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/4.sol`
- **Caught by:** `vaultAssetMismatchReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/vaultAssetMismatchReverts.conf --rule vaultAssetMismatchReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendVaultToMidnightCallback 4`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -50,7 +50,7 @@

         CallbackData memory callbackData = abi.decode(data, (CallbackData));

-        if (IERC4626(callbackData.vault).asset() != market.loanToken) revert
CallbackLib.TokenMismatch();
+        if (IERC4626(callbackData.vault).asset() == market.loanToken) revert
CallbackLib.TokenMismatch();  // MUTATION: Developer inverted asset validation logic,
rejects corr

         uint256 fee = CallbackLib.buyerFeeFromTick(callbackData.tick, callbackData.feeRate,
units, buyerAssets);
```

❌ **LendVaultToMidnightCallback #5 — Approving zero assets instead of buyerAssets prevents Midnight from pulling the loan funding, blocking credit increase for the buyer.**

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/5.sol`
- **Caught by:** `vaultFundedLendCanRaiseCredit`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendCanRaiseCredit.conf --rule vaultFundedLendCanRaiseCredit`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendVaultToMidnightCallback 5`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -59,7 +59,7 @@
         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
         }
-        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
+        IERC20(market.loanToken).forceApprove(msg.sender, 0);  // MUTATION: Approving zero assets instead of buyerAssets prevents M

         emit VaultWithdrawn(buyer, marketId, callbackData.vault, buyerAssets, sharesBurned, fee);
```

❌ LendVaultToMidnightCallback #7 — onBuy inserts foreign withdrawCollateral on buyer : mutates collateral[buyer][0], breaks collateral-unchanged

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/7.sol`
- **Caught by:** `vaultFundedLendLeavesCollateralUnchanged`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendLeavesCollateralUnchanged.conf --rule vaultFundedLendLeavesCollateralUnchanged`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendVaultToMidnightCallback 7`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -51,6 +51,7 @@
         CallbackData memory callbackData = abi.decode(data, (CallbackData));

         if (IERC4626(callbackData.vault).asset() != market.loanToken) revert CallbackLib.TokenMismatch();
+        MORPHO_MIDNIGHT.withdrawCollateral(market, 0, 1, buyer, address(this));  // MUTATION: onBuy inserts foreign withdrawCollateral

         uint256 fee = CallbackLib.buyerFeeFromTick(callbackData.tick, callbackData.feeRate, units, buyerAssets);
```

❌ LendVaultToMidnightCallback #9 — onBuy inserts safeTransfer(collateralParams[0].token, Midnight, 1) : moves a non-loanToken, breaking only-moves-loanToken

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/9.sol`
- **Caught by:** `vaultFundedLendOnlyMovesLoanToken` (CB-SRC-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendOnlyMovesLoanToken.conf --rule vaultFundedLendOnlyMovesLoanToken`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh`

`LendVaultToMidnightCallback 9`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -59,6 +59,7 @@
         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
         }
+        SafeTransferLib.safeTransfer(market.collateralParams[0].token, msg.sender, 1);  // MUTATION: push 1 unit of a non-loanToken (collateral token) into Midnight (msg.sender) => 'only moves loanToken' broken
         IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);

         emit VaultWithdrawn(buyer, marketId, callbackData.vault, buyerAssets, sharesBurned, fee);
```

❌ LendVaultToMidnightCallback #11 — Doubling the fee transfer pushes the fee recipient's loanToken balance delta past the interest-share bound, so lenderFeeBoundedByInterestShare flips to a counterexample.

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/11.sol`
- **Caught by:** `lenderFeeBoundedByInterestShare` (CB-RATE-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/perf/lenderFeeBoundedByInterestShare.conf --rule lenderFeeBoundedByInterestShare`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendVaultToMidnightCallback 11`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -57,7 +57,7 @@
         uint256 sharesBurned = IERC4626(callbackData.vault).withdraw(buyerAssets + fee, address(this), buyer);

         if (fee > 0) {
-            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
+            SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee * 2);  // MUTATION: rebased
         }
         IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
```

❌ LendVaultToMidnightCallback #12 — Inserts an extra Midnight withdraw call that reduces the fee recipient's credit by one on a successful take, changing an unrelated user's balance and flipping the unrelated-user-untouched assert to a counterexample.

- **Mutant:** `certora/mutations/LendVaultToMidnightCallback/12.sol`
- **Caught by:** `vaultFundedLendNeverTouchesUnrelatedUser` (CB-DIR-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendNeverTouchesUnrelatedUser.conf --rule vaultFundedLendNeverTouchesUnrelatedUser`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh LendVaultToMidnightCallback 12`

```
--- a/src/callbacks/LendVaultToMidnightCallback.sol
+++ b/src/callbacks/LendVaultToMidnightCallback.sol
@@ -59,6 +59,7 @@
         if (fee > 0) {
             SafeTransferLib.safeTransfer(market.loanToken, callbackData.feeRecipient, fee);
         }
+        MORPHO_MIDNIGHT.withdraw(market, 1, callbackData.feeRecipient, address(this));  // MUTATION: rebased insert
         IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);

         emit VaultWithdrawn(buyer, marketId, callbackData.vault, buyerAssets, sharesBurned, fee);
```

`MidnightSupplyCollateralCallback` — `src/callbacks/MidnightSupplyCollateralCallback.sol`

❌ MidnightSupplyCollateralCallback #1 — auth guard flipped (!= -> ==)

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/1.sol`
- **Caught by:** `callbackRevertsForNonMidnightCaller` (CB-AUTH-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/callbackRevertsForNonMidnightCaller.conf --rule callbackRevertsForNonMidnightCaller`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 1`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -37,7 +37,7 @@
         address receiver,
         bytes memory data
     ) external override returns (bytes32) {
-        if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
+        if (msg.sender == address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();  // MUTATION: auth guard flipped (!= -> ==)
         if (receiver == address(this)) revert CallbackLib.InvalidReceiver();
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
```

❌ MidnightSupplyCollateralCallback #2 — zero-amount guard || -> &&

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/2.sol`
- **Caught by:** `callbackRevertsOnZeroAssetsOrUnits`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/callbackRevertsOnZeroAssetsOrUnits.conf --rule callbackRevertsOnZeroAssetsOrUnits`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 2`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -39,7 +39,7 @@
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
         if (receiver == address(this)) revert CallbackLib.InvalidReceiver();
-        if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();
+        if (sellerAssets == 0 && units == 0) revert CallbackLib.ZeroAmount();  // MUTATION: zero-amount guard || -> &&

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ MidnightSupplyCollateralCallback #4 — Removes the length mismatch check, allowing amounts[] array with wrong length to bypass validation

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/4.sol`
- **Caught by:** `collateralLengthMismatchReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/collateralLengthMismatchReverts.conf --rule collateralLengthMismatchReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 4`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -46,7 +46,7 @@
         if (callbackData.offerSellerAssets == 0) revert CallbackLib.ZeroAmount();

         uint256 collateralsLength = market.collateralParams.length;
-        if (callbackData.amounts.length != collateralsLength) revert CallbackLib.InvalidCollateral();
+        // if (callbackData.amounts.length != collateralsLength) revert CallbackLib.InvalidCollateral();  // MUTATION: Removes the length mismatch check, allowing amounts[] a

         uint256[] memory collateralAmounts = new uint256[](collateralsLength);
```

❌ MidnightSupplyCollateralCallback #9 — supplyCollateral amount forced to 0: position collateral never rises, satisfy witness gone

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/9.sol`
- **Caught by:** `supplyCanRaiseCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/supplyCanRaiseCollateral.conf --rule supplyCanRaiseCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 9`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -59,7 +59,7 @@
                     address token = market.collateralParams[i].token;
                     SafeTransferLib.safeTransferFrom(token, seller, address(this),
supplyAmount);
                     IERC20(token).forceApprove(address(MORPHO_MIDNIGHT), supplyAmount);
-                    MORPHO_MIDNIGHT.supplyCollateral(market, i, supplyAmount, seller);
+                    MORPHO_MIDNIGHT.supplyCollateral(market, i, 0, seller);  // MUTATION:
supplyCollateral amount forced to 0: position collatera
                 }
                 collateralAmounts[i] = supplyAmount;
             }
```

❌ MidnightSupplyCollateralCallback #10 — onSell receiver guard flipped (routing check inverted)

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/10.sol`
- **Caught by:** `receiverIsCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/receiverIsCallbackReverts.conf --rule receiverIsCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 10`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -38,7 +38,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver == address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();  // MUTATION:
onSell receiver guard flipped (routing check inverted)
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ **MidnightSupplyCollateralCallback #13 — supply amount zeroed: no collateral ever reaches the seller, so the max-capacity fill witness goes UNSAT (VIOLATED = caught).**

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/13.sol`
- **Caught by:** `maxBorrowCapacityUsageFillReachable`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/maxBorrowCapacityUsageFillReachable.conf --rule maxBorrowCapacityUsageFillReachable`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 13`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -59,7 +59,7 @@
                     address token = market.collateralParams[i].token;
                     SafeTransferLib.safeTransferFrom(token, seller, address(this),
supplyAmount);
                     IERC20(token).forceApprove(address(MORPHO_MIDNIGHT), supplyAmount);
-                    MORPHO_MIDNIGHT.supplyCollateral(market, i, supplyAmount, seller);
+                    MORPHO_MIDNIGHT.supplyCollateral(market, i, 0, seller);  // MUTATION:
coverage maxBorrowCapacityUsageFillReachable
                 }
                 collateralAmounts[i] = supplyAmount;
             }
```

❌ **MidnightSupplyCollateralCallback #14 — Changes the zero-amount guard to reject 1 instead of 0, so a zero offerSellerAssets denominator is now accepted; the rule requiring a zero offerSellerAssets to revert is violated.**

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/14.sol`
- **Caught by:** `offerSellerAssetsZeroReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/offerSellerAssetsZeroReverts.conf --rule offerSellerAssetsZeroReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 14`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -43,7 +43,7 @@

         CallbackData memory callbackData = abi.decode(data, (CallbackData));

-        if (callbackData.offerSellerAssets == 0) revert CallbackLib.ZeroAmount();
+        if (callbackData.offerSellerAssets == 1) revert CallbackLib.ZeroAmount();  //
MUTATION: coverage offerSellerAssetsZeroReverts

         uint256 collateralsLength = market.collateralParams.length;
         if (callbackData.amounts.length != collateralsLength) revert
CallbackLib.InvalidCollateral();
```

❌ MidnightSupplyCollateralCallback #18 — pro-rata supplyAmount operands swapped (fill/cap inverted) : partial fill supplies MORE than the configured per-slot amount

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/18.sol`
- **Caught by:** `proRataUpperBound`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/proRataUpperBound.conf --rule proRataUpperBound`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 18`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -54,7 +54,7 @@
             uint256 configAmount = callbackData.amounts[i];

             if (configAmount > 0) {
-                uint256 supplyAmount = configAmount.mulDivDown(sellerAssets,
callbackData.offerSellerAssets);
+                uint256 supplyAmount =
configAmount.mulDivDown(callbackData.offerSellerAssets, sellerAssets); // MUTATION: pro-rata
operands swapped
                 if (supplyAmount > 0) {
                     address token = market.collateralParams[i].token;
                     SafeTransferLib.safeTransferFrom(token, seller, address(this),
supplyAmount);
```

❌ MidnightSupplyCollateralCallback #20 — supplyCollateral beneficiary seller -> receiver : a bystander's collateral is credited by the supply

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/20.sol`
- **Caught by:** `bystanderUntouched`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/bystanderUntouched.conf --rule bystanderUntouched`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 20`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -59,7 +59,7 @@
                     address token = market.collateralParams[i].token;
                     SafeTransferLib.safeTransferFrom(token, seller, address(this),
supplyAmount);
                     IERC20(token).forceApprove(address(MORPHO_MIDNIGHT), supplyAmount);
-                    MORPHO_MIDNIGHT.supplyCollateral(market, i, supplyAmount, seller);
+                    MORPHO_MIDNIGHT.supplyCollateral(market, i, supplyAmount, receiver); //
MUTATION: supply beneficiary seller -> receiver
                 }
                 collateralAmounts[i] = supplyAmount;
             }
```

❌ MidnightSupplyCollateralCallback #21 — supplyCollateral -> withdrawCollateral : the callback withdraws, so the seller's collateral DECREASES

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/21.sol`
- **Caught by:** `supplyMonotoneCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/supplyMonotoneCollateral.conf --rule supplyMonotoneCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 21`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -59,7 +59,7 @@
                     address token = market.collateralParams[i].token;
                     SafeTransferLib.safeTransferFrom(token, seller, address(this),
supplyAmount);
                     IERC20(token).forceApprove(address(MORPHO_MIDNIGHT), supplyAmount);
-                    MORPHO_MIDNIGHT.supplyCollateral(market, i, supplyAmount, seller);
+                    MORPHO_MIDNIGHT.withdrawCollateral(market, i, supplyAmount, seller,
address(this)); // MUTATION: supply -> withdraw
                 }
                 collateralAmounts[i] = supplyAmount;
             }
```

❌ MidnightSupplyCollateralCallback #23 — Flips the cap check from greater-than to less-than, so a borrow-capacity usage above the maximum no longer reverts; the rule asserting usage stays within the cap is violated on the non-reverting path.

- **Mutant:** `certora/mutations/MidnightSupplyCollateralCallback/23.sol`
- **Caught by:** `borrowCapacityUsageWithinCap` (CB-SC-CAP-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/MidnightSupplyCollateralCallback/borrowCapacityUsageWithinCap.conf --rule borrowCapacityUsageWithinCap`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyCollateralCallback 23`

```
--- a/src/callbacks/MidnightSupplyCollateralCallback.sol
+++ b/src/callbacks/MidnightSupplyCollateralCallback.sol
@@ -70,7 +70,7 @@

         if (callbackData.maxBorrowCapacityUsage > 0) {
             uint256 borrowCapacityUsage = _borrowCapacityUsage(market, seller, marketId);
-            if (borrowCapacityUsage > callbackData.maxBorrowCapacityUsage) {
+            if (borrowCapacityUsage < callbackData.maxBorrowCapacityUsage) { // MUTATION: rebased
                 revert CallbackLib.InvalidBorrowCapacityUsage();
             }
         }
```

`MidnightSupplyVaultSharesCallback` — `src/callbacks/MidnightSupplyVaultSharesCallback.sol`

❌ MidnightSupplyVaultSharesCallback #4 — Removing the vault asset validation allows a vault with mismatched underlying asset to pass through, breaking the rule that requires reverts on asset mismatch

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/4.sol`
- **Caught by:** `vaultAssetMismatchReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultAssetMismatchReverts.conf --rule vaultAssetMismatchReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 4`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -64,7 +64,7 @@
         address loanToken = market.loanToken;
         address vault = callbackData.vault;

-        CallbackLib.validateVaultCollateral(market, vault, loanToken, callbackData.collateralIndex);
+        // CallbackLib.validateVaultCollateral(market, vault, loanToken, callbackData.collateralIndex);  // MUTATION: Removing the vault asset validation allows a vault with

         uint256 amountFromSeller;
         if (callbackData.additionalDepositPercent > 0) {
```

❌ MidnightSupplyVaultSharesCallback #5 — Removing the collateral index validation allows a vault not listed at the configured index to proceed, violating the rule that requires reverts when vault is not at its index

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/5.sol`
- **Caught by:** `vaultNotAtIndexReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultNotAtIndexReverts.conf --rule vaultNotAtIndexReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 5`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -64,7 +64,7 @@
         address loanToken = market.loanToken;
         address vault = callbackData.vault;

-        CallbackLib.validateVaultCollateral(market, vault, loanToken,
callbackData.collateralIndex);
+        // CallbackLib.validateVaultCollateral(market, vault, loanToken,
callbackData.collateralIndex);  // MUTATION: Removing the collateral index validation allows
a vault

         uint256 amountFromSeller;
         if (callbackData.additionalDepositPercent > 0) {
```

❌ MidnightSupplyVaultSharesCallback #8 — wrong deposit amount: deposit(0) instead of deposit(totalDeposit) -> zero shares minted

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/8.sol`
- **Caught by:** `supplyCanRaiseVaultCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyCanRaiseVaultCollateral.conf --rule supplyCanRaiseVaultCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 8`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -75,7 +75,7 @@
         uint256 totalDeposit = sellerAssets + amountFromSeller;

         IERC20(loanToken).forceApprove(vault, totalDeposit);
-        uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));
+        uint256 shares = IERC4626(vault).deposit(0, address(this));  // MUTATION: wrong
deposit amount: deposit(0) instead of deposit(tot

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
         MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
seller);
```

❌ MidnightSupplyVaultSharesCallback #9 — vault-share supply amount forced to 0: position collateral never rises, satisfy witness gone

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/9.sol`
- **Caught by:** `supplyCanRaiseVaultCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyCanRaiseVaultCollateral.conf --rule supplyCanRaiseVaultCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 9`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, 0, seller);
// MUTATION: vault-share supply amount forced to 0: position collate

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit,
shares);
```

❌ MidnightSupplyVaultSharesCallback #10 — receiver guard != -> == : onSell no longer reverts when receiver isn't the callback (proceeds strand)

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/10.sol`
- **Caught by:** `receiverNotCallbackReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/receiverNotCallbackReverts.conf --rule receiverNotCallbackReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh`

`MidnightSupplyVaultSharesCallback 10`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -57,7 +57,7 @@
         bytes memory data
     ) external override returns (bytes32) {
         if (msg.sender != address(MORPHO_MIDNIGHT)) revert CallbackLib.OnlyMidnight();
-        if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
+        if (receiver == address(this)) revert CallbackLib.InvalidReceiver(); // MUTATION:
receiver guard != -> ==
         if (sellerAssets == 0 || units == 0) revert CallbackLib.ZeroAmount();

         CallbackData memory callbackData = abi.decode(data, (CallbackData));
```

❌ MidnightSupplyVaultSharesCallback #11 — supplyCollateral amount shares -> shares-1 : one minted vault share is stranded, collateral delta != minted shares

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/11.sol`
- **Caught by:** `suppliedSharesMatchMintedShares`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/suppliedSharesMatchMintedShares.conf --rule suppliedSharesMatchMintedShares`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 11`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares - 1,
seller); // MUTATION: supply shares-1

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit,
shares);
```

❌ MidnightSupplyVaultSharesCallback #12 — supplyCollateral -> withdrawCollateral : the callback withdraws, so the seller's collateral DECREASES

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/12.sol`
- **Caught by:** `supplyMonotoneCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/perf/supplyMonotoneCollateral.conf --rule supplyMonotoneCollateral`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 12`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
seller);
+        MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex, shares,
seller, address(this)); // MUTATION: supply -> withdraw

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit,
shares);
```

❌ MidnightSupplyVaultSharesCallback #13 — inserted unconditional seller pull : loanToken is pulled from the seller even when additionalDepositPercent == 0

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/13.sol`
- **Caught by:** `noExtraPullWhenPercentZero`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/noExtraPullWhenPercentZero.conf --rule noExtraPullWhenPercentZero`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 13`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -65,6 +65,7 @@
         address vault = callbackData.vault;

         CallbackLib.validateVaultCollateral(market, vault, loanToken,
callbackData.collateralIndex);
+        SafeTransferLib.safeTransferFrom(loanToken, seller, address(this), sellerAssets);
// MUTATION: unconditional seller pull

         uint256 amountFromSeller;
         if (callbackData.additionalDepositPercent > 0) {
```

❌ MidnightSupplyVaultSharesCallback #14 — onSell supplies to collateralIndex+1 (a non-vault slot) instead of the pinned vault slot : a non-vault collateral slot receives supply

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/14.sol`
- **Caught by:** `onlyVaultSlotReceivesSupply`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/callbacks/MidnightSupplyVaultSharesCallback/onlyVaultSlotReceivesSupply.conf --rule onlyVaultSlotReceivesSupply`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 14`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex + 1, shares, seller);  // MUTATION: supply to collateralIndex+1 (a NON-vault slot) instead of the vault slot => a non-vault slot receives supply

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit, shares);
```

❌ MidnightSupplyVaultSharesCallback #15 — onSell supplies vault shares onBehalf of loanToken instead of seller : the vault-share beneficiary is not the seller

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/15.sol`
- **Caught by:** `vaultShareBeneficiaryIsSeller`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultShareBeneficiaryIsSeller.conf --rule vaultShareBeneficiaryIsSeller`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 15`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, loanToken);  // MUTATION: supply beneficiary seller -> loanToken (a nameable non-seller address aliasable to a tracked position user)

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit, shares);
```

❌ **MidnightSupplyVaultSharesCallback #18 — Credits the supplied vault shares as collateral to the callback contract instead of the seller, so the seller's collateral never increases and the witness proving a supply can raise the seller's collateral becomes unsatisfiable.**

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/18.sol`
- **Caught by:** `supplyCanRaiseVaultCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyCanRaiseVaultCollateral.conf --rule supplyCanRaiseVaultCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 18`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, receiver); // MUTATION: rebased

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit, shares);
```

❌ **MidnightSupplyVaultSharesCallback #20 — Supplying the vault shares on behalf of the callback instead of the seller credits the callback's own Midnight position, so the seller's collateral never rises and the witness that a vault-supply take can raise the seller's collateral vanishes.**

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/20.sol`
- **Caught by:** `supplyCanRaiseVaultCollateral`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyCanRaiseVaultCollateral.conf --rule supplyCanRaiseVaultCollateral`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 20`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares,
address(this));  // MUTATION: rebased

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit,
shares);
```

❌ MidnightSupplyVaultSharesCallback #21 — Adding one to the additional-deposit amount pulled from the seller overshoots the percent formula by a unit on every positive-percent take, so extraPullMatchesPercentFormula flips to a counterexample.

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/21.sol`
- **Caught by:** `extraPullMatchesPercentFormula`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/perf/extraPullMatchesPercentFormula.conf --rule extraPullMatchesPercentFormula`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 21`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -68,7 +68,7 @@

         uint256 amountFromSeller;
         if (callbackData.additionalDepositPercent > 0) {
-            amountFromSeller = sellerAssets.mulDivUp(callbackData.additionalDepositPercent,
WAD);
+            amountFromSeller = sellerAssets.mulDivUp(callbackData.additionalDepositPercent,
WAD) + 1;  // MUTATION: rebased
             SafeTransferLib.safeTransferFrom(loanToken, seller, address(this),
amountFromSeller);
         }
```

❌ **MidnightSupplyVaultSharesCallback #22 — Supplying the vault shares on behalf of the loan token address instead of the seller credits an unrelated third account's Midnight position, so a bystander's collateral rises and the rule that a supply take never touches a bystander's position produces a counterexample.**

- **Mutant:** `certora/mutations/MidnightSupplyVaultSharesCallback/22.sol`
- **Caught by:** `bystanderUntouched`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/bystanderUntouched.conf --rule bystanderUntouched`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightSupplyVaultSharesCallback 22`

```
--- a/src/callbacks/MidnightSupplyVaultSharesCallback.sol
+++ b/src/callbacks/MidnightSupplyVaultSharesCallback.sol
@@ -78,7 +78,7 @@
         uint256 shares = IERC4626(vault).deposit(totalDeposit, address(this));

         IERC20(vault).forceApprove(address(MORPHO_MIDNIGHT), shares);
-        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, seller);
+        MORPHO_MIDNIGHT.supplyCollateral(market, callbackData.collateralIndex, shares, loanToken); // MUTATION: onBehalf redirected from the seller to the loanToken address (an unrelated third account)

         emit VaultSharesSupplied(seller, marketId, vault, sellerAssets, totalDeposit, shares);
```

`MidnightWithdrawVaultSharesCallback` — `src/callbacks/MidnightWithdrawVaultSharesCallback.sol`

❌ **MidnightWithdrawVaultSharesCallback #1 — off-by-one over-withdraw: sharesToWithdraw + 1 leaves a residual vault share in the callback**

- **Mutant:** `certora/mutations/MidnightWithdrawVaultSharesCallback/1.sol`
- **Caught by:** `takeLeavesVaultShareBalanceUnchanged` (CB-VAULT-WD-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/takeLeavesVaultShareBalanceUnchanged.conf --rule takeLeavesVaultShareBalanceUnchanged`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightWithdrawVaultSharesCallback 1`

```
--- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
@@ -55,7 +55,7 @@

         uint256 sharesToWithdraw =
IERC4626(callbackData.vault).previewWithdraw(buyerAssets);

-        MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex,
sharesToWithdraw, buyer, address(this));
+        MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex,
sharesToWithdraw + 1, buyer, address(this));  // MUTATION: off-by-one over-withdraw:
sharesToWithdraw + 1 leaves a

         IERC4626(callbackData.vault).withdraw(buyerAssets, address(this), address(this));
```

❌ MidnightWithdrawVaultSharesCallback #2 — leave 1 wei allowance to Midnight (approve buyerAssets+1)

- **Mutant:** `certora/mutations/MidnightWithdrawVaultSharesCallback/2.sol`
- **Caught by:** `callbackHoldsZeroAllowance` (CB-DUST-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/callbackHoldsZeroAllowance.conf --rule callbackHoldsZeroAllowance`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightWithdrawVaultSharesCallback 2`

```
--- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
@@ -59,7 +59,7 @@

         IERC4626(callbackData.vault).withdraw(buyerAssets, address(this), address(this));

-        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
+        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets + 1);  // MUTATION:
leave 1 wei allowance to Midnight (approve buyerAssets+

         emit VaultSharesWithdrawn(buyer, marketId, callbackData.vault, buyerAssets,
sharesToWithdraw);
```

❌ MidnightWithdrawVaultSharesCallback #5 — Withdrawing 0 collateral instead of the computed amount prevents collateral reduction; the assertion that collateral < collateralBefore fails.

- **Mutant:** `certora/mutations/MidnightWithdrawVaultSharesCallback/5.sol`
- **Caught by:** `takeCanDropCollateralOnNarrowedMarket` (CB-VAULT-WD-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/takeCanDropCollateralOnNarrowedMarket.conf --rule takeCanDropCollateralOnNarrowedMarket`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightWithdrawVaultSharesCallback 5`

```
--- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
@@ -55,7 +55,7 @@

         uint256 sharesToWithdraw =
IERC4626(callbackData.vault).previewWithdraw(buyerAssets);

-        MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex,
sharesToWithdraw, buyer, address(this));
+        MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex, 0, buyer,
address(this));  // MUTATION: Withdrawing 0 collateral instead of the computed amount

         IERC4626(callbackData.vault).withdraw(buyerAssets, address(this), address(this));
```

❌ MidnightWithdrawVaultSharesCallback #6 — Withdrawing only half the assets leaves the callback holding half of the vault shares; the assertion that callback balance == 0 fails.

- **Mutant:** `certora/mutations/MidnightWithdrawVaultSharesCallback/6.sol`
- **Caught by:** `callbackNeverHoldsTokens` (CB-DUST-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/callbackNeverHoldsTokens.conf --rule callbackNeverHoldsTokens`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightWithdrawVaultSharesCallback 6`

```
--- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
@@ -57,7 +57,7 @@

         MORPHO_MIDNIGHT.withdrawCollateral(market, callbackData.collateralIndex,
sharesToWithdraw, buyer, address(this));

-        IERC4626(callbackData.vault).withdraw(buyerAssets, address(this), address(this));
+        IERC4626(callbackData.vault).withdraw(buyerAssets / 2, address(this),
address(this));  // MUTATION: Withdrawing only half the assets leaves the callback ho

         IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
```

❌ **MidnightWithdrawVaultSharesCallback #8 — The callback approves Midnight for zero loanToken instead of buyerAssets, so Midnight cannot pull the funds and take() reverts, leaving the rule unable to witness a successful withdraw fill.**

- **Mutant:** `certora/mutations/MidnightWithdrawVaultSharesCallback/8.sol`
- **Caught by:** `takeLeavesVaultShareBalanceUnchanged__satisfy` (CB-VAULT-WD-1)
- **Channel:** satisfy-twin — the mutation makes `take()` (or its antecedent branch) revert, so the witness becomes UNSAT (**VIOLATED** = mutant caught); the clean-`src/` witness is proven **SUCCESS**.
- **Run without the mutation (clean `src/` → witness FOUND, `SUCCESS`):** `certoraRun certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/debug_satisfy/takeLeavesVaultShareBalanceUnchanged.conf --rule takeLeavesVaultShareBalanceUnchanged__satisfy`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MidnightWithdrawVaultSharesCallback 8`

```
--- a/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
+++ b/src/callbacks/MidnightWithdrawVaultSharesCallback.sol
@@ -59,7 +59,7 @@

         IERC4626(callbackData.vault).withdraw(buyerAssets, address(this), address(this));

-        IERC20(market.loanToken).forceApprove(msg.sender, buyerAssets);
+        IERC20(market.loanToken).forceApprove(msg.sender, 0); // MUTATION: approve 0 -> Midnight cannot pull loanToken -> take reverts

         emit VaultSharesWithdrawn(buyer, marketId, callbackData.vault, buyerAssets, sharesToWithdraw);
```

`MigrationRatifier` — `src/ratifiers/MigrationRatifier.sol`

❌ **MigrationRatifier #2 — Comment out the assignment so setParams does not actually write the tuple, breaking the storage fidelity invariant**

- **Mutant:** `certora/mutations/MigrationRatifier/2.sol`
- **Caught by:** `setParamsWritesTupleAndLeavesOthers` (ORCH-15, REG-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule setParamsWritesTupleAndLeavesOthers`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 2`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -88,7 +88,7 @@
         if (msg.sender != onBehalf && !MORPHO_MIDNIGHT.isAuthorized(onBehalf, msg.sender))
{
             revert Unauthorized();
         }
-        userParams[onBehalf][callback][sourceTenorMarketId][targetTenorMarketId] = params;
+        // userParams[onBehalf][callback][sourceTenorMarketId][targetTenorMarketId] =
params;  // MUTATION: rebased
         emit ParamsSet(onBehalf, callback, sourceTenorMarketId, targetTenorMarketId,
params);
     }
```

❌ MigrationRatifier #3 — Comment out the delete statement so clearParams does not actually zero the tuple, breaking the clear invariant

- **Mutant:** `certora/mutations/MigrationRatifier/3.sol`
- **Caught by:** `clearParamsZeroesTupleAndLeavesOthers` (REG-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule clearParamsZeroesTupleAndLeavesOthers`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 3`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -99,7 +99,7 @@
         if (msg.sender != onBehalf && !MORPHO_MIDNIGHT.isAuthorized(onBehalf, msg.sender))
{
             revert Unauthorized();
         }
-        delete userParams[onBehalf][callback][sourceTenorMarketId][targetTenorMarketId];
+        // delete userParams[onBehalf][callback][sourceTenorMarketId][targetTenorMarketId];
// MUTATION: rebased
         emit ParamsCleared(onBehalf, callback, sourceTenorMarketId, targetTenorMarketId);
     }
```

❌ MigrationRatifier #5 — ratifierData market-match guard flipped != to == : accepts a source-market mismatch

- **Mutant:** `certora/mutations/MigrationRatifier/5.sol`
- **Caught by:** `ratifierDataMustMatchCallbackMarkets` (DEFAULT-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule ratifierDataMustMatchCallbackMarkets`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 5`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -133,7 +133,7 @@
         bytes32 callbackSourceMarketId,
         bytes32 callbackTargetMarketId
     ) internal pure override {
-        if (callbackSourceMarketId != sourceTenorMarketId || callbackTargetMarketId !=
targetTenorMarketId) {
+        if (callbackSourceMarketId == sourceTenorMarketId || callbackTargetMarketId !=
targetTenorMarketId) {  // MUTATION: rebased
             revert InvalidCallbackData();
         }
     }
```

❌ **MigrationRatifier #8 — auth guard short-circuited to false: caller can change params on behalf of an unauthorizing owner**

- **Mutant:** `certora/mutations/MigrationRatifier/8.sol`
- **Caught by:** `userParamsChangeRequiresAuthorization` (REG-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/access_control.conf --rule userParamsChangeRequiresAuthorization`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 8`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -85,7 +85,7 @@
         bytes32 targetTenorMarketId,
         UserMigrationParams calldata params
     ) external override {
-        if (msg.sender != onBehalf && !MORPHO_MIDNIGHT.isAuthorized(onBehalf, msg.sender))
{
+        if (false && msg.sender != onBehalf && !MORPHO_MIDNIGHT.isAuthorized(onBehalf,
msg.sender)) {  // MUTATION: auth guard short-circuited to false: caller can change
             revert Unauthorized();
         }
         userParams[onBehalf][callback][sourceTenorMarketId][targetTenorMarketId] = params;
```

❌ **MigrationRatifier #9 — invalid-length guard != -> == : accepts non-64-byte ratifierData**

- **Mutant:** `certora/mutations/MigrationRatifier/9.sol`
- **Caught by:** `invalidRatifierDataLengthReverts`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule invalidRatifierDataLengthReverts`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 9`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -115,7 +115,7 @@
         virtual
         returns (bytes32)
     {
-        if (ratifierData.length != 64) revert InvalidRatifierData();
+        if (ratifierData.length == 64) revert InvalidRatifierData();  // MUTATION: rebased
         (bytes32 src, bytes32 tgt) = abi.decode(ratifierData, (bytes32, bytes32));
         if (offer.receiverIfMakerIsSeller != (offer.buy ? address(0) : offer.callback))
revert InvalidReceiver();
         if ((offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER) revert
InvalidGroup();
```

❌ MigrationRatifier #10 — pinned-receiver guard != -> == : accepts unpinned receiver

- **Mutant:** `certora/mutations/MigrationRatifier/10.sol`
- **Caught by:** `makerReceiverMustBePinned`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule makerReceiverMustBePinned`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 10`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -117,7 +117,7 @@
     {
         if (ratifierData.length != 64) revert InvalidRatifierData();
         (bytes32 src, bytes32 tgt) = abi.decode(ratifierData, (bytes32, bytes32));
-        if (offer.receiverIfMakerIsSeller != (offer.buy ? address(0) : offer.callback))
revert InvalidReceiver();
+        if (offer.receiverIfMakerIsSeller == (offer.buy ? address(0) : offer.callback))
revert InvalidReceiver();  // MUTATION: rebased
         if ((offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER) revert
InvalidGroup();
         UserMigrationParams memory params = userParams[offer.maker][offer.callback][src]
[tgt];
         _ratify(offer.maker, taker, offer.callback, offer.callbackData, offer, src, tgt,
params);
```

❌ MigrationRatifier #11 — group-namespace guard != -> == : accepts out-of-namespace group

- **Mutant:** `certora/mutations/MigrationRatifier/11.sol`
- **Caught by:** `migrationGroupNamespaceEnforced`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/revert.conf --rule migrationGroupNamespaceEnforced`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 11`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -118,7 +118,7 @@
         if (ratifierData.length != 64) revert InvalidRatifierData();
         (bytes32 src, bytes32 tgt) = abi.decode(ratifierData, (bytes32, bytes32));
         if (offer.receiverIfMakerIsSeller != (offer.buy ? address(0) : offer.callback))
revert InvalidReceiver();
-        if ((offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER) revert
InvalidGroup();
+        if ((offer.group & MIGRATION_GROUP_HEADER_MASK) == MIGRATION_GROUP_HEADER) revert
InvalidGroup();  // MUTATION: rebased
         UserMigrationParams memory params = userParams[offer.maker][offer.callback][src]
[tgt];
         _ratify(offer.maker, taker, offer.callback, offer.callbackData, offer, src, tgt,
params);
         return CALLBACK_SUCCESS;
```

❌ MigrationRatifier #12 — isRatified key swap [src][tgt]->[tgt][src]: reads a non-addressed tuple

- **Mutant:** `certora/mutations/MigrationRatifier/12.sol`
- **Caught by:** `isRatifiedReadsOnlyAddressedParams`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule isRatifiedReadsOnlyAddressedParams`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 12`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -119,7 +119,7 @@
         (bytes32 src, bytes32 tgt) = abi.decode(ratifierData, (bytes32, bytes32));
         if (offer.receiverIfMakerIsSeller != (offer.buy ? address(0) : offer.callback))
revert InvalidReceiver();
         if ((offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER) revert
InvalidGroup();
-        UserMigrationParams memory params = userParams[offer.maker][offer.callback][src]
[tgt];
+        UserMigrationParams memory params = userParams[offer.maker][offer.callback][tgt]
[src];  // MUTATION: rebased
         _ratify(offer.maker, taker, offer.callback, offer.callbackData, offer, src, tgt,
params);
         return CALLBACK_SUCCESS;
     }
```

❌ MigrationRatifier #13 — Replace the success-token return with bytes32(0) so an accepting isRatified no longer produces the CALLBACK_SUCCESS value take() requires

- **Mutant:** `certora/mutations/MigrationRatifier/13.sol`
- **Caught by:** `isRatifiedReturnsCallbackSuccess`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun`

`certora/confs/ratifier/unit.conf --rule isRatifiedReturnsCallbackSuccess`

- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh MigrationRatifier 13`

```
--- a/src/ratifiers/MigrationRatifier.sol
+++ b/src/ratifiers/MigrationRatifier.sol
@@ -121,7 +121,7 @@
         if ((offer.group & MIGRATION_GROUP_HEADER_MASK) != MIGRATION_GROUP_HEADER) revert
InvalidGroup();
         UserMigrationParams memory params = userParams[offer.maker][offer.callback][src]
[tgt];
         _ratify(offer.maker, taker, offer.callback, offer.callbackData, offer, src, tgt,
params);
-        return CALLBACK_SUCCESS;
+        return bytes32(0);   // MUTATION: accepting path no longer returns CALLBACK_SUCCESS
     }

     /// @dev Requires the maker-declared route to equal the callback-derived markets. The
params lookup is already
```

`PriceLib` — `src/libraries/PriceLib.sol`

❌ PriceLib #1 — swapped buyer/seller rounding (mulDivDown<->mulDivUp)

- **Mutant:** `certora/mutations/PriceLib/1.sol`
- **Caught by:** `priceFollowsZeroCouponFormula` (PRICE-1) · `priceRoundsInProtectedUserFavor` (PRICE-2)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule priceFollowsZeroCouponFormula priceRoundsInProtectedUserFavor`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 1`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -24,7 +24,7 @@
     /// @return price The unit price (assets per unit), in WAD.
     function computePrice(bool isBuy, uint256 ratePerSecond, uint256 durationSeconds)
internal pure returns (uint256) {
         uint256 denominator = WAD + ratePerSecond * durationSeconds;
-        return isBuy ? WAD.mulDivDown(WAD, denominator) : WAD.mulDivUp(WAD, denominator);
+        return isBuy ? WAD.mulDivUp(WAD, denominator) : WAD.mulDivDown(WAD, denominator);
// MUTATION: rebased
     }

     /// @dev Returns the effective rate for the position side: max(policyRate, limitRate)
for lenders (isBuy == true,
```

❌ PriceLib #2 — denominator rate*dur -> rate+dur (formula broken, rounding intact)

- **Mutant:** `certora/mutations/PriceLib/2.sol`
- **Caught by:** `priceFollowsZeroCouponFormula` (PRICE-1)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule priceFollowsZeroCouponFormula`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 2`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -23,7 +23,7 @@
     /// @param durationSeconds The duration in seconds.
     /// @return price The unit price (assets per unit), in WAD.
     function computePrice(bool isBuy, uint256 ratePerSecond, uint256 durationSeconds)
internal pure returns (uint256) {
-        uint256 denominator = WAD + ratePerSecond * durationSeconds;
+        uint256 denominator = WAD + ratePerSecond + durationSeconds;  // MUTATION: rebased
         return isBuy ? WAD.mulDivDown(WAD, denominator) : WAD.mulDivUp(WAD, denominator);
     }
```

❌ PriceLib #3 — computeEffectiveRate buy-side max->min (> to <)

- **Mutant:** `certora/mutations/PriceLib/3.sol`
- **Caught by:** `effectiveRateSelectsTighterBound` (PRICE-3)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule effectiveRateSelectsTighterBound`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 3`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -35,7 +35,7 @@
     function computeEffectiveRate(bool isBuy, uint256 policyRate, uint256 limitRate)
internal pure returns (uint256) {
         return
             isBuy
-                ? (policyRate > limitRate ? policyRate : limitRate)
+                ? (policyRate < limitRate ? policyRate : limitRate)  // MUTATION: rebased
                 : (policyRate < limitRate ? policyRate : limitRate);
     }
```

❌ **PriceLib #4 — satisfiesRateLimit lender <= to >= [same diff as PriceLib#9, re-proven under highlevel.conf]**

- **Mutant:** `certora/mutations/PriceLib/4.sol`
- **Caught by:** `satisfiesRateLimitComparisonDirection` (PRICE-4) · `satisfiesRateLimitMonotoneInLenderLimit`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule satisfiesRateLimitComparisonDirection satisfiesRateLimitMonotoneInLenderLimit`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 4`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -63,7 +63,7 @@
         uint256 effectiveRate = computeEffectiveRate(isBuy, policyRate, limitRate);
         uint256 price = computePrice(isBuy, effectiveRate, duration);
         if (isBuy) {
-            return assets * WAD <= units * price;
+            return assets * WAD >= units * price;  // MUTATION: rebased
         } else {
             return assets * WAD >= units * price;
         }
```

❌ **PriceLib #7 — borrower compare >= -> <= (copy-paste of lender branch): inverts limit-monotonicity [same diff as PriceLib#8, re-proven under highlevel.conf]**

- **Mutant:** `certora/mutations/PriceLib/7.sol`
- **Caught by:** `satisfiesRateLimitMonotoneInBorrowerLimit` · `satisfiesRateLimitComparisonDirection` (PRICE-4)
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule satisfiesRateLimitMonotoneInBorrowerLimit satisfiesRateLimitComparisonDirection`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 7`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -65,7 +65,7 @@
         if (isBuy) {
             return assets * WAD <= units * price;
         } else {
-            return assets * WAD >= units * price;
+            return assets * WAD <= units * price;  // MUTATION: rebased
         }
     }
 }
```

❌ **PriceLib #8 — borrower rate-limit compare >= -> <= : breaks gate-vs-reconstruction binding [same diff as PriceLib#7, re-proven under unit.conf]**

- **Mutant:** `certora/mutations/PriceLib/8.sol`
- **Caught by:** `higherFeeOnlyTightensBorrowerRateGate`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule higherFeeOnlyTightensBorrowerRateGate`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 8`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -65,7 +65,7 @@
         if (isBuy) {
             return assets * WAD <= units * price;
         } else {
-            return assets * WAD >= units * price;
+            return assets * WAD <= units * price;  // MUTATION: rebased
         }
     }
 }
```

❌ **PriceLib #9 — lender rate-limit compare <= -> >= : breaks gate-vs-reconstruction binding [same diff as PriceLib#4, re-proven under unit.conf]**

- **Mutant:** `certora/mutations/PriceLib/9.sol`
- **Caught by:** `higherFeeOnlyTightensLenderRateGate`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/highlevel.conf --rule higherFeeOnlyTightensLenderRateGate`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh PriceLib 9`

```
--- a/src/libraries/PriceLib.sol
+++ b/src/libraries/PriceLib.sol
@@ -63,7 +63,7 @@
         uint256 effectiveRate = computeEffectiveRate(isBuy, policyRate, limitRate);
         uint256 price = computePrice(isBuy, effectiveRate, duration);
         if (isBuy) {
-            return assets * WAD <= units * price;
+            return assets * WAD >= units * price;  // MUTATION: rebased
         } else {
             return assets * WAD >= units * price;
         }
```

`RouterLib` — `src/libraries/RouterLib.sol`

❌ RouterLib #1 — net-seller min -> max : breaks fee-monotone-decreasing

- **Mutant:** `certora/mutations/RouterLib/1.sol`
- **Caught by:** `netSellerPriceMonotoneInFee`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule netSellerPriceMonotoneInFee`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh RouterLib 1`

```
--- a/src/libraries/RouterLib.sol
+++ b/src/libraries/RouterLib.sol
@@ -74,6 +74,6 @@
         uint256 midnightPrice = offerPrice > settlementFee ? offerPrice - settlementFee :
0;
         if (feeRate == 0) return midnightPrice;
         uint256 tenorPrice = CallbackLib.sellerEffectivePrice(offerPrice, feeRate);
-        return midnightPrice < tenorPrice ? midnightPrice : tenorPrice;
+        return midnightPrice > tenorPrice ? midnightPrice : tenorPrice;  // MUTATION:
rebased
     }
 }
```

❌ RouterLib #2 — net-buyer max -> min : breaks fee-monotone-increasing

- **Mutant:** `certora/mutations/RouterLib/2.sol`
- **Caught by:** `netBuyerPriceMonotoneInFee`
- **Run without the mutation (clean `src/` → `VERIFIED`):** `certoraRun certora/confs/ratifier/unit.conf --rule netBuyerPriceMonotoneInFee`
- **Run with the mutation (rule `VIOLATED` = mutant caught):** `./certora/mutations/run_mutation.sh RouterLib 2`

```
--- a/src/libraries/RouterLib.sol
+++ b/src/libraries/RouterLib.sol
@@ -55,7 +55,7 @@
         uint256 midnightPrice = offerPrice + settlementFee;
         if (feeRate == 0) return midnightPrice;
         uint256 tenorPrice = CallbackLib.buyerEffectivePrice(offerPrice, feeRate);
-        return midnightPrice > tenorPrice ? midnightPrice : tenorPrice;
+        return midnightPrice < tenorPrice ? midnightPrice : tenorPrice;  // MUTATION:
rebased
     }

     /// @dev Returns the net per-unit price the seller-as-taker receives onchain, used to
invert remainingBudget to
```

Setup and Execution

The Certora Prover runs remotely (Certora's cloud) or locally (built from source); both modes share setup steps 1-5.

Common Setup (Steps 1-5)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup tutorial.

All `certoraRun` commands are executed from the repository root at the audit commit, with the git submodules initialized first (the scenes compile sources out of `lib/midnight`, `lib/morpho-blue`, `lib/vault-v2`, and their transitive dependencies):

```
git submodule update --init --recursive
```

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI. To match the prover version used for this audit, pin it explicitly

```
pipx install certora-cli==8.16.2
```

4. Install solc-select and the Solidity compiler versions required by the project (0.8.34 for the callbacks and the Midnight model, 0.8.19 for the Morpho Blue model, 0.8.28 for the Morpho Vault V2 target used by the four vault-touching callbacks)

```
pipx install solc-select
solc-select install 0.8.34
solc-select install 0.8.28
solc-select install 0.8.19
solc-select use 0.8.34
```

5. Create versioned solc symlinks for Certora. Configuration files reference the compiler as `solc0.8.34` / `solc0.8.28` / `solc0.8.19` (without dashes), but solc-select only creates a generic `solc` binary:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.34/solc-0.8.34 ~/.local/bin/solc0.8.34
ln -sf ~/.solc-select/artifacts/solc-0.8.28/solc-0.8.28 ~/.local/bin/solc0.8.28
ln -sf ~/.solc-select/artifacts/solc-0.8.19/solc-0.8.19 ~/.local/bin/solc0.8.19
```

Verify `~/.local/bin` is in your `PATH`. If not, add it:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Remote Execution

Set up a Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

> **Note:** If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:
>
> ```
> certoraRun
> certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendCanRaiseCredit.conf
> --server production
> ```

Local Execution

Follow the full build instructions in the CertoraProver repository (v8.16.2).

1. Install prerequisites

```
# JDK 19+
sudo apt install openjdk-21-jdk

# SMT solvers (z3 and cvc5 are required, others are optional)
# Download binaries and place them in PATH:
#   z3:   https://github.com/Z3Prover/z3/releases
#   cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Graphviz (optional, for visual reports)
sudo apt install graphviz
```

2. Set up build output directory

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"
```

3. Clone and build

```
git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.16.2
./gradlew assemble
```

4. Verify installation with test example

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Verification

Every property has its own configuration file under `certora/confs/callbacks/<Callback>/<ruleName>.conf` (flat, single make-on-behalf scenario). The listings below show the callback-specific confs; the applicable shared safety-rule confs live in the same directory and are re-verified under each callback's setup. Non-vacuity is covered by the `debug_satisfy` channel plus advanced-sanity runs; the `debug_advanced*` subdirectories are sanity tooling, not part of the proof suite.

BorrowBlueToMidnightCallback (BBM)

```
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/borrowerFeeBoundedByInterestShare.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/clearingOldDebtAlsoEmptiesOldCollateral
.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/fullCollateralMigrationClearsAllOldDebt
.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationCanFullyCloseOldPosition.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationCanMoveCollateralBlueToMidnigh
t.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationConservesMigratedCollateral.co
nf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationOnlyAddsNewMidnightCollateral.
conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationOnlyReducesOldBlueDebt.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationOnlyWithdrawsOldBlueCollateral
.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/migrationReducesOldDebtOnAtMostOneMarke
t.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/receiverNotCallbackReverts.conf
certoraRun
certora/confs/callbacks/BorrowBlueToMidnightCallback/sourceLoanTokenMismatchReverts.conf
certoraRun certora/confs/callbacks/BorrowBlueToMidnightCallback/tickFeeVanishesAtPar.conf
```

BorrowMidnightToBlueCallback (BMB)

```
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanFullyCloseOldPosition.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanMoveCollateralMidnightToBlu
e.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCanOpenNewBlueDebt.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationCannotDepositMoreCollateralTha
nWithdrawn.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationFinalFillTransfersAllOldMidnig
htCollateral.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationOnlyAddsNewBlueCollateral.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationOnlyOpensNewBlueDebt.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationOnlyWithdrawsOldMidnightCollat
eral.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/migrationReducesOldDebtOnAtMostOneMarke
t.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/oldMidnightDebtAndNewBlueDebtMoveTogeth
er.conf
certoraRun
certora/confs/callbacks/BorrowMidnightToBlueCallback/percentageFeeRateAboveCapReverts.conf
```

BorrowMidnightRenewalCallback (BMR)

```
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/borrowerFeeBoundedByInterestShare.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/callbackRevertsForSameSourceMarket.con
f
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/receiverNotCallbackReverts.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalAddsDebtOnAtMostOneMarket.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCallbackNeverPullsExternalLoanT
oken.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCanFullyCloseOldPosition.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCanMigrateCollateralBetweenMark
ets.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCanMoveDebtBetweenMarkets.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCannotAddCollateralWhenReducing
Debt.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCannotMoveMoreCollateralThanWit
hdrawn.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalCannotRemoveCollateralWhenOpeni
ngDebt.conf
certoraRun
certora/confs/callbacks/BorrowMidnightRenewalCallback/renewalReducesDebtOnAtMostOneMarket.co
nf
certoraRun certora/confs/callbacks/BorrowMidnightRenewalCallback/tickFeeVanishesAtPar.conf
```

LendVaultToMidnightCallback (LVM)

```
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/lenderFeeBoundedByInterestShare.conf
certoraRun certora/confs/callbacks/LendVaultToMidnightCallback/tickFeeVanishesAtPar.conf
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/vaultAssetMismatchReverts.conf
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendCanRaiseCredit.conf
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendLeavesCollateralUnchanged
.conf
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendNeverTouchesUnrelatedUser
.conf
certoraRun
certora/confs/callbacks/LendVaultToMidnightCallback/vaultFundedLendOnlyMovesLoanToken.conf
```

LendMidnightToVaultCallback (LMV)

```
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/receiverNotCallbackReverts.conf
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/vaultAssetMismatchReverts.conf
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitCanFullyCloseCredit.conf
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitConservesMidnightBalanceMinusFe
e.conf
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitLeavesCollateralUnchanged.conf
certoraRun
certora/confs/callbacks/LendMidnightToVaultCallback/vaultExitNeverTouchesUnrelatedUser.conf
```

LendMidnightRenewalCallback (LMR)

```
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/callbackRevertsForSameSourceMarket.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalAddsCreditOnAtMostOneMarket.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalCallbackNeverPullsExternalLoanTok
en.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalCanFullyCloseOldCredit.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalCanMoveCreditWithPositiveFee.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalNeverTouchesUnrelatedLenderCredit
.conf
certoraRun
certora/confs/callbacks/LendMidnightRenewalCallback/renewalReducesCreditOnAtMostOneMarket.co
nf
certoraRun certora/confs/callbacks/LendMidnightRenewalCallback/tickFeeVanishesAtPar.conf
```

MidnightSupplyCollateralCallback (MSC)

```
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/supplyMonotoneCollateral.conf
certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/bystanderUntouched.conf
certoraRun certora/confs/callbacks/MidnightSupplyCollateralCallback/proRataUpperBound.conf
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/borrowCapacityUsageWithinCap.conf
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/maxBorrowCapacityUsageFillReachable
.conf
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/supplyCanRaiseCollateral.conf
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/collateralLengthMismatchReverts.con
f
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/offerSellerAssetsZeroReverts.conf
certoraRun
certora/confs/callbacks/MidnightSupplyCollateralCallback/receiverIsCallbackReverts.conf
```

MidnightSupplyVaultSharesCallback (MSV)

```
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyMonotoneCollateral.conf
certoraRun certora/confs/callbacks/MidnightSupplyVaultSharesCallback/bystanderUntouched.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/onlyVaultSlotReceivesSupply.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/suppliedSharesMatchMintedShares.co
nf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultShareBeneficiaryIsSeller.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/supplyCanRaiseVaultCollateral.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/noExtraPullWhenPercentZero.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultAssetMismatchReverts.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/vaultNotAtIndexReverts.conf
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/extraPullMatchesPercentFormula.con
f
certoraRun
certora/confs/callbacks/MidnightSupplyVaultSharesCallback/receiverNotCallbackReverts.conf
```

MidnightWithdrawVaultSharesCallback (MWV)

```
certoraRun
certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/takeCanDropCollateralOnNarrowedM
arket.conf
certoraRun
certora/confs/callbacks/MidnightWithdrawVaultSharesCallback/takeLeavesVaultShareBalanceUncha
nged.conf
```

> The heavier coupling and source-funding rules (`renewalCallbackNeverPullsExternalLoanToken`, `renewalCannotMoveMoreCollateralThanWithdrawn`, `renewalReducesCreditOnAtMostOneMarket`, and the `_one`-regime `oldMidnightDebtAndNewBlueDebtMoveTogether`) use a solver portfolio (`prover_args` with multiple solvers and an increased split depth) to push through the 4-hour SMT timeout.

Migration Ratifier

The ratifier runs as six per-category configurations; the advanced-sanity variants in `debug_advanced/` re-run the same rules under `rule_sanity: advanced` (`override_base_config` → the base conf):

```
# production (basic sanity)
certoraRun certora/confs/ratifier/valid_state.conf
certoraRun certora/confs/ratifier/unit.conf
certoraRun certora/confs/ratifier/revert.conf
certoraRun certora/confs/ratifier/highlevel.conf
certoraRun certora/confs/ratifier/access_control.conf
certoraRun certora/confs/ratifier/reachability.conf
# advanced-sanity variants
certoraRun certora/confs/ratifier/debug_advanced/valid_state.conf
certoraRun certora/confs/ratifier/debug_advanced/unit.conf
certoraRun certora/confs/ratifier/debug_advanced/revert.conf
certoraRun certora/confs/ratifier/debug_advanced/highlevel.conf
certoraRun certora/confs/ratifier/debug_advanced/access_control.conf
certoraRun certora/confs/ratifier/debug_advanced/reachability.conf
```

Resources

- Certora Tutorials -- Official Certora documentation and guided tutorials
- AlexZoid FV Resources -- Curated collection of formal verification resources, examples, and references
- Updraft Assembly & Formal Verification Course -- Comprehensive video course covering assembly and formal verification from the ground up
- RareSkills Certora Book -- Structured tutorial covering CVL syntax, patterns, and common pitfalls